

Laplace-domain temporal compression for deep-learning reduced-order models

Keyvan Attarian^{1,3} and Amine Ammar^{2,3}

¹École polytechnique, Institut Polytechnique de Paris

²LAMPA Laboratory & ESI Group Chair, Arts et Metiers Institute of Technology

³DesCartes Program, WP8, CNRS@CREATE

Printed July 23, 2026

Abstract

Deep-learning reduced-order models (DL-ROMs) keep one latent code per time step, so their output size and cost scale with the number of time steps N_t . We compress the time axis with the Laplace transform, predicting $K \ll N_t$ spectral coefficients in one shot and recovering the trajectory by a differentiable regularised inversion, applied either in the physical domain (Spatial Laplace AE, SLAE) or in the latent space (Latent Laplace AE, LLAE). A bias–amplification bound makes the inversion triplet (s_k, α_t, λ) selectable before any training. On a parametric transient combustion problem ($N_t = 150$, $K = 16$), LLAE reaches 3.84% median relative L^2 error and SLAE 7.46%, against 8.92% for a linear POD surrogate and 3.80% for a Laplace-free DL-ROM ablation: in latent space, temporal compression costs nothing in accuracy. The gain is cost, unevenly. LLAE shrinks the surrogate alone, still decoding N_t frames; SLAE decodes only the K frequency frames and runs 2.0× faster than the DL-ROM on 3.9× less memory at identical output. Both also answer, from one checkpoint, at any temporal resolution above a sampling floor set by their poles, where the DL-ROM needs one training per grid. The offline rule, finally, adapts the contour to the problem instead of fixing it by convention, worth up to 1.85 pp on the deployed error and bringing SLAE to 5.04% at $K = 32$.

Contents

1	Introduction	2
2	Methodology	4
3	Results	17
4	Comparison across Cost and Resolution	38
5	Discussion	43
6	Conclusion	48
	References	49
	Appendices	52

1 Introduction

High-fidelity simulations of parametric transient partial differential equations (PDEs), such as those governing combustion, reactive transport, and atmospheric dispersion, rely on computationally expensive numerical solvers. In many-query contexts like optimisation, uncertainty quantification, real-time control, and digital twins, the complete spatiotemporal field $U(x, t; \mu)$ must be evaluated for thousands of parameter values μ [11, 27]. The prohibitive cost of these repeated evaluations motivates the development of surrogate models capable of reproducing the full dynamics in milliseconds. Applications such as air quality and pollution forecasting concretely illustrate this need: global physical models are accurate but unusable in real-time without approximation [15, 8].

Classical model order reduction (MOR) methods [18], such as Reduced Basis (RB) and Proper Orthogonal Decomposition (POD) combined with interpolation, construct a low-dimensional linear subspace by projecting high-fidelity solutions onto spatial modes. These approaches are theoretically well-founded and highly effective for smooth dynamics. However, they suffer from a *reconstruction floor* linked to the linearity of the basis: for problems with sharp transients, localized structures, or strongly non-linear dynamics (such as convection-diffusion-reaction equations), the number of required modes explodes. The transition toward *non-linear manifolds* becomes necessary as linear techniques struggle [9].

Beyond the spatial compression bottleneck, classical MOR methods also struggle with the temporal dimension. The treatment of time is either sequential (time-stepping on the projected ROM) or space-time (variational), both of which impose a computational cost that grows with the temporal horizon [12, 22, 32]. This dual bottleneck, spatial and temporal, motivates the approach developed in this paper. The *parameter-to-field* setting, predicting a full spatiotemporal field from a low-dimensional parameter vector $\mu \in \mathbb{R}^p$, is addressed by *DL-ROMs* (Deep Learning Reduced-Order Models) [23, 9, 10]: a convolutional autoencoder compresses the spatial field into a compact latent code, a surrogate network learns $\mu \mapsto z$, and the decoder reconstructs the full field at inference, yielding significant gains over linear ROMs. This differs from *field-to-field* operator learning (FNO [24], DeepONet [25, 26]), which takes an input function rather than a scalar parameter vector; the two settings are not directly comparable. Parameter-conditioned U-Nets [20] are a lighter DL-ROM variant for 2D problems, operating directly in the time domain. All existing DL-ROM approaches, however, share a temporal bottleneck: they retain the full temporal resolution of the trajectory, one latent vector per time step. Whether the surrogate takes time as an input coordinate and is queried once per step, $g(\mu, t) \mapsto z(t)$ [9], or propagates the latent sequentially, often autoregressively [31] at the cost of error accumulation, the temporal work, that is the number of network evaluations or the output size, scales with N_t . For long transients this makes the learning problem harder and inference heavier, however compact the spatial latent.

A natural path to address this bottleneck is to work in the *frequency domain*: compressing the time axis into $K \ll N_t$ spectral coefficients reduces the surrogate output dimension dramatically. The spectral intuition is well-established in the neural operator literature, FNO [24] parameterising its kernel in Fourier space, but the Fourier transform is poorly suited for transient signals: it assumes periodicity and exhibits spectral leakage for non-periodic dynamics. The *Laplace transform* $s = \gamma + i\omega$ generalises Fourier by adding a real decay axis γ : for a signal that rises, peaks and decays rather than repeats, it is the natural spectral object, encoding transients and exponential stability without assuming

periodicity. That axis constitutes an additional degree of freedom rather than a uniformly preferable contour. The favourable value of γ depends on the problem and on the stage at which the representation is evaluated, and we therefore treat it as a quantity to be selected rather than prescribed (Sections 2.4 and 3.5). On the present problem it proves nearly neutral, LLAE varying by 0.16 pp over $\gamma \in [0, 0.01]$ and the offline criterion returning $\sigma_k \approx 0.035$, a contour close to the imaginary axis. What we establish here is therefore the spectral compression of the time axis. The real abscissa remains a lever, and its distinctive payoff, extrapolation beyond the training horizon where Fourier has no counterpart, is left to Section 5.4. Each Laplace frequency field $\hat{U}(s_k)$ is a static spatial field, structurally simpler to compress than a full temporal sequence. The Laplace Neural Operator (LNO) [3] confirms this advantage in the field-to-field setting. On the linear reduction side, Reyes [34] and Guglielmi and Manucci [14] show that Laplace-domain POD bases are exponentially accurate for parametric evolution problems, yet these approaches remain fundamentally linear. The remaining gap is to combine the Laplace spectral compression with the non-linear expressivity of autoencoders within the DL-ROM framework, enabling *end-to-end* differentiable training thanks to the exact invertibility of the transform [13].

This paper fills that gap. We situate our work in the DL-ROM family and introduce the Laplace transform as a *temporal* compression stage on top of the standard AE-surrogate pipeline: instead of predicting N_t latent frames, the surrogate predicts $K \ll N_t$ spectral coefficients, from which the full trajectory is recovered by inversion. Here $U(t) \in \mathbb{R}^{N_x}$ is the discretised spatial field at time step t , N_x the number of spatial nodes, N_t the number of time steps, and K the number of Laplace frequencies. Two instantiations are studied, differing in where the transform is applied: the physical domain (Spatial Laplace AE, SLAE) or the latent space (Latent Laplace AE, LLAE). Both are compared against two baselines that each remove one ingredient, a linear POD baseline in the Laplace domain and a Laplace-free DL-ROM keeping the full temporal grid, taken in the head-indexed formulation whose surrogate carries one predictor per time step (Section 2.6.2). Our contributions are as follows:

- *Temporal compression via the Laplace transform (main contribution)*: we compress the temporal dimension of the surrogate target from N_t frames to $K \ll N_t$ spectral coefficients, making the surrogate’s size and output dimension independent of the temporal horizon. We state the trade once, here: temporal compression is an argument about cost and about what a trained model can afterwards be asked for, not about accuracy.
- *Where the transform is applied (SLAE vs. LLAE)*: we compare the two instantiations systematically and establish that their sole structural difference, the domain in which the transform acts, decides which cost axes the compression reaches (Section 4.2). It reaches the *complete* forward pass for SLAE, whose decoder sees only the K frequency frames, and the surrogate alone for LLAE, whose decoder still emits N_t physical frames; the parameter count is accordingly the axis least representative of the cost a user measures.
- *One trained model, many output time grids*: since the trajectory is materialised by a regularised linear solve assembled on whatever time nodes are requested, the predicted coefficients refer to no time grid, and a single trained model answers at *any* output *temporal* resolution above a sampling floor set by its own poles (Section 4.3), the spatial discretisation remaining fixed by the convolutional decoder, whereas a model predicting one latent per time step is tied to its training grid, or

at best pays one network evaluation per requested step. This weighs most in the many-query setting this paper targets, where a coarse parameter scan refined at a few points is served, above that floor, by one training instead of one per grid, and where, the workload being throughput- rather than latency-bound, the axis that limits it is peak memory (Section 4.3.2).

- *Designing the regularised inversion:* we treat the inversion as an object to be optimised rather than fixed by convention, and by that we mean the triplet (s_k, α_t, λ) , the poles together with the two regularisation weights, not the contour alone. For LLAE the triplet is learned end-to-end; for SLAE we derive a bias–amplification decomposition of the reconstruction error and optimise it offline, before any training, by minimising a proxy loss that trades regularisation bias against the amplification of the surrogate approximation error.
- *Latent post-compression, a negative result:* we compress the surrogate target further, along one mode by SVD and along two jointly by Tucker, and find that it improves neither accuracy nor latency (Sections 3.4.2–3.4.3): the decoder, not the predictor, is the binding constraint. Its value is diagnostic. A rank-4 basis over the frequency mode performs as well as a rank-8 one, so the $K = 16$ coefficients span a subspace of markedly lower dimension than their count. A tensor factorisation fitted blind to the physics recovers that redundancy, and therefore corroborates the premise of the transform from a direction independent of the argument that motivated it.

On the parametric transient combustion problem of Section 3.1 ($N_t = 150$, $K = 16$), LLAE reaches 3.84% median relative L^2 error and SLAE 7.46%. The two references stand at 8.92% for the linear POD surrogate and 3.80% for the Laplace-free DL-ROM: in the latent variant the temporal compression sits within 0.04 pp of the ablation it replaces, and in the physical-domain one it costs some three points. The gains lie on the cost and flexibility axes. SLAE runs $2.0\times$ faster than the DL-ROM on $3.9\times$ less memory at identical 150-step output, and a single Laplace checkpoint serves a range of output time grids for which the DL-ROM requires one training each.

The remainder of this paper is organised as follows. Section 2 describes the methodology, detailing the Laplace transform and its regularised inversion, the two autoencoder families, the surrogate architectures, the design of the inversion, the range of output grids a trained model can serve, and the two baselines, and closes with a synoptic table of the eleven pipelines studied. Section 3 introduces the governing problem and the dataset, then presents the numerical results, characterising each pipeline in turn. Section 4 compares them against one another on accuracy, inference cost and output resolution. Section 5 attributes the gains to the ingredients of the ablation, positions the work with respect to related literature, and states the limitations and perspectives; Section 6 concludes. The appendices detail the governing equations, the architectural hyperparameters and the derivations behind the contour optimisation.

2 Methodology

2.1 Laplace Transform and Inversion

Throughout this section, $u(t) \in \mathbb{R}$ denotes the field value at a single fixed spatial location (lowercase = one pixel); for the full spatial field $U(t) \in \mathbb{R}^{N_x}$ (uppercase = full grid of N_x nodes), all operations below are applied independently at each grid point. The forward Laplace transform is evaluated at discrete frequencies $s = \gamma + i\omega$ using a temporal

quadrature rule (e.g., trapezoidal rule with weights w_n):

$$\hat{u}(s) = \int_0^T u(t) e^{-st} dt \approx \Delta t \sum_{n=0}^{N_t-1} w_n u(t_n) e^{-st_n} \quad (1)$$

Since $u(t)$ is strictly real-valued, its Laplace transform satisfies the conjugate symmetry property $\hat{u}(\bar{s}) = \overline{\hat{u}(s)}$. Consequently, we restrict our numerical evaluation to frequencies with non-negative imaginary parts $\omega_k \geq 0$, which halves the forward evaluation cost while providing the full spectral information required for inversion.

Regularized Inversion (Tikhonov). The inversion is posed not on the K stored frequencies but on their *conjugate-symmetric completion*

$$\mathcal{S} = \{s_k\}_{k=1}^K \cup \{\bar{s}_k : \omega_k > 0\}, \quad K_{\text{full}} = |\mathcal{S}| = 2K - \#\{k : \omega_k = 0\}, \quad (2)$$

whose additional members follow from the stored half-spectrum through $\hat{u}(\bar{s}_k) = \overline{\hat{u}(s_k)}$. This doubles the sampling points for free: the conjugate coefficients require no additional forward evaluation, surrogate output or storage, yet enter the least-squares problem below as genuine extra constraints, so that the inversion is posed on $K_{\text{full}} \simeq 2K$ points while the network only ever predicts K . Let $\mathbf{F} \in \mathbb{C}^{K_{\text{full}} \times N_t}$ be the forward evaluation matrix assembled on $\mathcal{S}$, with $\mathbf{F}_{k,n} = \Delta t w_n e^{-s_k t_n}$, and let $\hat{\mathbf{u}} \in \mathbb{C}^{K_{\text{full}}}$, $\mathbf{u} \in \mathbb{R}^{N_t}$ be the corresponding vectors of Laplace coefficients and temporal values. Throughout this paper $\mathbf{F}$ and $\hat{\mathbf{u}}$ always denote these conjugate-extended objects. Taking advantage of the real-valued constraint on $\mathbf{u}$, the reconstruction is formulated as a regularised complex least-squares problem:

$$\mathbf{u}^* = \arg \min_{\mathbf{u} \in \mathbb{R}^{N_t}} \|\mathbf{F}\mathbf{u} - \hat{\mathbf{u}}\|_{\mathbb{C}^{K_{\text{full}}}}^2 + \alpha_t \|\mathbf{D}_t \mathbf{u}\|_2^2 + \lambda \|\mathbf{u}\|_2^2 \quad (3)$$

where $\mathbf{D}_t \in \mathbb{R}^{(N_t-1) \times N_t}$ is the first-difference operator penalizing temporal roughness, and $\lambda > 0$ is a Tikhonov ridge regularisation parameter. Regularisation is not optional here: numerical Laplace inversion is severely ill-posed, the forward operator damping precisely the components the inverse must recover [7], and (3) is the standard remedy for such discrete ill-posed problems [17]. The corresponding optimality condition (normal equations) yields a real symmetric linear system:

$$\left(\Re(\mathbf{F}^H \mathbf{F}) + \alpha_t \mathbf{D}_t^T \mathbf{D}_t + \lambda \mathbf{I} \right) \mathbf{u}^* = \Re(\mathbf{F}^H \hat{\mathbf{u}}) \quad (4)$$

Two properties of $\mathcal{S}$ are worth making explicit, since the half-spectrum $\{s_k\}_{k=1}^K$ alone would give a different problem. First, a real $\mathbf{u}$ produces conjugate residuals at s_k and $\bar{s}_k$, so restoring the conjugates weights every frequency with $\omega_k > 0$ twice in (3) relative to a purely real pole, the discrete counterpart of integrating along the whole Bromwich contour rather than over $\omega \geq 0$ only. Second, on $\mathcal{S}$ both $\mathbf{F}^H \mathbf{F}$ and $\mathbf{F}^H \hat{\mathbf{u}}$ are real, their imaginary parts cancelling pairwise between conjugate members, so the operator $\Re(\cdot)$ in (4) records the real-valuedness of $\mathbf{u}$ without discarding information. The $N_t \times N_t$ system (4) is assembled and factorised in double precision (at $K = 16$ the columns of $\mathbf{F}$ are nearly collinear and $\kappa(\mathbf{A})$ exceeds the reach of single precision) by Cholesky when $\mathbf{A}$ is reused across a batch of nodes and by a dense LU solve otherwise, at negligible cost. It is fully differentiable in (s_k, α_t, λ) , a property Section 2.4 exploits, and holds for any set of frequencies (s_k) , with no constraint on their distribution in the complex plane.

Choice of contour: truncated Bromwich. A key design choice is the set of evaluation frequencies $(s_1, \dots, s_K)$. We adopt throughout the *truncated Bromwich contour*: a

vertical line in the complex plane at $\text{Re}(s) = \gamma$, with imaginary parts equal to the first K discrete Fourier frequencies,

$$s_k = \gamma + i \frac{2\pi k}{N_t \Delta t}, \quad k = 0, \dots, K-1, \quad (5)$$

where $\gamma \geq 0$ is the *damping parameter* and $K \leq \lfloor N_t/2 \rfloor + 1$ controls the spectral truncation. We allow $\gamma = 0$, a purely imaginary contour, i.e. a truncated DFT, since it is one of the settings we report, and the optimisation of Section 2.4 may even return slightly negative abscissae; over a finite horizon the quadrature (1) is a finite sum and remains well defined for any $s \in \mathbb{C}$. With this choice, the forward sum (1) coincides with a truncated DFT of the exponentially damped sequence:

$$\hat{u}(s_k) = \text{DFT}\{u(t_n) \Delta t w_n e^{-\gamma t_n}\}_k, \quad k = 0, \dots, K-1, \quad (6)$$

the damped-sequence form on which the classical Fourier-series inversion methods rest [1], and it could therefore be evaluated by an FFT; with $K \leq 32$ and $N_t = 150$, our implementation simply forms the dense $K \times N_t$ matrix product, whose $O(KN_t)$ cost per node is negligible against the convolutional stages. Since $K < \lfloor N_t/2 \rfloor + 1$, the spectrum is incomplete and we rely on the Tikhonov inversion (3)–(4) in all cases. The Bromwich structure, however, forces all poles onto a common real part γ and a regular frequency grid, a strong constraint that may be suboptimal. Since the inversion accepts any set of frequencies, the poles can in principle be placed freely in the complex plane; this is the motivation for Section 2.4.

2.2 Autoencoders in the Laplace Domain

We investigate two autoencoder architectures that differ solely in *where* the Laplace transform is applied, as illustrated in Figure 1. In the *Spatial Laplace AE (SLAE)*, the transform is applied first in the physical domain, and a frequency-conditioned autoencoder then compresses each of the K resulting complex frequency frames independently. In the *Latent Laplace AE (LLAE)*, the order is reversed: a time-conditioned autoencoder first encodes the full temporal sequence into a compact latent trajectory, and the Laplace transform is subsequently applied within that low-dimensional space.

In both cases, an L_2 ridge penalty on the latent representations enforces a sufficiently regular structure, which makes the latent space easier for the surrogate to navigate (Section 2.3).

Both architectures build on the same convolutional encoder–decoder backbone, applied directly to the spatial field with no POD pre-reduction; this is the standard deep convolutional-autoencoder compression of the DL-ROM lineage [23, 9], as opposed to the POD-then-autoencoder pre-reduction of POD-DL-ROM [10]. On top of this shared backbone, both architectures use the same conditioning mechanism: a sinusoidal coordinate encoding combined with Feature-wise Linear Modulation (FiLM) [33]. The coordinate $\tau \in [0, 1]$ represents either the frequency ratio $f_k = k/(K-1)$ for SLAE, or the normalised time step t/T for LLAE.

Sinusoidal Coordinate Encoding. The coordinate $\tau \in [0, 1]$ is embedded via sinusoidal basis functions inspired by NeRF positional encoding [28]:

$$e(\tau) = \text{MLP}\left(\left[\sin(2^i \pi \tau), \cos(2^i \pi \tau)\right]_{i=0}^{L-1}\right), \quad L = 8. \quad (7)$$

A plain scalar τ would conflate nearby coordinates and fail to capture sharp transitions in either the frequency or time domains. The multi-scale sinusoidal basis explicitly represents τ at L different resolutions, providing the network with a rich, non-saturating signal that allows fine discrimination across the entire range.

FiLM Conditioning. To share encoder/decoder weights across all coordinates (frequencies or time steps) while adapting to each specific configuration, the FiLM mechanism is employed. At each convolutional layer ℓ , the coordinate embedding $e(\tau)$ is projected to a channel-wise gain $\alpha_\ell \in \mathbb{R}^{C_\ell}$ and bias $\delta_\ell \in \mathbb{R}^{C_\ell}$, modulating the activations element-wise:

$$\mathbf{x}_\ell \leftarrow (\mathbf{x}_\ell \odot (1 + \alpha_\ell(e(\tau)))) + \delta_\ell(e(\tau)). \quad (8)$$

The residual formulation $(1 + \alpha_\ell)$ initializes the modulation close to the identity, which stabilizes early training. All coordinates share the same convolutional weights, and the FiLM parameters act as a lightweight per-coordinate adapter at each layer, enabling a single autoencoder to process the full spectrum or temporal sequence without training separate model copies.

2.2.1 Spatial Laplace AE (SLAE)

SLAE first applies the Laplace transform at the pixel level, converting the N_t -frame temporal sequence $U(t)$ into K complex-valued spatial fields $\hat{U}(s_k)$. Each frequency frame, represented as a two-channel (real/imaginary) image, is independently compressed by a FiLM-conditioned convolutional encoder into a real latent vector $z(s_k) \in \mathbb{R}^{d_z}$:

$$U(t) \xrightarrow{\mathcal{L}_{\text{pixel-wise}}} \hat{U}(s_k) \xrightarrow{\text{Encoder}(f_k)} z(s_k) \xrightarrow{\text{Decoder}(f_k)} \tilde{U}(s_k) \xrightarrow{\mathcal{L}^{-1}} U_{\text{rec}}(t) \quad (9)$$

The decoder reconstructs $\hat{U}(s_k)$ from $z(s_k)$ via separate output heads for the real and imaginary components. Training minimises the Laplace-domain reconstruction error together with an L_2 regularisation term:

$$\mathcal{L}_{\text{SLAE}} = \|\tilde{U}(s_k) - \hat{U}(s_k)\|_{\mathbb{C}^{N_x}}^2 + \beta \|z(s_k)\|_2^2 \quad (10)$$

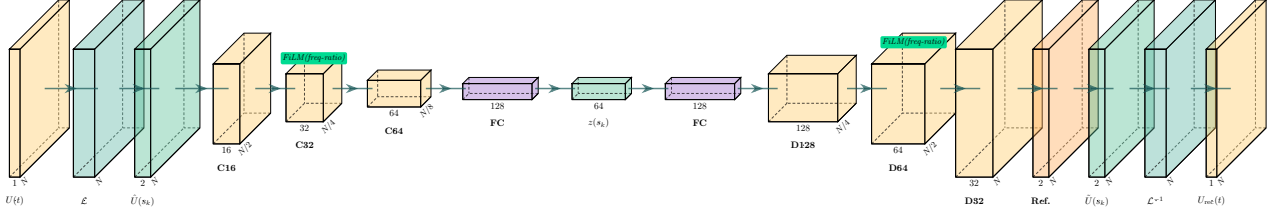
where $\|\cdot\|_{\mathbb{C}^{N_x}}^2$ is the squared L_2 norm over the N_x spatial nodes (real and imaginary parts combined), and β is a regularisation hyperparameter.

2.2.2 Latent Laplace AE (LLAE)

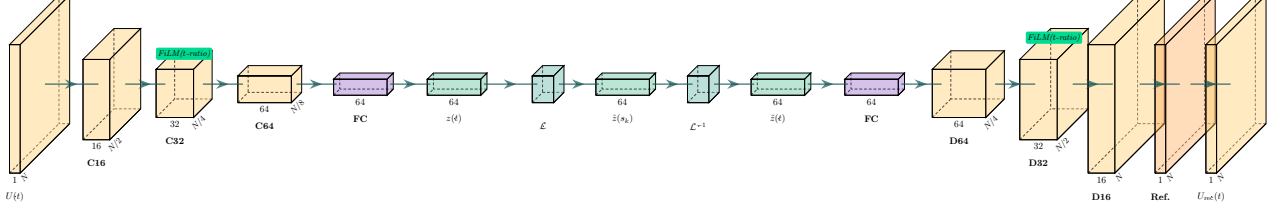
LLAE encodes the physical frames $U(t)$ into a latent trajectory $z(t) \in \mathbb{R}^{d_z}$ using a time-conditioned convolutional encoder (conditioning on the normalised ratio t/T), then applies the Laplace transform directly within this compact latent space:

$$U(t) \xrightarrow{\text{Encoder}(t)} z(t) \xrightarrow{\mathcal{L}} \hat{z}(s_k) \xrightarrow{\mathcal{L}^{-1}} \tilde{z}(t) \xrightarrow{\text{Decoder}(t)} U_{\text{rec}}(t) \quad (11)$$

Compressing in the latent space rather than the physical domain offers two advantages: the transform operates on d_z -dimensional vectors instead of N_x -dimensional fields, and the encoder produces a smoother temporal representation than the raw complex frequency frames, easing the spatial compression task. The main counterpart is computational: at each training step and at inference, the encoder and decoder must process all N_t physical frames, whereas SLAE processes only the K frequency frames. Since $N_t \gg K$, LLAE is



(a) SLAE: pixel-wise Laplace transform, then per-frequency convolutional AE ($d_z = 64$, $K = 16$).



(b) LLAE: convolutional AE first, then Laplace transform in latent space ($d_z = 64$, $K = 16$).

Figure 1: SLAE (a) and LLAE (b) autoencoder pipelines.

significantly heavier, which limits the network capacity that can be deployed under a fixed compute budget. The loss combines three terms:

$$\begin{aligned}
 \mathcal{L}_{\text{LLAE}} = & \frac{1}{N_t} \sum_{n=0}^{N_t-1} \|U_{\text{rec}}(t_n) - U(t_n)\|_2^2 \\
 & + \beta_{\text{latent}} \frac{1}{N_t} \sum_{n=0}^{N_t-1} \|\tilde{z}(t_n) - z(t_n)\|_2^2 \\
 & + \beta \frac{1}{K} \sum_{k=0}^{K-1} \|\hat{z}(s_k)\|_{\mathbb{C}^{d_z}}^2
 \end{aligned} \tag{12}$$

which combines the physical-time reconstruction error, a latent consistency term penalizing the round-trip error $\tilde{z}(t_n) - z(t_n)$, and an L_2 regularisation penalty on the Laplace-domain latent codes $\hat{z}(s_k)$. The latent consistency term is unique to LLAE: it explicitly enforces the strict invertibility of the continuous Laplace transform within the learned discrete latent space, a structural constraint not required by SLAE since it encodes each frequency frame independently.

2.3 Surrogate Architecture and Targets

Following the DL-ROM paradigm of [9, 10], the surrogate MLP g_θ (shared trunk + per-frequency heads; architecture detailed in Figure 2 and Appendix G) replaces the encoding steps by learning the direct mapping $\mu \mapsto w_k$, where w_k denotes the Laplace-domain latent at frequency s_k that feeds the decoder: $w_k = z(s_k) \in \mathbb{R}^{d_z}$ for SLAE, and $w_k = \hat{z}(s_k) \in \mathbb{C}^{d_z}$ for LLAE. Two strategies for constructing the surrogate target w_k are described below.

2.3.1 Direct Latent Prediction

In the direct approach, the surrogate predicts the full latent vector at each index without any preliminary compression. For SLAE, the encoder produces frequency-domain latents

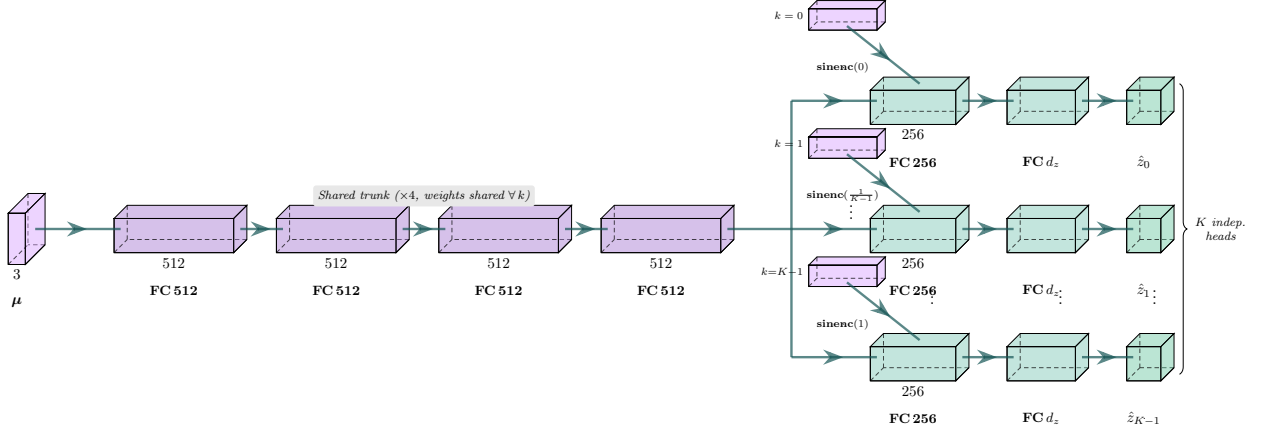


Figure 2: *Surrogate MLP architecture.* A shared trunk maps physical parameters μ to a latent representation, which is broadcast to K independent heads. Each head is conditioned on a frequency index via sinusoidal encoding and predicts the target latent w_k for that frequency.

$z(s_k) \in \mathbb{R}^{d_z}$ directly, and the pipeline is:

$$\mu \xrightarrow{g_\theta} z_{\text{pred}}(s_k) \xrightarrow{\text{Decoder}} \tilde{U}(s_k) \xrightarrow{\mathcal{L}^{-1}} U_{\text{rec}}(t) \quad (13)$$

For LLAE, the time-domain latent sequence $z(t) \in \mathbb{R}^{d_z}$ is first transformed to the Laplace domain, and the surrogate predicts those Laplace-domain codes directly:

$$\mu \xrightarrow{g_\theta} \hat{z}_{\text{pred}}(s_k) \xrightarrow{\mathcal{L}^{-1}} \tilde{z}_{\text{pred}}(t) \xrightarrow{\text{Decoder}} U_{\text{rec}}(t) \quad (14)$$

2.3.2 Latent Sequence SVD Compression

To reduce the effective output dimension of the surrogate and simplify its learning task, one can apply a preliminary offline SVD compression to the Laplace-domain latents before training the surrogate, an approach inspired by [30]. For both SLAE and LLAE, the SVD is applied in the frequency domain: given the K Laplace-domain latent codes $\{\hat{z}(s_k)\}_{k=1}^K$ collected over the training set, a truncated SVD yields a projection matrix $V \in \mathbb{R}^{d_z \times k_{\text{svd}}}$ (with $k_{\text{svd}} \ll d_z$) such that $\hat{z}(s_k) \approx (\hat{z}(s_k)V)V^\top$. The reduced codes $\hat{G}(s_k) = \hat{z}(s_k)V \in \mathbb{R}^{k_{\text{svd}}}$ become the surrogate targets, reducing the prediction problem from dimension d_z to k_{svd} .

In the *SLAE-SVD* variant (latents indexed by frequency k): the projection yields $G(s_k) = z(s_k)V$, and the surrogate predicts in the compressed frequency-domain space:

$$\mu \xrightarrow{g_\theta} G_{\text{pred}}(s_k) \xrightarrow{V^\top} z_{\text{pred}}(s_k) \xrightarrow{\text{Decoder}} \tilde{U}(s_k) \xrightarrow{\mathcal{L}^{-1}} U_{\text{rec}}(t) \quad (15)$$

In the *LLAE-SVD* variant, the SVD is likewise applied in the Laplace domain: the projection yields $\hat{G}(s_k) = \hat{z}(s_k)V$, and the surrogate predicts the compressed frequency-domain latents before back-projection and inversion:

$$\mu \xrightarrow{g_\theta} \hat{G}_{\text{pred}}(s_k) \xrightarrow{V^\top} \hat{z}_{\text{pred}}(s_k) \xrightarrow{\mathcal{L}^{-1}} \tilde{z}_{\text{pred}}(t) \xrightarrow{\text{Decoder}} U_{\text{rec}}(t) \quad (16)$$

Because V is real and acts on the latent channel index while $\mathcal{L}^{-1}$ acts on the frequency index, the two commute: back-projecting before or after the inversion is equivalent, and the implementation applies whichever order is cheaper.

In both strategies, the entire pipeline is trained end-to-end to minimise the total physical-time loss:

$$\mathcal{L}_{\text{total}} = \alpha_{\text{lat}} \|w_k^{\text{pred}} - w_k^{\text{true}}\|^2 + \alpha_{\text{spat}} \|U_{\text{rec}} - U\|^2 \quad (17)$$

where w_k^{pred} and w_k^{true} denote the predicted and true Laplace-domain latents (either $z(s_k)$ or $\hat{G}(s_k)$ depending on the chosen strategy). End-to-end backpropagation allows gradients from the spatiotemporal MSE to flow through the decoder and the inverse transform, jointly updating the surrogate weights, the AE decoder, and, in the SVD variant, the projection basis V .

2.3.3 Tucker Decomposition in Latent Space

Just like the SVD approach described in the previous section, the goal here is to further compress the latent representations of both LLAE and SLAE before training the surrogate MLP, which leads to two distinct variants: *LLAE-Tucker* and *SLAE-Tucker*. While the SVD approach reduces the latent sequence along a single mode at a time (for LLAE-SVD it collapses the latent dimension d_z ; for SLAE-SVD it collapses d_z independently at each frequency, ignoring cross-frequency correlations), Tucker decomposition [21] generalises this by simultaneously factorising two modes of the latent tensor.

Crucially, for both variants, the Tucker decomposition is applied in the latent space *after* the Laplace transform. We assemble the frequency-domain latents of the training set into a three-way tensor $\hat{\mathcal{Z}} \in \mathbb{C}^{N_{\text{train}} \times K \times d_z}$ (real-valued for SLAE, complex-valued for LLAE). A Tucker- (r_μ, r_s, r_z) decomposition approximates each scalar element of the tensor as a triple sum:

$$\hat{\mathcal{Z}}_{i,j,m} \approx \sum_{p=1}^{r_\mu} \sum_{l=1}^{r_s} \sum_{q=1}^{r_z} \hat{C}_{p,l,q} U_{i,p}^{(\mu)} U_{j,l}^{(s)} U_{m,q}^{(z)}, \quad (18)$$

where $\hat{C} \in \mathbb{C}^{r_\mu \times r_s \times r_z}$ is the global core tensor, and $U^{(\mu)}, U^{(s)}, U^{(z)}$ are the orthogonal factor matrices for the parameter (sample), frequency, and latent modes, respectively. To enable predictions on unseen test samples, we absorb the parameter-dependent terms into a simulation-specific core matrix $\hat{\mathcal{G}}_i \in \mathbb{C}^{r_s \times r_z}$, whose elements are $(\hat{\mathcal{G}}_i)_{l,q} = \sum_{p=1}^{r_\mu} \hat{C}_{p,l,q} U_{i,p}^{(\mu)}$. This allows us to rewrite the approximation for the full frequency-domain latent matrix $\hat{\mathcal{Z}}_i \in \mathbb{C}^{K \times d_z}$ of the i -th simulation in a compact matrix form:

$$\hat{\mathcal{Z}}_i \approx U^{(s)} \hat{\mathcal{G}}_i U^{(z)\top}, \quad U^{(s)} \in \mathbb{C}^{K \times r_s}, \quad U^{(z)} \in \mathbb{C}^{d_z \times r_z}. \quad (19)$$

The shared orthogonal factors $U^{(s)}$ and $U^{(z)}$ are computed offline by the Higher-Order Orthogonal Iteration (HOOI) algorithm [4] and kept frozen. The surrogate then learns the mapping $\mu \mapsto \hat{\mathcal{G}}(\mu) \in \mathbb{C}^{r_s \times r_z}$, whose dimensionality $r_s \cdot r_z$ is substantially smaller than $K \cdot d_z$ because it jointly exploits cross-frequency and latent correlations.

For LLAE-Tucker, the surrogate predicts the complex-valued core matrix, from which the full Laplace-domain latents are reconstructed and inverted back to the time domain before decoding:

$$\mu \xrightarrow{g_\theta} \hat{\mathcal{G}}_{\text{pred}} \xrightarrow{U^{(s)}, U^{(z)\top}} \hat{z}_{\text{pred}}(s_k) \xrightarrow{\mathcal{L}^{-1}} \tilde{z}_{\text{pred}}(t) \xrightarrow{\text{Decoder}} U_{\text{rec}}(t) \quad (20)$$

For SLAE-Tucker, the surrogate predicts the real-valued core matrix, which is expanded to the real-valued frequency latents, decoded to spatial Laplace fields, and finally inverted to the physical time domain:

$$\mu \xrightarrow{g_\theta} \mathcal{G}_{\text{pred}} \xrightarrow{U^{(s)}, U^{(z)\top}} z_{\text{pred}}(s_k) \xrightarrow{\text{Decoder}} \tilde{U}(s_k) \xrightarrow{\mathcal{L}^{-1}} U_{\text{rec}}(t) \quad (21)$$

Unlike the real basis V of the SVD variant, the frequency factor $U^{(s)}$ mixes the Laplace frequencies and $U^{(z)}$ is complex-valued; neither commutes with $\mathcal{L}^{-1}$, so the back-projection necessarily precedes the inversion. Both variants share the same conceptual advantage: instead of predicting K independent vectors of size d_z , the surrogate predicts a highly compact core matrix that captures the essential spatiotemporal dynamics.

2.4 Designing the Regularised Inversion

The object to be designed is not the contour alone but the *triplet* (s_k, α_t, λ) : the K poles together with the two regularisation weights of (3). The reference against which we compare fixes all three by convention, poles on the truncated Bromwich contour and (α_t, λ) set a priori and never tuned. Whether the triplet can be designed instead depends on where the transform is applied.

In LLAE it is applied in the low-dimensional latent space, where the triplet is directly differentiable: the forward quadrature (1) is a finite sum of complex exponentials $e^{-s_k t_n}$ smooth in s_k , and the Tikhonov inversion (4) a linear solve whose matrix A depends smoothly on s_k , α_t and λ . The triplet can therefore be carried as parameters of the model and optimised end-to-end, which is what our LLAE implementation does: $\sigma_k = \text{Re}(s_k)$ and $\omega_k = \text{Im}(s_k)$ are learned independently for each pole, together with α_t and λ in log-parameterisation, from an initialisation on the truncated Bromwich contour (ω_k at the K first FFT frequencies, $\sigma_k = \gamma_{\text{init}}$ uniform). This case requires no further development, and we report it as the latent counterpart of the procedure below: it yields the most accurate autoencoder of the study without improving the deployed error (Appendix E).

In SLAE the poles enter the transform of the physical field, and one gradient step on them would require recomputing all N_t spatial forward passes, negating the computational advantage of the family. The triplet must therefore be selected before training, hence scored without a trained network to score it against.

We present a theoretical framework for this purpose. The goal is to decompose the total reconstruction error $\mathbf{u}(\boldsymbol{\mu}) - \mathbf{u}_{\text{rec}}(\boldsymbol{\mu})$ into a *bias* term (introduced by regularisation and the finite contour) and a *variance* term (the surrogate error propagated through the inversion), and to jointly optimise the contour and regularisation parameters by minimising an upper bound on the expected reconstruction error.

2.4.1 Bounding the Reconstruction Error

Formal setup. We fix a spatial location and work with scalar per-pixel quantities, in the notation of Section 2.1: coefficient vectors carry the K_{full} entries of the conjugate-symmetric set $\mathcal{S}$ of (2) (the K predicted ones extended by conjugation exactly as the exact ones are), and $\mathbf{F} \in \mathbb{C}^{K_{\text{full}} \times N_t}$ is assembled on $\mathcal{S}$. Let $\mathbf{u}(\boldsymbol{\mu}) \in \mathbb{R}^{N_t}$ be the true temporal signal at that location, $\hat{\mathbf{u}}(\boldsymbol{\mu}) = \mathbf{F}\mathbf{u}(\boldsymbol{\mu})$ its *discrete* Laplace coefficients, and $\tilde{\mathbf{u}}(\boldsymbol{\mu}) \in \mathbb{C}^{K_{\text{full}}}$ the surrogate-predicted ones, the per-pixel counterpart of $\tilde{U}(s_k)$ of Section 2.2.1. The coefficients are discrete in the sense that the quadrature (1) produces them, rather than the continuous transform it approximates, so that the quadrature error enters neither term below. The regularised reconstructions from the true and predicted coefficients are

$$\mathbf{u}^*(\boldsymbol{\mu}) = A^{-1} \Re \left(\mathbf{F}^H \hat{\mathbf{u}}(\boldsymbol{\mu}) \right), \quad \mathbf{u}_{\text{rec}}(\boldsymbol{\mu}) = A^{-1} \Re \left(\mathbf{F}^H \tilde{\mathbf{u}}(\boldsymbol{\mu}) \right), \quad (22)$$

with $A = \Re(\mathbf{F}^H \mathbf{F}) + L_{\text{reg}}$ the normal-equation matrix of (4) and $L_{\text{reg}} = \alpha_t \mathbf{D}_t^T \mathbf{D}_t + \lambda \mathbf{I}$. Both are unique: $\alpha_t \geq 0$, $\lambda > 0$ and $\mathbf{D}_t^T \mathbf{D}_t \succeq 0$ make $L_{\text{reg}} \succeq \lambda \mathbf{I}$ and hence A symmetric

positive definite (Appendix H).

Bias–amplification decomposition. Adding and subtracting $\mathbf{u}^*(\boldsymbol{\mu})$,

$$\begin{aligned}\mathbf{u}(\boldsymbol{\mu}) - \mathbf{u}_{\text{rec}}(\boldsymbol{\mu}) &= (\mathbf{u}(\boldsymbol{\mu}) - \mathbf{u}^*(\boldsymbol{\mu})) + (\mathbf{u}^*(\boldsymbol{\mu}) - \mathbf{u}_{\text{rec}}(\boldsymbol{\mu})) \\ &= (\mathbf{u}(\boldsymbol{\mu}) - A^{-1}\Re(\mathbf{F}^H \mathbf{F})\mathbf{u}(\boldsymbol{\mu})) + A^{-1}\Re(\mathbf{F}^H(\hat{\mathbf{u}}(\boldsymbol{\mu}) - \tilde{\mathbf{u}}(\boldsymbol{\mu}))).\end{aligned}\quad (23)$$

and using $A = \Re(\mathbf{F}^H \mathbf{F}) + \mathbf{L}_{\text{reg}}$, the first term simplifies:

$$\mathbf{u}(\boldsymbol{\mu}) - \mathbf{u}^*(\boldsymbol{\mu}) = A^{-1}(A - \Re(\mathbf{F}^H \mathbf{F}))\mathbf{u}(\boldsymbol{\mu}) = A^{-1}\mathbf{L}_{\text{reg}}\mathbf{u}(\boldsymbol{\mu}).\quad (24)$$

Taking norms in (23) and applying the triangle inequality then yields

$$\|\mathbf{u}(\boldsymbol{\mu}) - \mathbf{u}_{\text{rec}}(\boldsymbol{\mu})\| \leq \underbrace{\|A^{-1}\mathbf{L}_{\text{reg}}\mathbf{u}(\boldsymbol{\mu})\|}_{\text{regularisation bias}} + \underbrace{\|A^{-1}\mathbf{F}^H\|}_{\text{amplification factor}} \cdot \underbrace{\|\hat{\mathbf{u}}(\boldsymbol{\mu}) - \tilde{\mathbf{u}}(\boldsymbol{\mu})\|_{\mathbb{C}^{K_{\text{full}}}}}_{\text{surrogate error}},\quad (25)$$

valid for any $\boldsymbol{\mu}$. Appendix H gives the derivation, together with the field-level counterpart (51) that carries the same amplification factor to the full $N_x \times N_t$ field, the form the criterion below discretises. We state it for the structure it exposes: the amplification factor $\|A^{-1}\mathbf{F}^H\|$, by which any surrogate error is magnified during reconstruction, depends on (s_k, α_t, λ) and on nothing else, and the bias on the triplet and on ground-truth trajectories alone. Neither involves a trained network, which is what makes a triplet scorable before training and is the whole basis of the offline spatial procedure below. Here and throughout, $\|\cdot\|$ on a matrix is the operator 2-norm; the implementation evaluates $\|A^{-1}\mathbf{F}^H\|_F \geq \|A^{-1}\mathbf{F}^H\|_2$, cheaper to differentiate and preserving the inequality.

2.4.2 The Offline Selection Criterion

Optimisation problem. The matrices $\mathbf{F}$ and $\mathbf{L}_{\text{reg}}$, and consequently A , depend explicitly on the contour $(s_1, \dots, s_K)$, on λ and on α_t . We would minimise the expected reconstruction error $\mathbb{E}_{\boldsymbol{\mu}}[\|\mathbf{u}(\boldsymbol{\mu}) - \mathbf{u}_{\text{rec}}(\boldsymbol{\mu})\|]$ over these, but evaluating it demands a trained surrogate for every candidate contour. We therefore treat them as learnable parameters of a proxy read off the bound (25):

$$\mathcal{L}^{\text{proxy}} = \mathcal{L}_{\text{bias}} + \mathcal{L}_{\text{var}}, \quad \mathcal{L}_{\text{bias}} = \mathbb{E}_{\boldsymbol{\mu}}[\|\mathbf{u}(\boldsymbol{\mu}) - \mathbf{u}^*(\boldsymbol{\mu})\|], \quad \mathcal{L}_{\text{var}} = \|A^{-1}\mathbf{F}^H\|_F \cdot \epsilon_{\text{surr}},\quad (26)$$

where $\epsilon_{\text{surr}} = \mathbb{E}_{\boldsymbol{\mu}}[\|\hat{\mathbf{u}}(\boldsymbol{\mu}) - \tilde{\mathbf{u}}(\boldsymbol{\mu})\|_{\mathbb{C}^{K_{\text{full}}}}]$ is the expected surrogate error. We work with the second moment $\epsilon_2 = \sqrt{\mathbb{E}_{\boldsymbol{\mu}}[\|\hat{\mathbf{u}} - \tilde{\mathbf{u}}\|^2]} \geq \epsilon_{\text{surr}}$, by Jensen: it is available in closed form and substituting it preserves the bound.

Surrogate error via an SVD residual. Training a surrogate per candidate contour being intractable, we replace ϵ_2 by the residual of the best *linear* reduction of the same rank. With $\hat{U}(s_k; \boldsymbol{\mu}) \in \mathbb{C}^{N_x}$ the full spatial Laplace field at s_k , each instance is the real vector

$$\mathbf{a}(\boldsymbol{\mu}) = [\Re \hat{U}(s_1; \boldsymbol{\mu}); \Im \hat{U}(s_1; \boldsymbol{\mu}); \dots; \Re \hat{U}(s_K; \boldsymbol{\mu}); \Im \hat{U}(s_K; \boldsymbol{\mu})] \in \mathbb{R}^{2N_x K},\quad (27)$$

and $\mathbf{U} = [\mathbf{a}(\boldsymbol{\mu}_1), \dots, \mathbf{a}(\boldsymbol{\mu}_{N_{\text{opt}}})] \in \mathbb{R}^{2N_x K \times N_{\text{opt}}}$ collects a sample of N_{opt} training instances. The best rank- d_z reduction is the truncated SVD, its residual the sum of the discarded squared singular values [6, 29]:

$$\mathcal{E}_{\text{SVD}} = \sqrt{\min_{\text{rank } \hat{\mathbf{U}} \leq d_z} \frac{1}{N_{\text{opt}}} \|\mathbf{U} - \hat{\mathbf{U}}\|_F^2} = \sqrt{\frac{1}{N_{\text{opt}}} \sum_{j>d_z} \sigma_j^2}, \quad (28)$$

The single basis is shared across the K frequencies, so the score depends on the contour through one spectrum. $\mathbf{U}$ is never formed: the $N_{\text{opt}} \times N_{\text{opt}}$ Gram matrix $\mathbf{U}^\top \mathbf{U}$ is accumulated over spatial chunks and (28) read off its eigenvalues, at a memory cost independent of N_x ; the rank is capped at $\min(d_z, N_{\text{opt}})$, and we use $N_{\text{opt}} = 100$ against $d_z = 64$.

Appendix H shows $\mathcal{E}_{\text{SVD}}$ is the smallest coefficient error any map $\boldsymbol{\mu} \mapsto \mathbf{D} g(\boldsymbol{\mu})$ with a rank- d_z linear decoder can reach on that sample, g arbitrary: a floor set by the bottleneck and the contour, not by any training. A contour whose Laplace fields nearly span a d_z -dimensional subspace poses an easier prediction problem, and this scores that. It bounds neither SLAE, the richer class, nor a trained network, which attains no floor in general, so that contours order the same way under $\mathcal{E}_{\text{SVD}}$ and under the deployed error is a working hypothesis, discussed in Appendix H. We use it as deterministic, differentiable in the triplet and network-free, with one scalar $\lambda_{\text{AE}} \geq 1$ setting where the variance term enters: $\epsilon_{\text{sur}} becomes $\mathcal{E}_{\text{SVD}}/\lambda_{\text{AE}}$ in (30).$

Smoothness heuristic on the bias term. One *heuristic* ingredient, not following from (25): at equal norm a smooth offset is far less damaging than the Gibbs-like oscillations characteristic of spectral truncation, which the L^2 norm does not distinguish, so we penalise the finite differences of the bias too. With $\mathbf{e}(\boldsymbol{\mu}) = \mathbf{U}(\boldsymbol{\mu}) - \mathbf{U}^*(\boldsymbol{\mu}) \in \mathbb{R}^{N_x \times N_t}$ and $\mathbf{D}_t, \mathbf{D}_x, \mathbf{D}_y$ the difference operators along time and the two grid axes,

$$\mathcal{L}_{\text{bias}} = \frac{1}{1 + \lambda_{\Delta t} + \lambda_{\Delta x}} \mathbb{E}_{\boldsymbol{\mu}} \left[\|\mathbf{e}(\boldsymbol{\mu})\|_F + \lambda_{\Delta t} \|\mathbf{D}_t \mathbf{e}(\boldsymbol{\mu})\|_F + \lambda_{\Delta x} (\|\mathbf{D}_x \mathbf{e}(\boldsymbol{\mu})\|_F + \|\mathbf{D}_y \mathbf{e}(\boldsymbol{\mu})\|_F) \right], \quad (29)$$

the normalisation keeping $\mathcal{L}_{\text{bias}}$ on the scale of the variance term. Zero weights recover the plain bias of (25); positive ones trade the upper-bound guarantee for a preference over the *shape* of the residual. The final objective is

$$\begin{aligned} \mathcal{L}^{\text{proxy}} = & \frac{1}{N_{\text{opt}} (1 + \lambda_{\Delta t} + \lambda_{\Delta x})} \sum_{i=1}^{N_{\text{opt}}} \left(\|\mathbf{e}(\boldsymbol{\mu}_i)\|_F + \lambda_{\Delta t} \|\mathbf{D}_t \mathbf{e}(\boldsymbol{\mu}_i)\|_F \right. \\ & \left. + \lambda_{\Delta x} (\|\mathbf{D}_x \mathbf{e}(\boldsymbol{\mu}_i)\|_F + \|\mathbf{D}_y \mathbf{e}(\boldsymbol{\mu}_i)\|_F) \right) \\ & + \frac{\|\mathbf{A}^{-1} \mathbf{F}^H\|_F}{\lambda_{\text{AE}}} \cdot \sqrt{\frac{1}{N_{\text{opt}}} \sum_{j>d_z} \sigma_j^2}. \end{aligned} \quad (30)$$

Gradient descent on (30) then optimises the poles and the regularisation parameters jointly. Both terms use the same N_{opt} instances, drawn at a single rotation angle so the sample spans the (k, C) plane rather than 36 views of a few simulations; the procedure runs in double precision and touches no network. Remaining settings, including the box constraints $\sigma_k \geq -0.05$, $\omega_k \geq 0$, $\alpha_t, \lambda \in [10^{-6}, 1]$, are in Appendix G; the optimised contours are used in Section 3.5.

2.5 Querying a Trained Model on a Requested Time Grid

The design of Sections 2.1 and 2.4 has a consequence that is not an accuracy claim and does not depend on any trained network: it fixes the set of output *time* grids a model can afterwards be asked for. The spatial discretisation is not at issue: the decoder is a convolutional network of fixed output size, and everything below concerns the time axis alone. Three properties follow from the inversion (3)–(4) alone, and together they determine what a trained model can be asked for at inference time.

Grid-independence of the coefficients. The Laplace coefficients $\hat{u}(s_k)$ carry no reference to any time grid. The trajectory is materialised by the solve (3), whose forward matrix $\mathbf{F}_{k,n} = \Delta t w_n e^{-s_k t_n}$ is assembled from whatever nodes t_n are requested, and whose unknown $\mathbf{u}$ is supported on those same nodes. A model predicting K coefficients therefore answers on any time grid over the horizon T , at any spacing and in principle non-uniformly, for the cost of reassembling a $K_{\text{full}} \times N_t$ matrix and one $N_t \times N_t$ solve; nothing in the trained weights is tied to the grid on which they were fitted. A model that instead emits one output per time step has no such freedom, its predictor being fixed at training time.

Rescaling the regularisation. The freedom is not entirely free: (α_t, λ) are calibrated at the training spacing and must be transported to the requested one, since both penalties in (3) are discrete sums whose value depends on Δt . With $u_{n+1} - u_n \simeq \Delta t u'(t_n)$,

$$\|\mathbf{D}_t \mathbf{u}\|_2^2 \simeq \Delta t \int_0^T u'(t)^2 dt, \quad \|\mathbf{u}\|_2^2 \simeq \frac{1}{\Delta t} \int_0^T u(t)^2 dt, \quad (31)$$

while the data term is a quadrature of $\hat{u}(s_k)$ and is already Δt -independent. Holding the underlying continuous functional fixed as the grid changes therefore requires $\alpha_t \Delta t$ and $\lambda / \Delta t$ to be invariant, that is

$$\alpha_t^{\text{out}} = \alpha_t \frac{\Delta t}{\Delta t_{\text{out}}}, \quad \lambda^{\text{out}} = \lambda \frac{\Delta t_{\text{out}}}{\Delta t}. \quad (32)$$

Without (32) a refinement of the grid under-smooths and a coarsening over-smooths, the penalties being read at the wrong strength. We apply it whenever a checkpoint is queried away from its training grid; Section 4.3 reports both settings.

The sampling floor. The freedom is bounded below, and the bound is known before any measurement. Write $T = N_t \Delta t$ for the horizon and $\Delta t_{\text{out}} = T / N_t^{\text{out}}$ for the spacing of the requested grid. Let $\omega_{\text{max}} = \max_k |\omega_k|$ be the fastest component the pole set encodes; the conjugate completion (2) adds the mirrored frequencies $-\omega_k$ and therefore leaves ω_{max} unchanged. Since (3) returns a vector supported on the requested nodes, that component can be carried by the output only if it lies below the Nyquist frequency of the requested grid, $\omega_{\text{Nyq}} = \pi / \Delta t_{\text{out}}$ [35]. The condition $\omega_{\text{max}} \leq \omega_{\text{Nyq}}$ reads

$$N_t^{\text{out}} \geq \frac{T \omega_{\text{max}}}{\pi}. \quad (33)$$

Below this bound the fastest encoded component is aliased on the output grid and no inversion can recover it, whatever the model: the resolution range is a property of the representation, and the structural price of encoding the transient spectrally. The condition is necessary for representability and not sufficient for accuracy, the inversion remaining ill-posed above the floor, and it is attained at the *critical* rate of two samples per period, at which recovery depends on phase; a contour sitting exactly at its floor is

therefore marginal rather than safe. It holds for any pole set, whatever the placement of the s_k in the complex plane. It takes a closed form on the contour we use throughout: the truncated Bromwich contour (5) puts the imaginary parts on the first K discrete Fourier frequencies, so the largest is ω_{K-1} and

$$\omega_{\max} = \frac{2\pi(K-1)}{T} \implies N_t^{\text{out}} \geq 2(K-1) \quad (\text{truncated Bromwich}). \quad (34)$$

The horizon cancels here: on this contour the floor is set by the number of Laplace coefficients alone, not by the resolution the model was trained on, so $K = 16$ serves down to $N_t^{\text{out}} = 30$ and $K = 8$ down to 14. The identity $2(K-1)$ is specific to (5) and does not survive a change of contour: a pole set whose imaginary parts are free, as produced by the optimisation of Section 2.4, has its own $\omega_{\max}$ and must be scored through the general bound (33).

2.6 Baselines

The approach described so far combines two ingredients: a non-linear autoencoder for spatial compression, and a Laplace transform for temporal compression. We therefore compare against two references, each removing exactly one of them. The first is *linear* in the spatial compression step but keeps the Laplace domain: a per-frequency POD basis with a non-intrusive surrogate on the projection coefficients (Section 2.6.1). The second is *non-linear* but Laplace-free, and keeps the full temporal grid: a standard DL-ROM (Section 2.6.2). Read together, they attribute the observed performance to one ingredient or the other rather than to their combination. Both reuse the surrogate of Section 2.3 and the two-phase training protocol unchanged.

2.6.1 Linear Baseline: POD in the Laplace Domain

As a linear reference, we construct a Proper Orthogonal Decomposition (POD) basis for each Laplace frequency independently. For each s_k , the training-set spatial fields are stacked into a data matrix $\mathbf{U}^{(k)} \in \mathbb{R}^{N_x \times N_{\text{train}}}$ (one per physical component), and a truncated basis of rank k_{pod} is extracted. Because $N_x = N^2 = 16,384$ and $N_{\text{train}} = 5,184$, a direct SVD of $\mathbf{U}^{(k)}$ would be memory-intensive; we instead use a two-pass randomized SVD [16] that reduces the cost to two passes over the data and a small k_{pod} -dimensional eigenvalue problem. The real and imaginary parts of $\hat{U}(s_k)$ are treated as separate real-valued fields, each with its own basis $\mathbf{V}^{(k,r)}, \mathbf{V}^{(k,i)} \in \mathbb{R}^{N_x \times k_{\text{pod}}}$ fitted on the training set:

$$\Re(\hat{U}(s_k)) \approx \mathbf{V}^{(k,r)} \mathbf{c}^{(k,r)}, \quad \Im(\hat{U}(s_k)) \approx \mathbf{V}^{(k,i)} \mathbf{c}^{(k,i)}, \quad (35)$$

where $\mathbf{c}^{(k,r)}, \mathbf{c}^{(k,i)} \in \mathbb{R}^{k_{\text{pod}}}$ are the projection coefficients. This is the classical Galerkin-POD representation applied frequency-by-frequency in the Laplace domain [18].

Surrogate for the POD coefficients. Given the compact coefficient vectors, the surrogate task reduces to predicting $2Kk_{\text{pod}}$ scalar targets from $\boldsymbol{\mu}$. This is the classical non-intrusive POD-NN strategy [19], here applied frequency-by-frequency in the Laplace domain. The surrogate MLP g_θ of Section 2.3 predicts the normalised coefficients for all frequencies jointly:

$$\mathbf{c}^{(k,r)}, \mathbf{c}^{(k,i)} = g_\theta(\boldsymbol{\mu}, k), \quad k = 0, \dots, K-1. \quad (36)$$

Training minimises a combined latent-coefficient MSE and physical reconstruction loss (after Tikhonov inversion):

$$\begin{aligned} \mathcal{L}_{\text{POD-surr}} = & \alpha_{\text{lat}} \sum_{k=0}^{K-1} \left(\|g_{\theta}^{(k,r)}(\mu) - \bar{c}^{(k,r)}\|_2^2 + \|g_{\theta}^{(k,i)}(\mu) - \bar{c}^{(k,i)}\|_2^2 \right) \\ & + \alpha_{\text{spat}} \|U_{\text{rec}} - U\|_2^2, \end{aligned} \quad (37)$$

where $\bar{c}^{(k,\cdot)}$ denotes the normalised coefficients and U_{rec} is the field reconstructed by back-projecting the predicted coefficients through the fixed bases and applying the inverse Laplace transform. The spatial bases $V^{(k,\cdot)}$ remain *fixed* throughout surrogate training.

The time series $U(t)$ is reconstructed by back-projecting the predicted coefficients, assembling the complex frequency field, and applying the Tikhonov inversion (3). This pipeline is fully linear in the spatial compression step, making it a clean reference that quantifies the gain of non-linear autoencoders. The irreducible truncation error of the linear basis, controlled by the discarded singular values of $U^{(k)}$, sets a hard floor that non-linear AEs are designed to overcome.

2.6.2 Non-Linear Baseline: DL-ROM without Laplace

The POD baseline removes the non-linearity; our second reference removes the Laplace transform instead. It follows the DL-ROM paradigm of [9]: a convolutional autoencoder for spatial compression, a network mapping μ to the latent trajectory, and no spectral compression anywhere in the pipeline,

$$U(t) \xrightarrow{\text{Encoder}(t)} z(t) \xrightarrow{\text{Decoder}(t)} U_{\text{rec}}(t) \quad (38)$$

This is exactly the LLAE pipeline of Section 2.2.2 with the latent round trip $z(t) \rightarrow \hat{z}(s_k) \rightarrow \tilde{z}(t)$ removed: the same convolutional backbone, the same FiLM conditioning on the normalised time t/T , the same L_2 ridge on the latent codes. The loss retains the first and third terms of (12),

$$\mathcal{L}_{\text{DL-ROM}} = \frac{1}{N_t} \sum_{n=0}^{N_t-1} \|U_{\text{rec}}(t_n) - U(t_n)\|_2^2 + \beta \frac{1}{N_t} \sum_{n=0}^{N_t-1} \|z(t_n)\|_2^2, \quad (39)$$

the ridge now applying to $z(t)$ directly rather than to $\hat{z}(s_k)$. The latent consistency term has no counterpart here: with no round trip, there is no invertibility to enforce. The comparison is therefore a controlled ablation of the Laplace block rather than a comparison between two independently tuned models. We take [9] as the inspiration for that ablation rather than reproducing it: its surrogate is a single time-conditioned network queried once per step, ours carries one head per time step, so that the predictor stays architecturally fixed across all four pipelines and only the Laplace block varies. “DL-ROM” accordingly designates our Laplace-free ablation throughout, in the sense of the family; Section 5.3 states which cost comparisons depend on that choice. Within the family we follow [9] rather than the POD-DL-ROM variant [10], whose POD pre-reduction is an efficiency device that leaves the learned representation unchanged; Section 5.2 returns to this point.

Surrogate cost. The surrogate of Section 2.3 carries over unchanged except in *what* it must predict. Where the Laplace pipelines regress the K Laplace-domain latents w_k , the

DL-ROM surrogate must regress the latent trajectory at every time step, $\mu \mapsto z(t_n)$ for $n = 0, \dots, N_t - 1$: the per-frequency heads become per-time-step heads, conditioned on $t_n/(N_t - 1)$ in place of f_k , and their number grows from K to N_t . With a shared trunk of N_{trunk} parameters and heads of N_{head} parameters each (N_{head} depending only on d_z and the head width, not on their number),

$$N_{\text{surr}} = N_{\text{trunk}} + K N_{\text{head}}, \quad K = N_t \text{ (DL-ROM)}, \quad K \ll N_t \text{ (SLAE, LLAE)}, \quad (40)$$

so the DL-ROM surrogate grows linearly with the temporal resolution, while the Laplace surrogates are independent of it. With the architecture of Appendix G and $d_z = 64$, this amounts to 23.5M parameters at $N_t = 150$ against 3.5M for LLAE at $K = 16$, a factor of 6.8, the two becoming equal in size at $N_t \approx 18$.

Temporal-resolution sweep. Since N_t is the sole driver of this cost, we sweep it by uniform subsampling of the time grid with a stride κ , so that $N_t \mapsto \lceil N_t/\kappa \rceil$ and $\Delta t \mapsto \kappa \Delta t$ while the horizon T is held fixed. This traces the DL-ROM cost/accuracy curve while the Laplace pipelines remain at $K = 16$, and it supplies a parameter-matched operating point: at $\kappa = 10$ the DL-ROM surrogate has essentially the size of LLAE at $K = 16$, both compressing time by roughly a factor of ten, uniformly on one side and spectrally on the other. The sweep concerns the DL-ROM alone; the Laplace pipelines are always trained on the full $N_t = 150$ grid, since their temporal compression is already carried by K .

2.7 Summary of the Eleven Pipelines

Table 1 provides a synthetic overview of all eleven pipelines studied. The three top rows describe the *autoencoder reconstruction* (Phase 1, input $U(t)$); the eight remaining rows compare the *surrogate inference* pipelines (Phase 2, input μ), the two baselines first, then grouped by the domain in which the Laplace transform is applied.

3 Results

3.1 Problem Setting

We consider the emulation of the methane mass fraction Y in a two-dimensional parametric transient reacting flow. The transport equation is formulated as a convection-diffusion-reaction problem over a spatial domain $\Omega \subset \mathbb{R}^2$ and a time interval $t \in [0, T]$:

$$\frac{\partial(\rho Y)}{\partial t} + \nabla \cdot (\rho \mathbf{U} Y) - \nabla \cdot (\mu_{\text{eff}} \nabla Y) = \dot{\omega}_{\text{eff}}(Y; \mu) \quad (41)$$

where ρ is the fluid density, $\mathbf{U}$ is the velocity field, μ_{eff} is the effective diffusivity, and $\dot{\omega}_{\text{eff}}$ is the effective reaction rate. A detailed formulation of the complete system of governing equations is provided in Appendix F. The system is governed by a vector of three physical parameters $\mu = [k, \phi, C]^\top \in \mathbb{R}^3$, specifically representing:

- k : Diffusion parameter,
- ϕ : Injection angle,
- C : Injection rate (kg/m³/s).

Model	Pipeline	$\mathcal{L}$ applied in
<i>Autoencoder reconstruction</i> (Phase 1, input: $U(t)$)		
SLAE		phys. domain
LLAE		latent space
DL-ROM		none
<i>Surrogate inference</i> (Phase 2, input: μ)		
POD		phys. domain
DL-ROM		none
SLAE-Direct		phys. domain
SLAE-SVD		phys. domain
SLAE-Tucker		phys. domain
LLAE-Direct		latent space
LLAE-SVD		latent space
LLAE-Tucker		latent space

Table 1: Architecture summary of the eleven pipelines studied. *Top block* (Phase 1): autoencoder reconstruction, input $U(t)$. *Bottom block* (Phase 2): surrogate inference pipelines mapping $\mu \rightarrow U(t)$, the two baselines first, then grouped by the domain in which $\mathcal{L}$ is applied, physical domain (SLAE family) or latent space (LLAE family). The DL-ROM applies no Laplace transform at either phase. Colour code: ■ MLP surrogate; ■ Laplace $\mathcal{L}$ / inverse $\mathcal{L}^{-1}$; ■ SVD back-projection; ■ Tucker back-projection; ■ convolutional encoder E (wide $\rightarrow$ narrow) / decoder D (narrow $\rightarrow$ wide); ● latent-space representation on the arrow.

The temporal domain is discretised into $N_t = 150$ time steps with uniform step size $\Delta t = 1$ s (total duration $T = 150$ s). The quantity of interest in the methodology notation is $U(t) \in \mathbb{R}^{N_x}$, the discretised spatial field of Y at time step t ; the full space–time field is a matrix in $\mathbb{R}^{N_x \times N_t}$. The number of Laplace frequency samples K is a model hyperparameter ($K = 16$ for both SLAE and LLAE; see Table 14).

The transient rises, peaks near $t \approx 70$ s and decays back to zero inside the window (Figure 3).

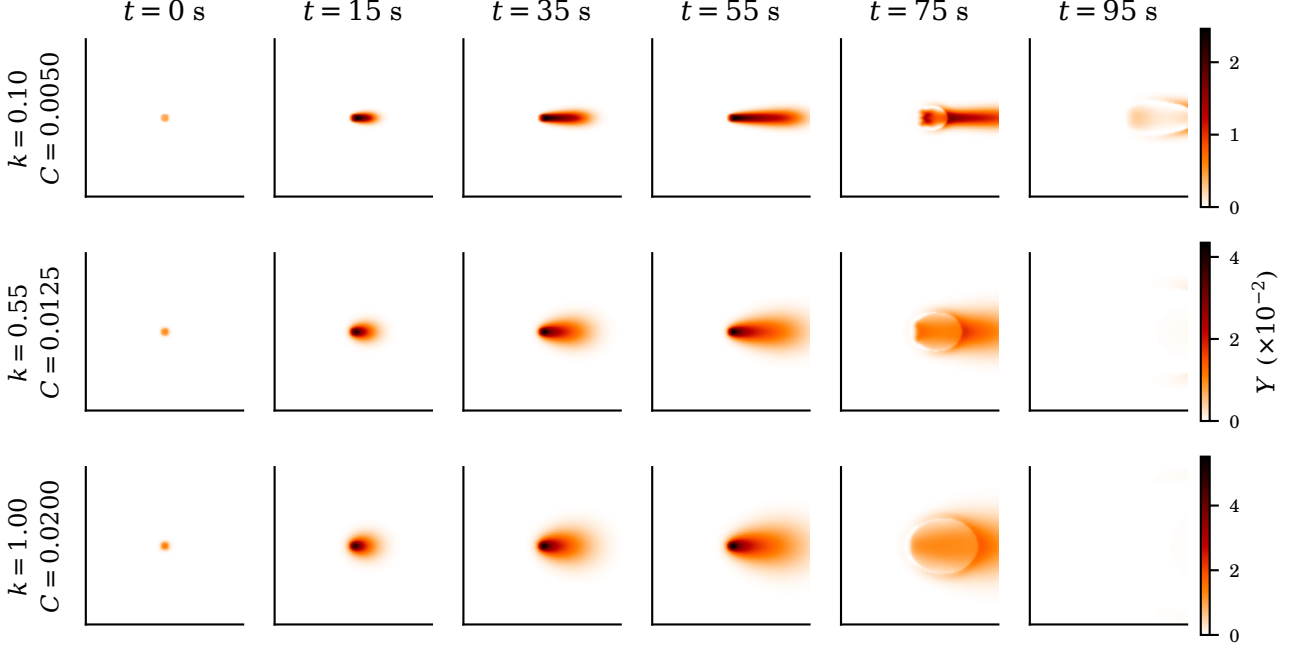


Figure 3: Methane mass fraction Y at six instants, for three instances spanning the parameter box all at $\phi = 0^\circ$. Colour scale shared along a row.

Dataset construction. The parameter sweep is carried out over (k, C) only: 225 high-fidelity simulations on the native 200×200 solver grid. The third parameter, the injection angle ϕ , is realised geometrically. The governing operator being isotropic and the injection centred, a rotation by ϕ about the domain centre maps a solution onto the solution of the problem with the injection rotated by ϕ , and each simulation is replicated at 36 uniformly spaced angles $\phi \in \{0^\circ, 10^\circ, \dots, 350^\circ\}$. The rotation is applied at native resolution and centre-cropped to 128×128 ; the corners of the retained window lying at a radius of 90.5 cells, inside the disc of radius 100 inscribed in the solver domain, the crop retains only the region untouched by the rotation fill, with no interpolation involved. This yields $N_{\text{sim}} = 225 \times 36 = 8,100$ parameter instances on a uniform grid of $N_x = 128 \times 128 = 16,384$ nodes.

Each parameter acts differently on the field (Figure 4).

Partition. A fixed random permutation reserves 20% of the instances, 1,620, as a held-out test set; the remaining 6,480 are split 80/20 into 5,184 training and 1,296 validation instances. Every quantity reported in this paper is measured on the 1,620 test instances. The validation set alone drives early stopping and checkpoint selection (Appendix G); the test set is never used for any training or selection decision, and the POD bases of Section 2.6.1 are fitted on the training partition only.

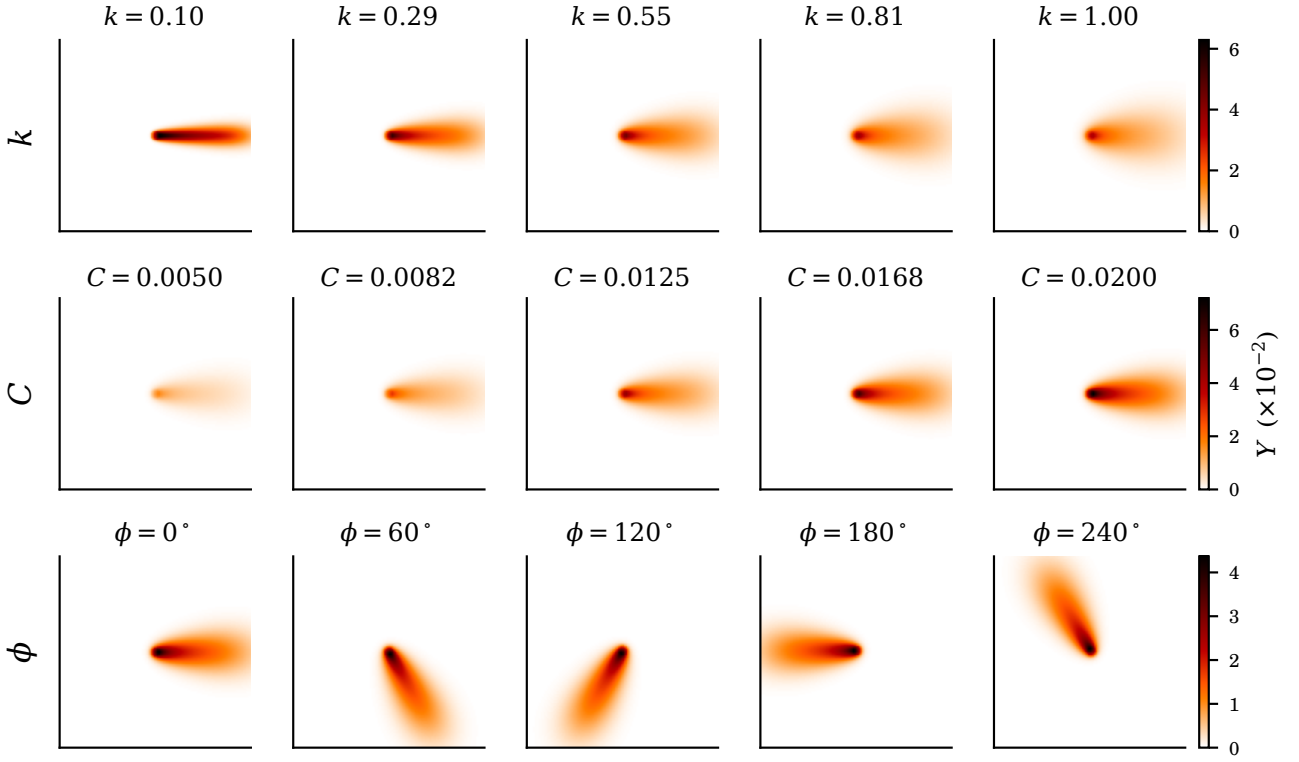


Figure 4: Effect of each parameter at $t = 60$ s, the other two held at the centre of the grid. Colour scale shared along a row.

This replication is a solver shortcut rather than data augmentation: each of the 8,100 instances is an exact solution of the governing system at its own point (k, ϕ, C) , and ϕ enters the design of experiments on the same footing as k and C .

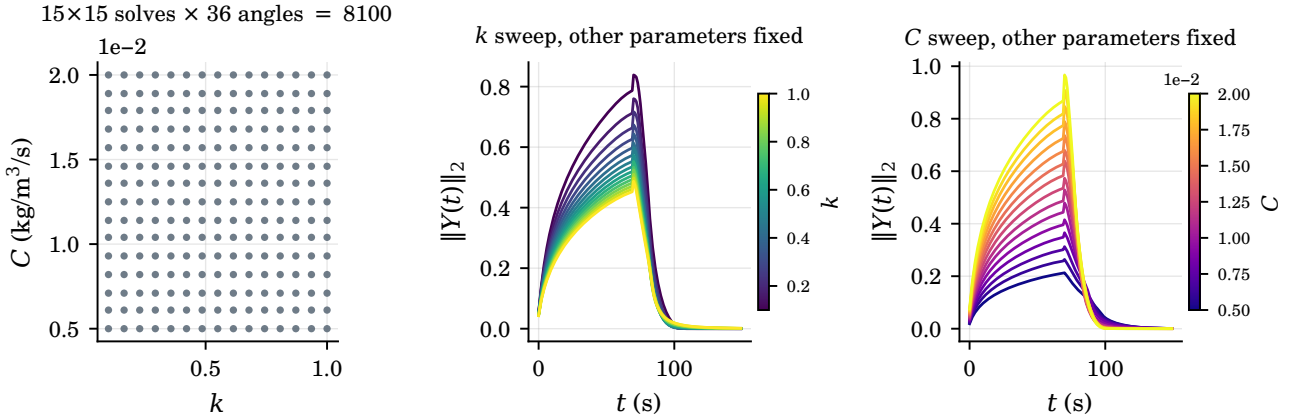


Figure 5: *Left*: design of experiments, a 15×15 grid on (k, C) replicated at 36 angles. *Centre and right*: field norm $\|Y(t)\|_2$ along each sweep, coloured by the swept parameter. All transients decay to zero inside the $T = 150$ s horizon.

The performance of the models is evaluated quantitatively using the relative L^2 error over the held-out test set, defined as:

$$\epsilon_{\text{rel}} = \frac{\|Y_{\text{true}} - Y_{\text{pred}}\|_2}{\|Y_{\text{true}}\|_2} \quad (42)$$

We first characterise the autoencoder architectures in isolation, then evaluate the end-to-end surrogate models.

3.2 Baseline Results

The two references of Section 2.6 remove one ingredient each (POD in the Laplace domain drops the non-linearity, the DL-ROM drops the Laplace transform) and both are characterised in the same two stages as the Laplace pipelines. The first isolates the *representation*, passing the true field through the compression and back with no surrogate involved, which sets the floor of the pipeline; the second is the deployed *surrogate*, predicting from μ alone. Reporting both separates what the representation cannot represent from what the regression fails to predict, and their difference, the Gap, decides whether a better floor is worth having.

3.2.1 POD in the Laplace Domain

The three hyperparameters of the linear baseline (Section 2.6.1) are swept independently around a base configuration ($K = 16$, $k_{\text{pod}} = 64$, $\gamma = 0.01$): $K \in \{8, 16, 32\}$, $k_{\text{pod}} \in \{32, 64, 128\}$ and $\gamma \in \{0, 0.001, 0.01\}$. Each configuration is evaluated at both stages on the 1620 test simulations: as a pure *reconstruction*, in which the true fields are projected onto the training bases with no surrogate involved, and *end-to-end*, in which a network predicts the coefficients from μ . Table 2 reports both.

k_{pod}	K	γ	Rec med	Rec p90	Surr med	Surr p90	Gap (pp)	Lat (ms)
64	32	0.01	7.57	15.08	8.75	17.45	+1.18	5.6 ± 0.0
64	16	0.01	8.09	14.67	8.92	16.35	+0.83	3.5 ± 0.2
64	16	0.001	8.38	14.29	8.95	15.37	+0.57	3.5 ± 0.1
64	16	0.0	8.42	14.22	8.98	15.56	+0.56	3.5 ± 0.1
128	16	0.01	7.04	9.79	11.30	21.76	+4.26	3.7 ± 0.1
32	16	0.01	11.42	22.99	11.72	23.47	+0.30	3.4 ± 0.2
64	8	0.01	12.01	17.39	12.45	18.42	+0.44	2.9 ± 0.2

Table 2: POD baseline on the test set (1620 simulations), sorted by surrogate median error. All errors are relative L^2 errors in %. *Rec*: pure POD reconstruction error (linear floor, no surrogate). *Surr*: end-to-end surrogate error. *Gap*: difference surrogate – reconstruction (positive = surrogate adds error on top of the linear floor). *Lat*: GPU inference latency at batch size 1 (50 timed runs).

Linear reconstruction floor. The floor is governed by the singular-value decay of the Laplace-domain field matrices $U^{(k)}$: the linear basis is accurate at low frequencies, where the spatial structure is smooth and low-rank, and degrades at higher ones, where the fields become localized. Increasing k_{pod} beyond 64 yields only a marginal improvement, indicating that the residual is an intrinsic limit of the linear subspace rather than a truncation artefact. The damping is nearly neutral on the physical field (8.42, 8.38 and 8.09% for $\gamma = 0, 0.001$ and 0.01) because two error sources cancel. A purely imaginary contour concentrates the energy and lowers the POD truncation error at every frequency, whereas a larger γ attenuates the inversion error that propagates those residuals back to the time domain. Appendix B reports the complete sweeps and the per-frequency breakdown that exposes this trade-off.

Surrogate overhead. The regression stage adds little above the floor, between +0.30 and +1.18 pp, with one exception: at $k_{\text{pod}} = 128$, $K = 16$, the best floor of the sweep (7.04%) yields its worst surrogate (11.30%, +4.26 pp). With $2Kk_{\text{pod}} = 4,096$ scalar targets, the higher-dimensional regression problem overwhelms the benefit of the larger basis, a trade-off between representation quality and learnability that recurs at every stage of this study. The POD reference used throughout is therefore $k_{\text{pod}} = 64$, $K = 16$, $\gamma = 0.01$: the configuration matching the common operating point of the deep models ($d_z = k_{\text{pod}}$, same K , same γ) rather than the lowest surrogate error of the sweep. It reaches 8.92% median error at 2.9–5.6 ms per sample at batch size 1, a cost dominated by the back-projection through $V^{(k)}$ and the Tikhonov solve rather than by the network forward pass. This is the floor that the non-linear autoencoders of Section 3.3 are designed to push below; per-sample distributions are provided in Appendix A.

3.2.2 DL-ROM without Laplace

The second reference removes the Laplace transform instead of the non-linearity (Section 2.6.2). It is trained on the full temporal grid ($N_t = 150$, $\Delta t = 1$ s) with the same convolutional backbone, the same latent dimension $d_z = 64$, the same two-phase protocol and the same data partition as the Laplace pipelines, the only structural difference being that its surrogate carries one head per time step instead of one per Laplace frequency. Table 3 reports the two stages.

Stage	Median (%)	Mean (%)	p90 (%)	Gap (pp)
Autoencoder (frame-wise reconstruction)	2.49	2.71	3.87	—
End-to-end surrogate	3.80	4.15	6.12	+1.31

Table 3: DL-ROM baseline at full temporal resolution ($N_t = 150$, $d_z = 64$), relative L^2 error on the physical field over the test set ($n = 1620$). *Autoencoder*: each of the 150 frames encoded and decoded, no surrogate and no Laplace round-trip. *Surrogate*: prediction from μ alone through the 150 per-time-step heads. *Gap*: surrogate – autoencoder.

Autoencoder stage. The frame-wise autoencoder reconstructs the field to 2.49% median error (2.71% mean, 3.87% p90). This is the lowest reconstruction error of the entire study, and necessarily so: sharing the backbone, the latent dimension and the training protocol with the Laplace pipelines, it differs from them by the absence of the Laplace round-trip alone. Because nothing else differs, the distance between this floor and theirs prices that round-trip exactly; the comparison is made at the matched configuration in Section 3.3.

Surrogate stage. The end-to-end error distribution over the 1620 held-out simulations (Figure 23, Appendix A) is unimodal and tightly concentrated: median 3.80%, mean 4.15%, p90 6.12%, with a thin upper tail reaching 10%. The regression stage adds 1.31 pp to the reconstruction floor, more than the +0.83 pp of the POD reference, as expected of a target of 150 latent frames against $2 \times 16 \times 64$ POD coefficients. At full temporal resolution the DL-ROM is the most accurate emulator of this study, 57% below the POD reference (8.92%, Section 3.2.1). The direction is the expected one: predicting the latent trajectory at every one of the 150 time steps discards no spectral content, a strictly easier approximation problem than one constrained to $K = 16$ coefficients. The price is on the

cost side, its surrogate carrying one head per time step rather than one per frequency (23.5M parameters, per equation (40)), and in the trajectory being delivered only on the training grid. Sections 4.1–4.3 weigh that accuracy against those costs alongside the Laplace surrogates.

3.3 Autoencoder Reconstruction

Table 4 reports the relative L^2 reconstruction error on the held-out test set (1620 simulations) for all trained AE checkpoints, evaluated on the physical field $U(t)$.

Model	d_z	K	γ	Median (%)	Mean (%)	p90 (%)	vs. POD (pp)
LLAE	128	16	0.01	2.87	3.09	4.34	−4.17
	64	32	0.01	3.12	3.47	5.08	−4.45
	64	16	0.01	3.13	3.41	4.99	−4.96
	64	16	0.0	3.19	3.55	5.21	−5.23
	64	16	0.001	3.24	3.57	5.11	−5.14
	64	8	0.01	3.28	3.59	5.18	−8.73
	32	16	0.01	3.37	3.79	5.55	−8.05
SLAE	64	16	0.0	7.79	8.12	9.92	−0.63
	64	16	0.001	7.83	8.28	10.37	−0.55
	128	16	0.01	7.99	8.41	10.46	+0.95
	64	32	0.01	8.36	9.28	13.46	+0.79
	64	16	0.01	8.93	9.54	12.58	+0.84
	32	16	0.01	10.76	11.15	14.78	−0.66
	64	8	0.01	12.27	12.75	15.40	+0.26

Table 4: Relative L^2 reconstruction error on the test set (1620 simulations). The last column gives the difference in median error relative to the linear POD baseline at matching $(K, k_{\text{pod}} = d_z, \gamma)$: negative values indicate improvement over POD, positive values indicate that POD outperforms the AE. Best per family in bold.

Reconstruction floor. Across all 14 checkpoints the two families contrast starkly. LLAE reaches a best median error of 2.87% ($d_z = 128, K = 16$), a factor of 2.5 below the POD reconstruction at matching configuration (7.04% at $k_{\text{pod}} = 128, K = 16$) and of 2.5–3.5 against the other POD configurations. SLAE yields 7.79%–12.27%, broadly comparable to POD: the *vs. POD* column hovers around zero, positive as often as negative. As a pure compression stage SLAE therefore offers no meaningful gain over the linear reference, though the picture changes substantially at the surrogate stage (Section 3.4).

Price of the Laplace round-trip. The Laplace-free DL-ROM autoencoder of Section 3.2.2 supplies the second reference, and the only one that prices the temporal compression on its own. At the matched configuration ($d_z = 64, K = 16, \gamma = 0.01$) it reconstructs to 2.49% against 3.13% for LLAE, so the latent Laplace round-trip costs 0.64 pp. The two share the backbone, the latent dimension, the training protocol and the data partition, and differ by that round-trip alone; this is therefore the cleanest

measurement of the truncation penalty in the study. It is a modest one, and the best LLAE closes it further, to 0.38 pp at $d_z = 128$ against a DL-ROM trained at half that latent width. The same comparison is severe for SLAE (8.93% against 2.49%): what costs 0.64 pp in latent space costs 6.44 pp in the physical domain, K complex frequency frames being a far more aggressive compression of the field itself than of its latent trajectory.

Why the two families diverge. Two compounding factors explain the gap (Sections 2.2.1–2.2.2). First, the input spaces differ: SLAE compresses high-dimensional complex frequency frames $\hat{U}(s_k)$, whereas LLAE encodes smoother physical frames, an intrinsically easier task. Second, the training objectives are asymmetric: LLAE’s loss (12) includes a direct physical-time reconstruction term that provides dense temporal supervision, while SLAE deliberately omits it during Phase 1 (Section 2.2.1) to preserve its computational advantage.

Hyperparameter sensitivity. The one-at-a-time ablations (Appendix C) separate the two families further. LLAE is insensitive to everything but its bottleneck. K moves the median by 0.16 pp over a fourfold range (3.28, 3.13 and 3.12% at $K = 8, 16$ and 32) and γ by less over $[0, 0.01]$, so 8 to 16 frequencies already carry the dominant transient dynamics. The latent width d_z still pays at the top of its range ($3.37 \rightarrow 2.87\%$ from 32 to 128) and has therefore not saturated at $d_z = 64$. SLAE binds on both bottlenecks at once, degrading sharply at $K = 8$ (12.27%) and at $d_z = 32$ (10.76%); raising K to 32 does not help either (8.36% against 7.79% at $K = 16$), the higher frequencies carrying less spatial energy while the spatial bottleneck stays where it is. Its error distribution also carries the heavier right tail of the two.

Per-frequency breakdown. For SLAE, Figure 6 breaks the reconstruction error down per Laplace frequency k . The error is lowest at low frequencies ($k = 0\text{--}3$, where most signal energy is concentrated) and rises monotonically toward higher frequencies, confirming that the spatial compression bottleneck is more severe for the fine-scale, high-frequency content. Against the POD per-frequency breakdown (Figure 28), SLAE achieves roughly half the per-frequency error of POD at each k , the convolutional encoder capturing non-linear structure the linear projection cannot. This advantage does not translate into a better overall spatial reconstruction (Table 4), because the inverse transform (3)–(4) damps all frequencies uniformly and partially cancels the benefit before the spatial field is recovered: the global spatial error is a *regularisation-weighted* aggregate of the per-frequency errors, not a direct reflection of per-frequency compression quality.

Inference cost. Table 5 reports the per-sample cost of a full reconstruction along three axes: arithmetic work (FLOPs, counted on the executed graph), wall-clock latency (batch size 1, 50 timed CUDA Event runs after 10 warm-up runs), and peak GPU memory. FLOPs measure the cost of the *architecture*, independently of how well a given implementation exploits the hardware; latency is what a user experiences, and the two do not agree here. The arithmetic follows the architectural prediction of Sections 2.2.1–2.2.2 exactly: SLAE decodes K frequency frames, so it scales linearly with K (15.5, 31.1 and 62.2 GFLOPs at $K = 8, 16$ and 32), whereas LLAE decodes all $N_t = 150$ time frames regardless and stays flat at 54.2 GFLOPs across a fourfold variation of K . At $K = 16$ SLAE performs $1.74\times$ fewer operations, $3.5\times$ fewer at $K = 8$, the ordering reversing only above $K \approx 28$.

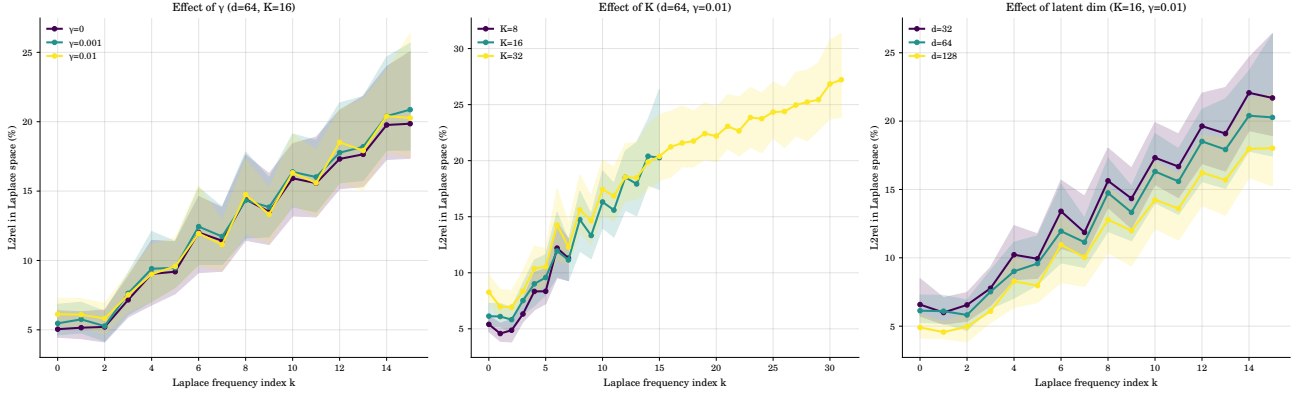


Figure 6: SLAE: per-frequency reconstruction error in the Laplace domain (median $\pm$ IQR, 200 test samples). Left: effect of γ ; centre: effect of K ; right: effect of d_z .

The wall-clock gain is smaller than the arithmetic one, $1.2\times$ at $K = 16$ (22.7 ± 1.1 against 27.0 ± 1.7 ms), because reconstruction from a physical field also pays the forward transform (1) and its regularised inverse, a fixed cost the FLOP ratio does not see and which the surrogate does not pay at all (Section 3.4.1). That same fixed cost governs the memory axis, the one on which the two families are comparable here (518 against 556 MB at $K = 16$): holding the entire physical field in memory to transform it is what brings SLAE up to LLAE’s level, and the surrogate, which never sees a physical time frame, recovers a $2.6\times$ advantage. The two regimes remain distinct underneath, SLAE’s allocation being set by K (404 to 794 MB from $K = 8$ to 32) and LLAE’s by N_t (within 3% of 556 MB throughout).

Model	K	d_z	GFLOPs	Latency (ms)	Peak GPU (MB)
SLAE	8	64	15.5	18.4 ± 1.3	404
SLAE	16	32	31.0	22.7 ± 1.1	510
SLAE	16	64	31.1	22.7 ± 1.1	518
SLAE	16	128	31.2	22.7 ± 1.0	534
SLAE	32	64	62.2	27.9 ± 1.4	794
LLAE	8	64	54.2	26.8 ± 2.1	556
LLAE	16	32	53.7	26.9 ± 1.8	550
LLAE	16	64	54.2	27.0 ± 1.7	556
LLAE	16	128	55.2	27.2 ± 2.0	568
LLAE	32	64	54.2	27.1 ± 1.9	556

Table 5: Reconstruction cost at $\gamma = 0.01$. GFLOPs are counted on the executed graph for one sample. Latency: batch size 1, 50 timed CUDA Event runs after 10 warm-up runs, median $\pm$ standard deviation. Peak GPU memory is the maximum allocated during one reconstruction. Best per column in bold.

3.4 Surrogate Prediction

3.4.1 Direct Surrogates

Table 6 reports the relative L^2 error on the held-out test set (1620 simulations) for all 14 end-to-end surrogate checkpoints, evaluated on the physical field $U(t)$. All three families (POD, SLAE, LLAE) use the same surrogate MLP architecture (Appendix G), so the comparison isolates the spatial compression strategy rather than the predictor capacity. Two columns aid interpretation: *Gap*, the difference between the surrogate median and the standalone AE median of Table 4 (positive for surrogate overhead, negative when joint fine-tuning of the decoder *improves* upon the frozen-AE baseline), and *vs POD*, the difference to the matching POD surrogate of Table 2 at the same (K, d_z, γ) , negative throughout.

Model	d_z	K	γ	Median (%)	Mean (%)	p90 (%)	Gap (pp)	vs POD (pp)
LLAE	128	16	0.01	3.84	4.12	6.03	+0.97	−7.46
	64	32	0.01	4.33	4.63	6.79	+1.21	−4.42
	64	16	0.01	4.06	4.37	6.47	+0.93	−4.86
	64	16	0.0	4.34	4.68	6.93	+1.15	−4.64
	64	16	0.001	4.07	4.37	6.48	+0.83	−4.88
	64	8	0.01	4.01	4.34	6.42	+0.73	−8.44
	32	16	0.01	4.34	4.71	7.08	+0.97	−7.38
SLAE	64	16	0.0	7.39	7.53	8.81	−0.40	−1.59
	64	16	0.001	7.43	7.57	8.90	−0.40	−1.52
	128	16	0.01	7.50	7.67	9.18	−0.49	−3.80
	64	32	0.01	5.70	6.07	7.98	−2.66	−3.05
	64	16	0.01	7.59	7.87	9.50	−1.34	−1.33
	32	16	0.01	7.62	7.85	9.49	−3.14	−4.10
	64	8	0.01	11.62	11.89	13.84	−0.65	−0.83

Table 6: Direct surrogate results on the test set (1620 simulations), same row order as Table 4. *Gap*: surrogate median – AE reconstruction median; negative values indicate that end-to-end fine-tuning improves upon the standalone AE. *vs POD*: surrogate median – matching POD surrogate median (Table 2). Best per family in bold.

Overall accuracy. LLAE achieves 3.84–4.34% median L^2 error across all configurations, compared to 5.70–11.62% for SLAE. The best LLAE surrogate ($d_z = 128$, $K = 16$, $\gamma = 0.01$, 3.84%) represents a 57% reduction in median error relative to the POD reference (8.92%), while the best SLAE surrogate ($d_z = 64$, $K = 32$, $\gamma = 0.01$, 5.70%) achieves a 36% reduction.

End-to-end fine-tuning effect. The Gap column reveals a qualitative asymmetry between the two families. For LLAE, the surrogate step introduces a small positive overhead of +0.73 to +1.21 pp above the AE reconstruction floor: the surrogate approximation is not perfectly lossless, but the cost is modest given that the AE floor itself already saturates below 3.5%.

For SLAE, by contrast, the Gap is *negative* across all configurations, from -0.40 to -3.14 pp: the end-to-end pipeline *outperforms* the standalone SLAE autoencoder. The mechanism is that the Phase-1 loss $\mathcal{L}_{\text{SLAE}}$ (equation (10)) is defined purely in the Laplace domain, the decoder never being supervised on physical-time reconstruction, since such a loss would require N_t -frame decoder passes and negate SLAE’s speed advantage. Phase 2 introduces that term for the first time through $\mathcal{L}_{\text{total}}$ (Section 2.3) and fine-tunes the decoder through it, correcting residual errors the Laplace-domain loss could not resolve. The effect is largest where the AE floor is highest: at $d_z = 32$ (-3.14 pp) and $K = 32$ (-2.66 pp).

Hyperparameter sensitivity. LLAE surrogates remain highly robust: varying K from 8 to 32, d_z from 32 to 128, or γ from 0 to 0.01 all yield errors in a tight 3.84–4.34% band, consistent with the insensitivity already observed at the AE level (Section 3.3). SLAE is more sensitive: $K = 8$ collapses to 11.62% (the spectral truncation is too severe for the surrogate to compensate), while $K = 32$ is the best SLAE configuration precisely because the decoder, given access to 32 Laplace frequencies, can recover finer spatial structure during fine-tuning. Ablation violin plots are provided in Appendix D.

Where the error lives in the design. The parameter design is factorial, 15 values of k crossed with 15 of C and 36 injection angles ϕ , the angle being a numerical rotation of the same base solution (Section 3.1), so an aggregate median mixes three directions of unequal difficulty. Table 7 decomposes the variance of the per-instance error ε by the correlation ratio

$$\eta_X^2 = \frac{\text{Var}(\mathbb{E}[\varepsilon \mid X])}{\text{Var}(\varepsilon)}, \quad X \in \{\phi, k, C\}. \quad (43)$$

The angle explains almost nothing, $\eta_\phi^2 = 0.02$ – 0.03 for every checkpoint of both families, against 0.13–0.26 for k and 0.53–0.74 for C : the error is flat over the 36 rotations and is carried by the two physical parameters, C foremost. The maps of Figure 7 locate it, the upper decile being confined to the low- C , low- k corner where reaction and diffusion are slowest and the front takes longest to establish, an identifiable physical regime rather than dispersion.

Checkpoint	η_ϕ^2	η_k^2	η_C^2
LLAE $d_z = 128, K = 16$	0.027	0.264	0.529
LLAE $d_z = 64, K = 16$	0.028	0.258	0.572
LLAE $d_z = 64, K = 8$	0.028	0.242	0.568
LLAE learnable, $K = 8$	0.025	0.185	0.690
LLAE learnable, $K = 16$	0.022	0.136	0.737
LLAE learnable, $K = 32$	0.021	0.127	0.755

Table 7: Correlation ratio η^2 of the per-instance relative L^2 error against each design factor, on the test set ($n = 1620$). k and C are binned into eight quantiles; ϕ is used at its 36 levels. Values sum to less than one because the factors do not exhaust the variance.

Inference cost. At the surrogate stage the arithmetic and wall-clock orderings coincide, the forward transform being absent: the network emits $\hat{U}(s_k)$ directly and only the

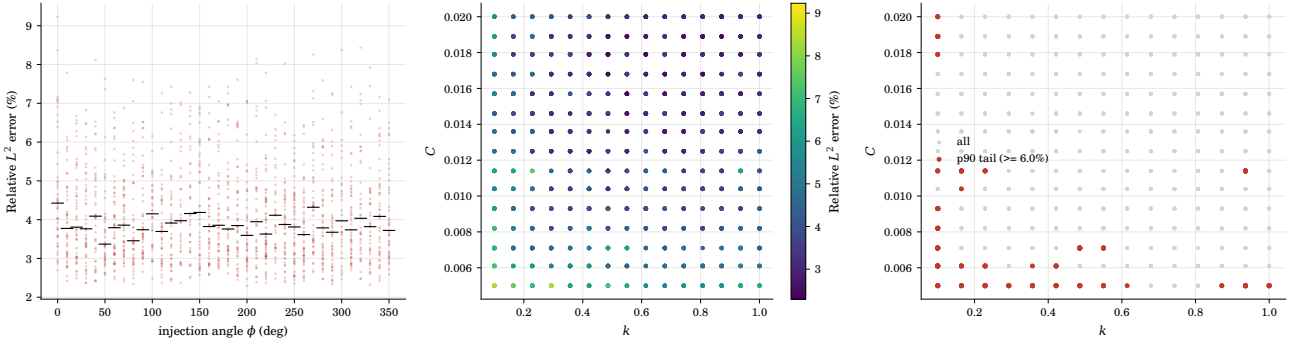


Figure 7: Error over the design of experiments for the best LLAE surrogate. Left: error against the injection angle ϕ , median per angle in black. Centre: error over the physical plane (k, C) , colour encoding the relative L^2 error. Right: location of the upper decile.

inversion is evaluated, so the fixed overhead that attenuated the reconstruction ratios (Section 3.3) no longer applies. At batch size 1 and $K = 16$, SLAE requires 30.3 GFLOPs, 15.6 ± 0.3 ms and 235 MB against 48.2 GFLOPs, 22.0 ± 4.2 ms and 622 MB for LLAE, factors of 1.59, 1.41 and 2.6. The decoder, not the predictor, governs the cost. LLAE’s arithmetic is invariant at 48.2 GFLOPs for $K \in \{8, 16, 32\}$, whereas SLAE scales from 15.2 to 60.7 GFLOPs and crosses LLAE within the sweep. SLAE’s latency dispersion is also an order of magnitude smaller (0.1–0.7 against 2.8–4.2 ms), the K frequency frames being decoded in a single batch. The POD surrogate bounds the attainable gain: 0.2 GFLOPs, some 150 $\times$ fewer than SLAE, yet 3.5 ms on 212 MB. A 150-fold reduction in arithmetic thus buys a factor of 4.5 in wall-clock time and 1.1 in memory, the residual cost being dominated by the Tikhonov solve and the back-projection through $V^{(k)}$. The regularised inversion is thus an incompressible component of every Laplace-domain pipeline at this problem size. Figure 8 details the 14 checkpoints.

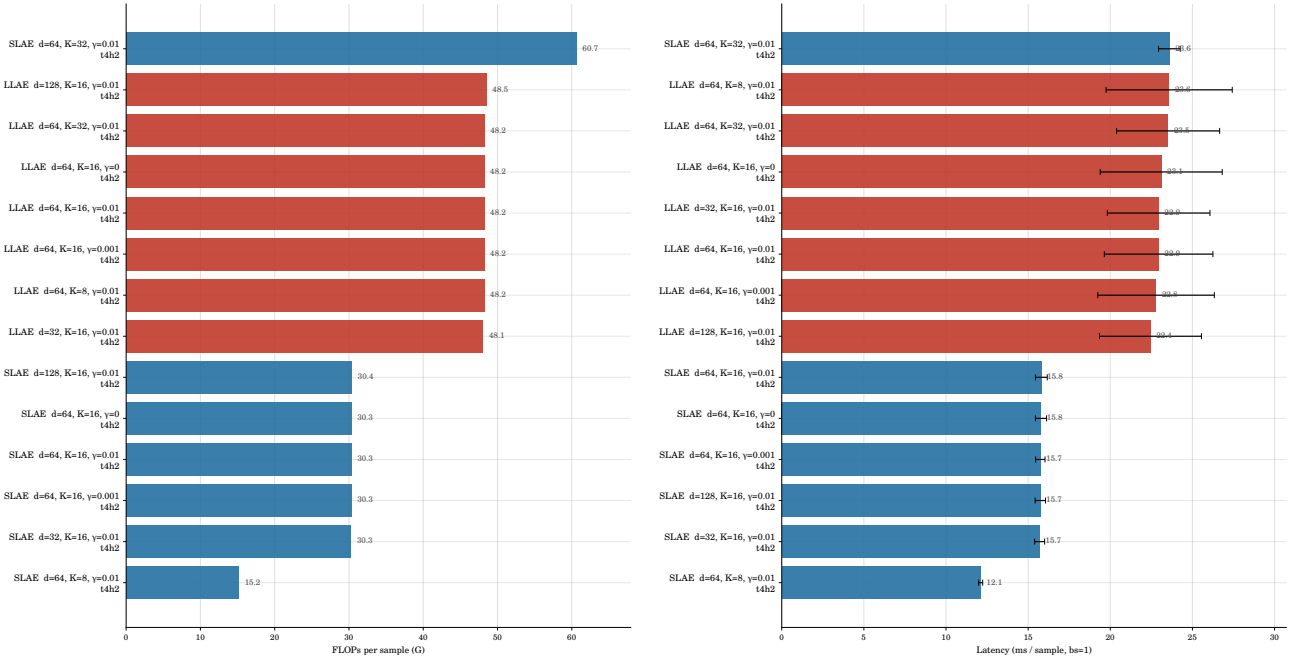


Figure 8: End-to-end surrogate cost for the 14 checkpoints (batch size 1, 50 timed CUDA Event runs). Left: arithmetic work per sample. Right: wall-clock latency.

Per-checkpoint error histograms for all 14 surrogate checkpoints are provided in Appendix A (Figure 22).

3.4.2 SVD Surrogates: Single-Mode Latent Compression

The direct surrogate of Section 3.4.1 predicts the full d_z -dimensional latent at each frequency. To test whether compressing that representation first improves the predictor’s accuracy, we assess a hybrid approach inspired by Rank Reduction Autoencoders (RRAE) [30]: projecting the latent trajectories onto a truncated SVD basis before the surrogate. We evaluate 26 AE+SVD checkpoints across the LLAE and SLAE families; the encoder is frozen while the decoder and the basis V are trained end-to-end alongside the surrogate, the latent dimension being reduced to $k_{\text{svd}} \in \{16, 32\}$ before an MLP predicts the reduced coefficients from μ .

Table 8 reports the relative L^2 error on the held-out test set evaluated on the physical field $U(t)$.

Does compressing the latent space help? The central question is whether predicting on a smaller, orthogonalised latent space simplifies the MLP’s task enough to offset the information lost to SVD truncation. It does not. Measured *at matching autoencoder configuration*, the SVD bottleneck adds a small penalty over the direct surrogate for both families: fixing $d_z = 64$, $\gamma = 0.01$, the best LLAE-SVD ($K = 16$, $k_{\text{svd}} = 32$, 4.24%) sits +0.18 pp above its direct counterpart (4.06%), and the best SLAE-SVD ($K = 32$, $k_{\text{svd}} = 32$, 6.03%) +0.33 pp above its own (5.70%). The MLP navigates the full d_z -dimensional latent space perfectly well, so the SVD projection merely discards spatial detail the direct surrogate had already learned to predict: compressing the latent before the surrogate degrades the reconstruction rather than helping it.

Impact of k_{svd} . Comparing pairs of configurations differing only in k_{svd} , Figure 9 shows that $k_{\text{svd}} = 32$ systematically improves accuracy (all points below the $y = x$ diagonal). The gain is markedly larger for LLAE (+1.06 to +1.94 pp reduction in median error) than for SLAE (+0.33 to +0.93 pp), indicating that LLAE latents carry more fine-grained information in the additional 16 modes, whereas SLAE discards spatial detail already at the AE stage.

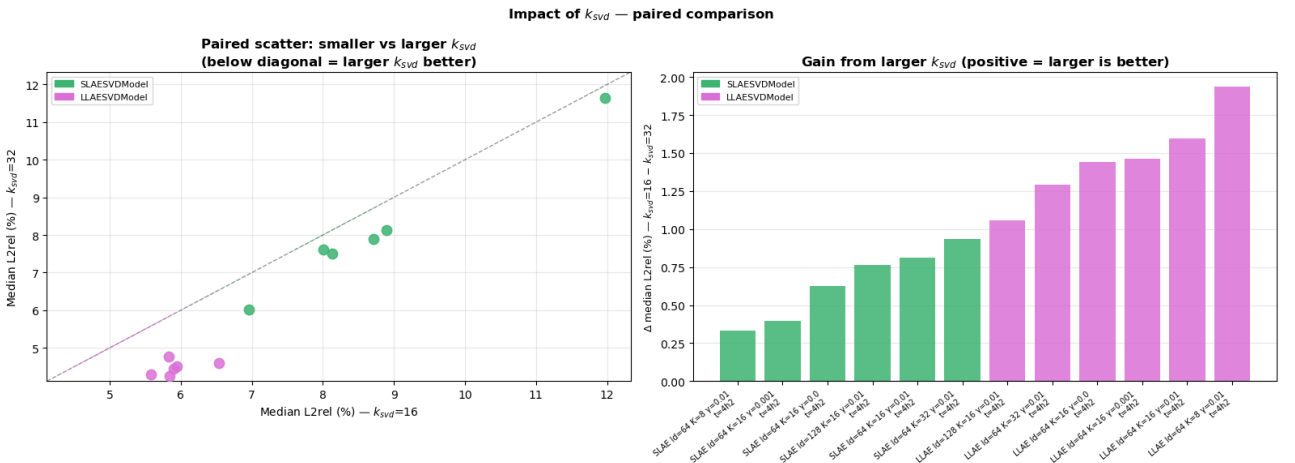


Figure 9: Impact of k_{svd} on SVD surrogate performance. Left: paired median errors, $k_{\text{svd}} = 16$ against $k_{\text{svd}} = 32$. Right: absolute error reduction from 16 to 32 modes, sorted by magnitude.

Model	d_z	K	γ	k_{svd}	Median (%)	Mean (%)	p90 (%)
LLAE	64	16	0.01	32	4.24	4.61	6.75
	64	32	0.01	32	4.29	4.69	7.02
	64	16	0.001	32	4.44	4.96	7.41
	64	16	0.0	32	4.50	4.97	7.29
	64	8	0.01	32	4.60	5.04	7.47
	128	16	0.01	32	4.77	5.30	7.82
	32	16	0.01	16	5.55	6.29	9.39
	64	32	0.01	16	5.58	6.22	9.35
	128	16	0.01	16	5.83	6.51	9.48
	64	16	0.01	16	5.84	6.58	9.79
	64	16	0.001	16	5.90	6.72	9.92
	64	16	0.0	16	5.94	6.76	10.18
	64	8	0.01	16	6.54	7.37	11.05
SLAE	64	32	0.01	32	6.03	6.58	9.02
	64	32	0.01	16	6.96	7.80	11.24
	64	16	0.0	32	7.50	7.73	9.26
	64	16	0.001	32	7.62	7.88	9.52
	64	16	0.01	32	7.90	8.32	10.54
	32	16	0.01	16	7.95	8.38	10.71
	64	16	0.001	16	8.01	8.43	10.80
	128	16	0.01	32	8.12	8.57	10.98
	64	16	0.0	16	8.13	8.61	11.06
	64	16	0.01	16	8.71	9.40	12.68
	128	16	0.01	16	8.89	9.71	13.43
	64	8	0.01	32	11.64	11.93	13.96
	64	8	0.01	16	11.97	12.38	14.88

Table 8: AE+SVD surrogate results on the test set ($n = 1620$), evaluating the physical field reconstruction error using LLAE and SLAE autoencoders coupled with an intermediate SVD compression ($k_{\text{svd}} \in \{16, 32\}$). Best median per family in bold.

Hyperparameter sensitivity. Sensitivity to the AE hyperparameters K , d_z , and γ mirrors the direct case (Section 3.4.1): SLAE again degrades sharply at $K = 8$ (11.6–12.0%), while LLAE stays within a tight band. The lever specific to this pipeline is k_{svd} , whose effect is quantified above: retaining 32 rather than 16 modes is uniformly beneficial, and more so for LLAE than for SLAE.

Inference latency. The SVD back-projection is inserted between the MLP and the spatial decoder, and costs two small matrix products. At batch size 1, LLAE-SVD requires approximately 22 ms per sample against about 9 ms for SLAE-SVD, which retains a $\approx 2.4\times$ speed advantage from its lighter decoder. Figure 10 presents the latency–accuracy trade-off.

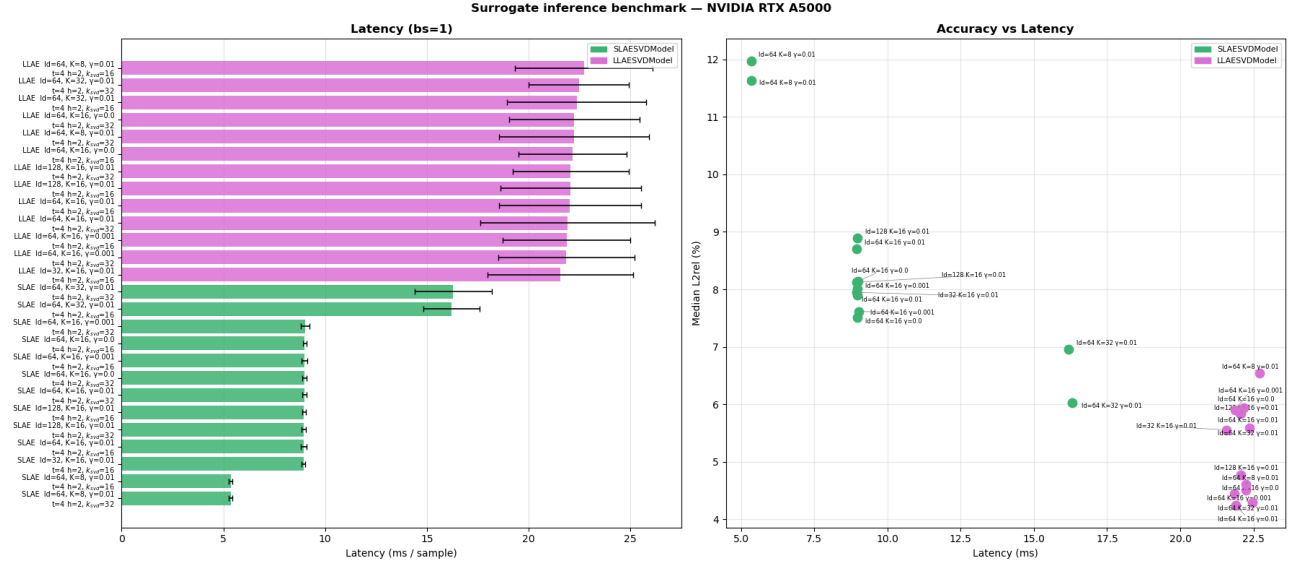


Figure 10: AE+SVD surrogate inference benchmark (batch size 1). Left: per-sample latency across checkpoints. Right: accuracy–latency scatter representing median L^2 error vs. latency.

3.4.3 Tucker Surrogates: Joint-Mode Latent Compression

Section 3.4.2 compressed the latent representation along a single mode. We now evaluate the Tucker variant of Section 2.3, which factorises the frequency and latent modes *jointly*: the surrogate predicts a compact core matrix $\hat{\mathcal{G}} \in \mathbb{C}^{r_s \times r_z}$ (real-valued for SLAE) instead of K independent latents of dimension d_z , back-projecting through the frozen HOOI factors $U^{(s)}, U^{(z)}$ before decoding. We sweep the Tucker ranks over a grid $r_s \in \{4, 8\}$ (frequency mode, $K = 16 \rightarrow r_s\}) \times r_z \in \{8, 16, 32\}$ (latent mode, $d_z = 64 \rightarrow r_z$), for both families, yielding 12 checkpoints. All share the same Phase 1 autoencoder ($d_z = 64$, $K = 16$, $\gamma = 0.01$); only the offline factors and the surrogate differ. The compression ratio $\rho = r_s r_z / (K d_z)$ measures the reduction of the surrogate target dimension relative to the direct pipeline (which predicts $K d_z = 1024$ scalars per sample).

Beyond the physical-field error $\varepsilon_{\text{field}}$ (relative L^2 on $U(t)$, full pipeline), we report two decoder-free diagnostics computed in the latent space, which split the difficulty between the offline compression and the surrogate MLP:

$$\varepsilon_{\text{trunc}} = \frac{\|\hat{Z} - \Pi \hat{Z}\|}{\|\hat{Z}\|}, \quad \varepsilon_{\text{reg}} = \frac{\|\hat{\mathcal{G}}_{\text{pred}} - \hat{\mathcal{G}}_{\text{true}}\|}{\|\hat{\mathcal{G}}_{\text{true}}\|}, \quad (44)$$

where $\Pi = U^{(s)}U^{(s)\top}(\cdot)U^{(z)}U^{(z)\top}$ is the Tucker projector. $\varepsilon_{\text{trunc}}$ measures the energy discarded by the rank truncation (factors only, independent of the surrogate), while ε_{reg} measures how well the MLP predicts the true core (surrogate only, independent of the truncation). Table 9 reports all three on the held-out test set.

Model	r_s	r_z	ρ	Median (%)	Mean (%)	p90 (%)	$\varepsilon_{\text{trunc}}$ (%)	ε_{reg} (%)	Lat (ms)
LLAE	8	32	0.250	4.33	4.75	7.16	36.98	7.25	18.2
	4	32	0.125	4.57	5.04	7.61	40.34	5.67	18.0
	8	16	0.125	4.87	5.43	8.11	61.86	4.38	18.3
	4	16	0.062	5.16	5.77	8.70	61.59	3.83	18.0
	8	8	0.062	6.85	7.78	11.75	81.21	4.99	18.3
	4	8	0.031	6.87	8.02	12.73	81.03	3.80	18.0
SLAE	8	32	0.250	7.92	8.35	10.55	42.40	6.76	7.4
	4	32	0.125	8.15	8.68	11.14	50.82	6.58	7.1
	4	16	0.062	8.68	9.36	12.79	69.59	5.18	7.1
	8	16	0.125	8.85	9.51	12.89	67.59	5.83	7.5
	4	8	0.031	9.93	10.92	15.61	80.63	5.70	7.1
	8	8	0.062	9.96	11.05	16.00	80.05	7.16	7.5

Table 9: Tucker surrogate results on the test set ($n = 1620$), sorted by median error within each family. $\rho = r_s r_z / (K d_z)$ is the compression ratio of the surrogate target relative to the direct pipeline. $\varepsilon_{\text{trunc}}$: median latent-energy discarded by the rank truncation (factors only). ε_{reg} : median relative error of the predicted core (surrogate only). *Lat*: GPU inference latency at batch size 1. Best median per family in bold.

Overall accuracy. As with SVD, the relevant question is what the extra compression costs relative to the direct and single-mode pipelines *at matching autoencoder configuration*. Fixing the shared AE ($d_z = 64$, $K = 16$, $\gamma = 0.01$), the median error grows only gently along the compression axis: $4.06\% \rightarrow 4.24\% \rightarrow 4.33\%$ for LLAE (direct $\rightarrow$ SVD with $k_{\text{svd}} = 32 \rightarrow$ Tucker with $r_s = 8$, $r_z = 32$), and $7.59\% \rightarrow 7.90\% \rightarrow 7.92\%$ for SLAE. Tucker thus adds +0.27 pp over the direct surrogate for LLAE and +0.33 pp for SLAE, and at most +0.09 pp over SVD, while shrinking the surrogate target to $r_s r_z = 256$ scalars, a $4\times$ reduction over the direct target ($K d_z = 1024$) and a $2\times$ reduction over the SVD target ($K k_{\text{svd}} = 512$). Factorising both modes at once is what buys this: exploiting cross-frequency correlations lets Tucker match the single-mode SVD accuracy at half the target dimension.

Rank sensitivity: r_z dominates r_s . The rank grid of Table 9 is strongly anisotropic, and this is the most informative outcome of the sweep. Increasing the latent-mode rank r_z from 8 to 32 nearly halves the LLAE error ($6.85\% \rightarrow 4.33\%$ at $r_s = 8$), and the SLAE error falls in step ($9.96\% \rightarrow 7.92\%$). Doubling the frequency-mode rank r_s from 4 to 8, by contrast, barely moves either ($4.57\% \rightarrow 4.33\%$ for LLAE, $8.15\% \rightarrow 7.92\%$ for SLAE at $r_z = 32$), an effect an order of magnitude smaller. The rank sweeps of Figure 11 confirm the pattern distributionally: the r_z violins separate cleanly while the r_s violins overlap almost completely.

The interpretation reaches beyond this ablation. A rank-4 basis over the frequency mode already captures almost all of the joint variation of the $K = 16$ Laplace coefficients across the training set: the spectral representation is redundant enough that discarding three quarters of its dimensions costs nothing measurable. The $d_z = 64$ latent channels behave in the opposite way, carrying richer, less redundant information that a rank-8 basis truncates too aggressively. A tensor factorisation, fitted blind to the physics on training statistics alone, thus rediscovers the premise of this work from a direction independent of the argument that motivated the transform: the Laplace representation is intrinsically compact along the frequency axis, and $K = 16$ already sits close to its useful floor.

In practice $r_s = 4$ is the parameter-efficient choice, halving the surrogate size (1.35 M vs. 1.9 M parameters for $r_s = 8$) at negligible accuracy cost. One caveat bounds the claim: the frequency axis is sampled at two points only ($r_s \in \{4, 8\}$), so the sweep establishes insensitivity to r_s over that range, not saturation. The uncompressed limit $r_s = K$ is structurally the single-mode compression of Section 3.4.2, the two differing only in the nature of the basis, real and trained there, complex and frozen here.

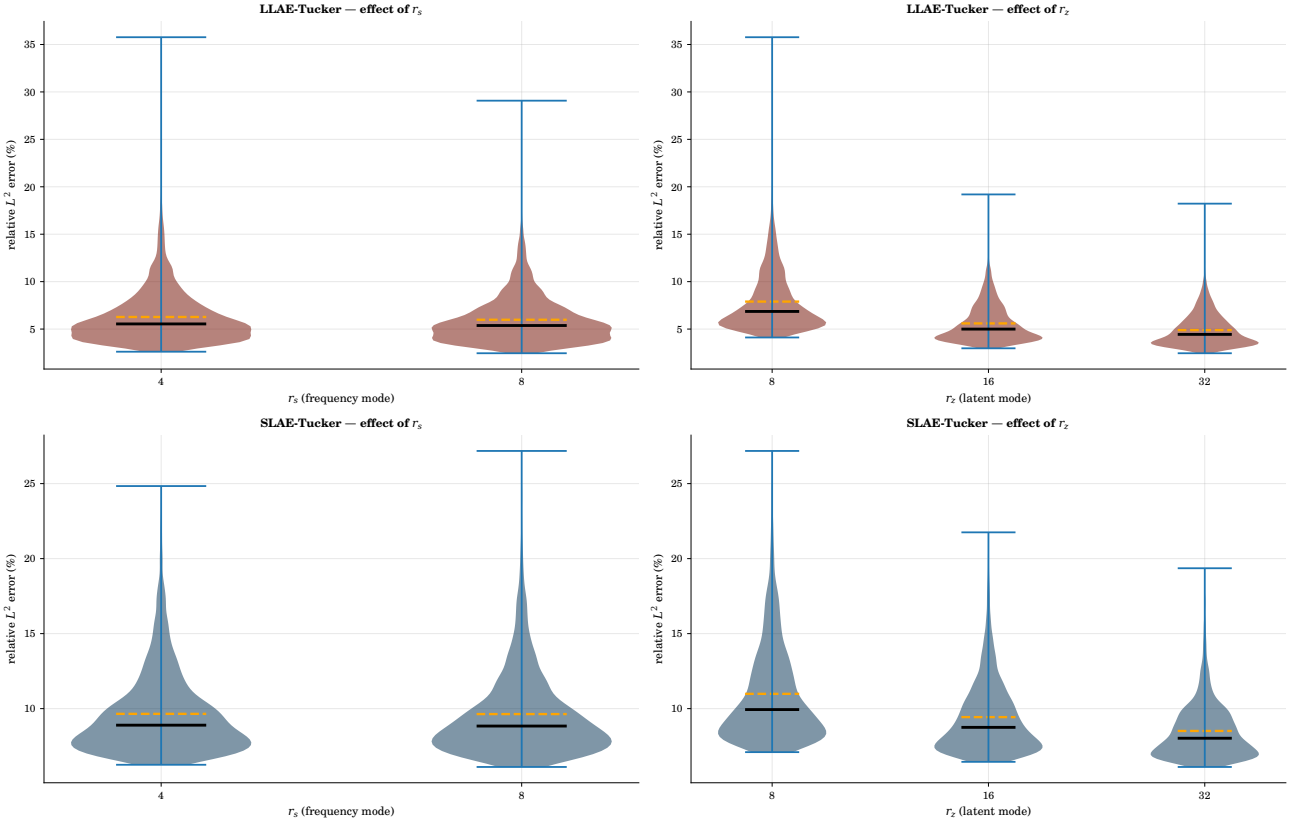


Figure 11: Tucker rank sweeps (test set, violin plots pooled over the other rank; black line = median, dashed orange = mean). Top row LLAE-Tucker, bottom row SLAE-Tucker; left column effect of r_s , right column effect of r_z .

Truncation vs regression. The two decoder-free diagnostics of Table 9, shown in Figure 12, locate the accuracy bottleneck. The regression error ε_{reg} stays small throughout (3.8–7.3%) and grows only mildly as the core enlarges ($r_z = 32$ being harder to fit than $r_z = 8$): the MLP predicts the compact core reliably. The truncation error $\varepsilon_{\text{trunc}}$ is large and is what the field error tracks, falling from $\sim 81\%$ at $r_z = 8$ to $\sim 37\%$ at $r_z = 32$ (LLAE) in step with the field error. It is a *latent-space energy*, not a field error: discarding 80% of the latent norm costs only $\sim 7\%$ on $U(t)$, the removed Tucker directions carrying little

decodable information, so its *variation* with the ranks rather than its absolute value governs accuracy. The Tucker compression, not the surrogate, is therefore the binding constraint, and any accuracy gain must come from a richer latent basis (larger r_z) rather than a larger predictor.

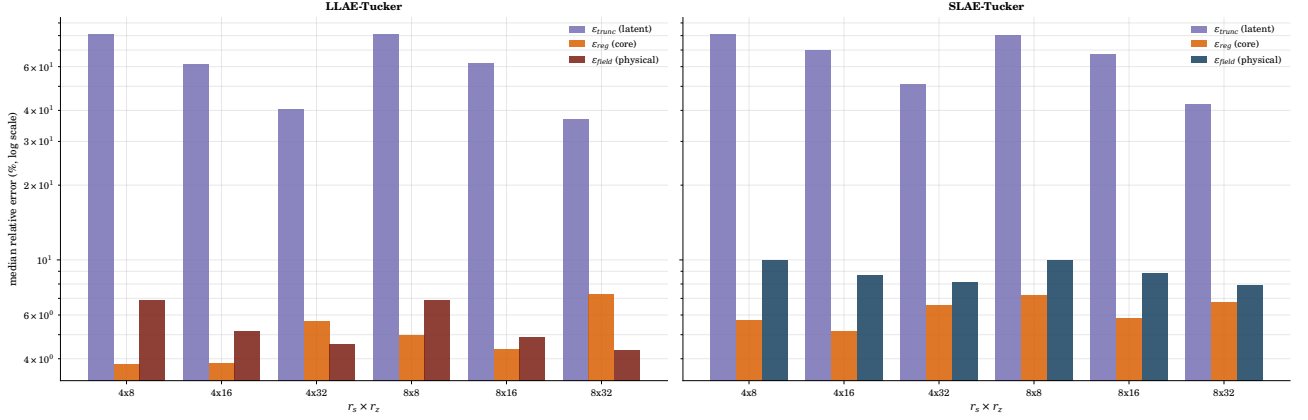


Figure 12: Error decomposition for LLAE-Tucker (left) and SLAE-Tucker (right): median latent truncation error ϵ_{trunc} , core regression error ϵ_{reg} , and physical-field error ϵ_{field} for each (r_s, r_z) pair (log scale).

Compression trade-off. Figure 13 plots the field error against the compression ratio ρ and against the surrogate size. Along the Pareto front, LLAE-Tucker degrades gracefully as the target is compressed: $r_s = 4$, $r_z = 32$ retains 4.57% median error at $\rho = 0.125$ (an $8\times$ smaller target than the direct surrogate), and $r_s = 4$, $r_z = 16$ still reaches 5.16% at $\rho = 0.062$ ($16\times$). Tucker therefore offers a tunable accuracy/compactness dial that the direct and SVD surrogates lack: when the surrogate output must be kept small, whether for storage, for a lightweight predictor or for downstream many-query use, Tucker sustains near-direct accuracy at a fraction of the target dimension. It does not, however, *beat* the direct surrogate on raw accuracy; as with SVD (Section 3.4.2), reducing the latent space before the surrogate discards information the direct predictor is capable of learning.

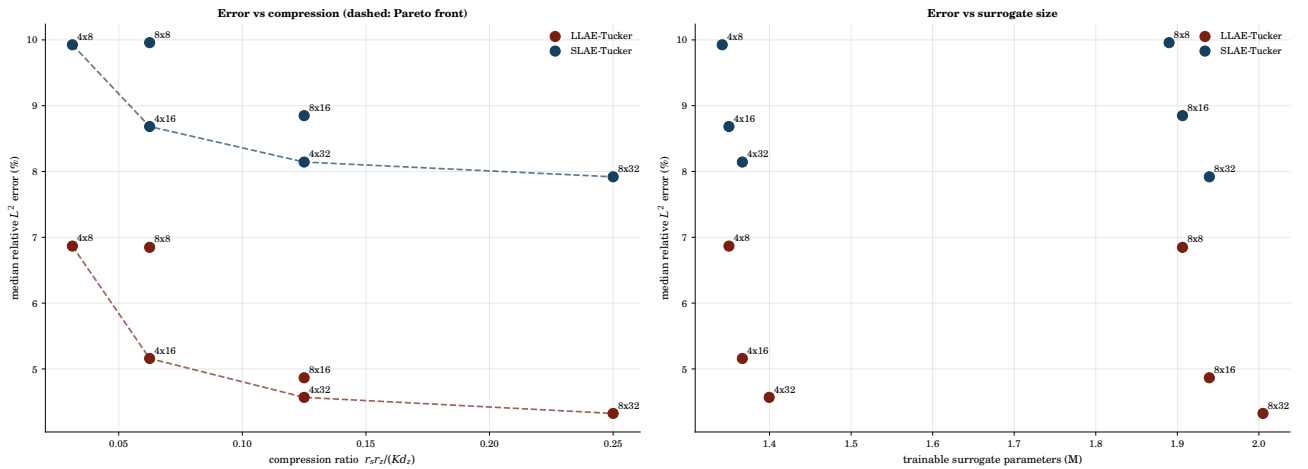


Figure 13: Left: median field error vs. compression ratio $\rho = r_s r_z / (K d_z)$ (dashed line = Pareto front per family; points sharing a ratio differ in (r_s, r_z)). Right: median field error vs. trainable surrogate parameters.

Decoder co-adaptation. Because the decoder is jointly updated during surrogate training ($\text{lr_decoder}=5 \times 10^{-5}$), it can co-adapt to the distribution of *predicted* cores rather than the true ones. We probe this by decoding the exact projected core in place of the prediction: had the decoder not co-adapted, the exact core would yield a lower error and serve as a valid oracle bound. For LLAE-Tucker the substitution is neutral (+0.07 pp) and the bound holds; for SLAE-Tucker it *raises* the error sharply (+11.1 pp on the probe subsample at $r_s = 8, r_z = 16$), the decoder having specialised to the predicted-core distribution, exactly the mechanism behind SLAE’s negative surrogate Gap in Section 3.4.1. A physical-space oracle is therefore not a valid lower bound for SLAE-Tucker, which is why the diagnostics above are computed in the latent space, where they remain decoder-independent.

Inference cost. The Tucker back-projection ($U^{(s)} \hat{\mathcal{G}} U^{(z)\top}$, two small matrix products) is negligible in the inference budget, and the measurements make this unusually clear. Across the twelve checkpoints the FLOP count is *constant* within each family (30.3 GFLOPs for every SLAE-Tucker and 48.2 for every LLAE-Tucker), although the ranks vary by a factor of eight in r_z and two in r_s . Compressing the surrogate target changes what the predictor must learn, not what the decoder must compute. Latency follows suit (Figure 14): ≈ 7.4 ms for SLAE-Tucker and ≈ 18.2 ms for LLAE-Tucker at batch size 1, each within 3% across the six checkpoints of its family and therefore essentially independent of the ranks. SLAE-Tucker thus keeps the $1.6\times$ arithmetic advantage of its lighter decoder, and a $2.5\times$ wall-clock one, at the cost of the higher error floor discussed above. The rank reduction, whose entire purpose was to lower cost, buys none.

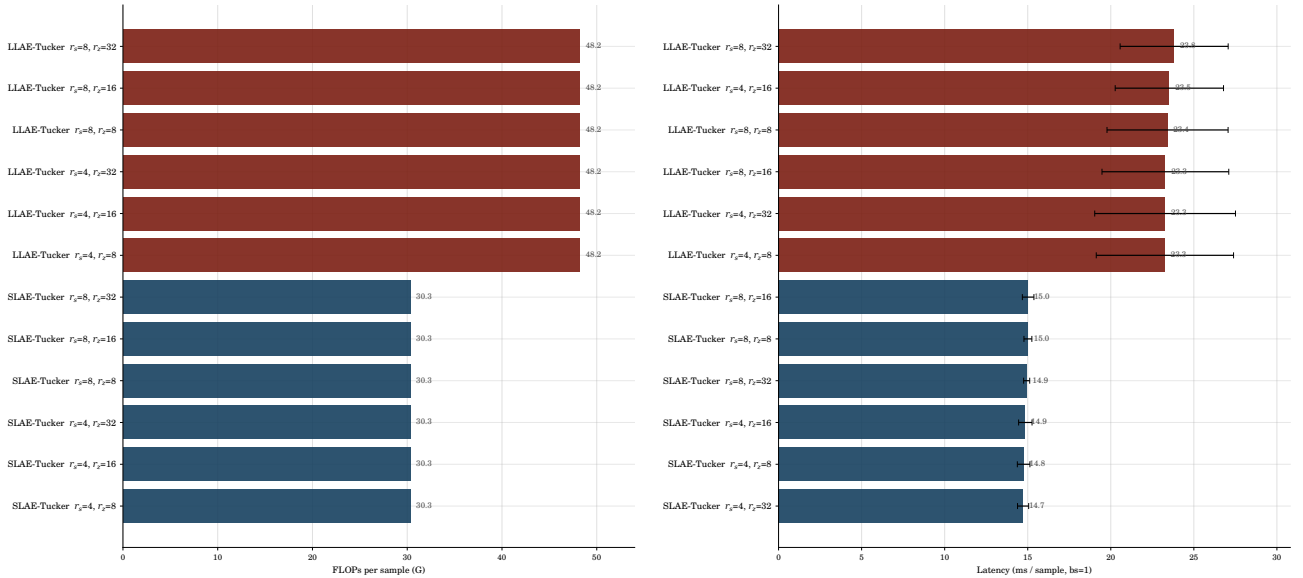


Figure 14: Tucker surrogate inference benchmark (50 CUDA Event timed runs, batch size 1). Left: per-checkpoint latency. Right: accuracy–latency scatter.

Per-checkpoint error histograms for all 12 Tucker surrogates are provided in Appendix A (Figure 25).

3.5 Optimising the Inversion

Every result reported so far rests on the prescribed triplet (s_k, α_t, λ) of Section 2.4. Six configurations are trained against it, one per (family, K) pair with $K \in \{8, 16, 32\}$ and $d_z = 64$: LLAE with the latent learnable contour, SLAE with the offline spatial one.

Architecture, optimiser, schedule and data partition are held at the values of Sections 3.3–3.4.1, so each configuration differs from its prescribed counterpart by the triplet alone. The offline triplet is selected before training and then held fixed throughout both phases, defining the Laplace-domain dataset, the inversion inside the surrogate and the inversion at inference alike; its minimisation settings are given in Appendix G. The latent variant is treated in Appendix E and the discussion below concerns SLAE.

Geometry of the optimised contour. Figure 15 shows the optimised sampling points against their prescribed counterparts, the bottom row being the offline contour. Its abscissae leave the imaginary axis for a positive real part ($\sigma_k \approx 0.035$), yielding a genuine Bromwich contour rather than a purely oscillatory one, while the imaginary parts dilate and widen the sampled spectral window. It also retains a heavy ridge, $\lambda \approx 4.3 \times 10^{-3}$ at $K = 16$ rising to 8.9×10^{-2} at $K = 32$, which is what the criterion is built to do: (30) charges the amplification factor $\|A^{-1}F^H\|$ against a nominal predicted-coefficient error, so conditioning is worth buying with bias at that magnitude.

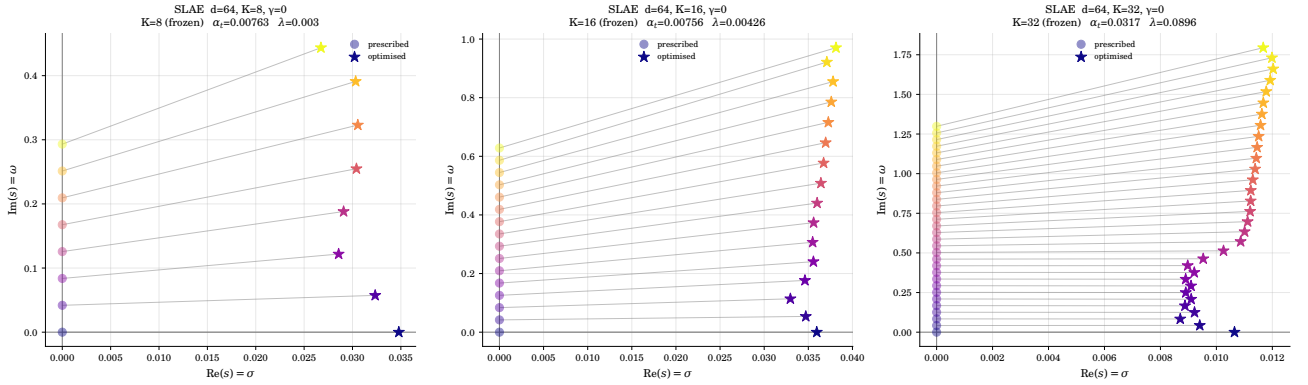


Figure 15: SLAE offline spatial contours in the complex plane ($d_z = 64$, $K \in \{8, 16, 32\}$). Circles: prescribed sampling points on the truncated Bromwich contour; stars: optimised points, joined to their prescribed position by a grey segment; colour encodes the frequency index k . The associated (α_t, λ) are reported in each panel title.

Autoencoder stage. Table 10 reports both stages for the six configurations, with the variation induced relative to the corresponding prescribed contour at $\gamma = 0.01$. At the reconstruction stage the offline contour *degrades* the error floor, moderately at $K = 8$ and $K = 32$ (+0.65 and +0.17 pp) and pronouncedly at $K = 16$ ($8.93 \rightarrow 16.05\%$). The prescribed damping was itself swept at that K , and the reference matters: the best prescribed floor is not the one tabulated but 7.79% at $\gamma = 0$ (against 7.83 and 8.93% at $\gamma = 0.001$ and 0.01), so the penalty is +7.12 pp against the conventional damping and +8.26 pp against the best prescribed configuration. The reconstruction stage is not the deliverable, however, and it is not what the criterion scores: (30) weighs the amplification factor against a nominal coefficient-error magnitude, and a triplet selected at any such magnitude has no reason to be the one minimising the reconstruction floor. The surrogate columns accordingly reverse this ordering; the full distributions are given in Appendix E.

Surrogate stage. The surrogate columns of Table 10 invert the ordering, and Figure 24 shows the corresponding error distributions. For SLAE, the offline spatial contour reduces the end-to-end error at $K = 8$ ($11.62 \rightarrow 9.76\%$, -1.85 pp) and at $K = 32$ ($5.70 \rightarrow 5.04\%$, -0.65 pp), notwithstanding the reconstruction penalty incurred at each. At $K = 16$, where the prescribed damping was swept, the gain does not survive the comparison:

Family	Contour	K	AE med. (%)	Δ AE (pp)	Surr. med. (%)	Δ Surr. (pp)	Gap (pp)
LLAE	learnable	8	2.89	−0.39	4.54	+0.53	+1.65
	learnable	16	2.68	−0.45	4.63	+0.57	+1.95
	learnable	32	2.60	−0.52	4.58	+0.25	+1.97
SLAE	offline	8	12.92	+0.65	9.76	−1.85	−3.16
	offline	16	16.05	+7.12	7.46	−0.13	−8.59
	offline	32	8.53	+0.17	5.04	−0.65	−3.49

Table 10: Optimised-contour configurations ($d_z = 64$), test set ($n = 1620$). *AE med.*: standalone reconstruction error; *Surr. med.*: end-to-end surrogate error. The Δ columns give the variation with respect to the corresponding prescribed-contour configuration of Tables 4–6 at identical (d_z, K) and $\gamma = 0.01$; negative values indicate an improvement brought by the optimised contour. *Gap*: surrogate median – AE median. Best per family in bold.

7.46% against 7.59% at $\gamma = 0.01$, but 7.39 and 7.43% at $\gamma = 0$ and 0.001, so the optimised contour improves on the conventional damping and matches, without beating, the best prescribed configuration. The Gap column is negative at every SLAE configuration and reaches −8.59 pp at $K = 16$: the entire reconstruction-stage penalty is recovered once the surrogate is trained. At $K = 32$, SLAE attains 5.04%, the most accurate SLAE result of this study and a 43% reduction relative to the POD reference (8.92%), while retaining the reduced decoding cost characteristic of the family.

Why the two stages disagree. The sign flip is not a trade the pipeline makes, and reading it as one would mistake the meter for the quantity. SLAE reconstructs at ϵ_{AE} and deploys at $\epsilon_{\text{surr}} \gg \epsilon_{\text{AE}}$, and the proxy (30) is minimised at a nominal magnitude fixed a priori (Section 2.4), at neither stage in particular: nothing in the criterion asks the optimised triplet to be better at ϵ_{AE} , and nothing forbids it either. The reconstruction penalty is therefore a meter the criterion never optimised rather than a price paid for the surrogate gain, and $K = 16$ shows how loosely the two levels constrain each other, +7.12 pp of floor against −0.13 pp of deployed error. Nor is the exchange the ridge term’s doing, $K = 32$ carrying the heaviest regularisation ($\lambda \approx 8.9 \times 10^{-2}$) for the smallest penalty (+0.17 pp) while $K = 16$ incurs +7.12 pp at $\lambda \approx 4.3 \times 10^{-3}$: bias and amplification alike are set by the joint geometry of (s_k, α_t, λ) . That the penalty is not merely amortised but recovered in full follows from Phase 2 optimising *through* the inversion, which Phase 1 never does (Section 2.2.1). The error is then free to settle where $A^{-1}F^H$ attenuates, and the bias operator $A^{-1}L_{\text{reg}}$, linear and known, is pre-compensated by the decoder as soon as the physical-time term enters, a term LLAE’s (12) already contains.

A rule for setting the triplet. What the procedure offers is a way of fixing (s_k, α_t, λ) , quantities otherwise posed by convention. It is deterministic (same data, same triplet, no seed and no judgement call) and requires no network training: 500 gradient steps on a purely linear-algebraic functional over 100 instances, against a complete two-phase training per candidate had the selection been empirical. It alters no tensor dimension either, the optimised surrogates consuming exactly the arithmetic of their prescribed counterparts. Measured against the convention it replaces, it is never worse at any K : −1.85 pp at $K = 8$, a genuine margin by the one-point standard of Section 5.3, and −0.65

at $K = 32$ and -0.13 at $K = 16$, both under that standard and therefore read as the rule matching the convention rather than beating it. The one configuration where an empirical search was affordable calibrates that claim, and is the informative one. At $K = 16$ the three prescribed damping values span 0.20 pp end-to-end (7.39 to 7.59%) and the optimised triplet lands at 7.46%: where a sweep is possible, the rule reaches it without running it. The two are not commensurate, however, the sweep moving one scalar where the rule moves $2K + 2$; the abscissa alone accounts for 0.20 pp against up to 1.85 for the full triplet (Section 2.4). The margin over the convention concentrates where the prescribed contour is least adequate, that is where the spectral truncation is most severe. On this problem the procedure is therefore not a general-purpose accuracy lever but a way of not choosing arbitrarily, and what it adds on the accuracy–cost trade-off is modest and free: SLAE at $K = 32$ attains 5.04%, within 1.2 pp of the most accurate configuration of the study where a prescribed contour left 1.9 pp.

4 Comparison across Cost and Resolution

4.1 Global Synthesis

The three references of this study isolate one ingredient each. POD in the Laplace domain (Section 3.2.1) keeps the spectral compression and removes the non-linearity; the DL-ROM (Section 3.2.2) keeps the non-linearity and removes the spectral compression; SLAE and LLAE keep both. Table 11 places them side by side at a common operating point ($d_z = 64$, $K = 16$ where applicable, one full 150-step trajectory per query), with accuracy and the four cost axes measured on the same hardware in a single benchmarking run.

Model	Contour	Median (%)	Targets	N_{surr}	Lat. (ms)	FLOPs (G)	Mem. (MB)
POD ($k_{\text{pod}} = 64$)	prescribed	8.92	16	—	3.5	0.2	212
DL-ROM	—	3.80	150	23.50 M	31.4	48.3	927
LLAE	prescribed	4.06	16	3.48 M	22.7	48.2	622
	learnable	4.63	16	3.48 M	23.8	48.2	621
SLAE	prescribed	7.59	16	3.22 M	15.7	30.3	235
	offline	7.46	16	3.22 M	15.6	30.3	235

Table 11: Synthesis at the common operating point ($d_z = 64$, $K = 16$, $\gamma = 0.01$ for the prescribed contours), test set ($n = 1620$), one 150-step trajectory per query. Median errors are those of Tables 2, 6 and 10. *Targets*: number of coefficient blocks the surrogate must predict (K Laplace frequencies, or N_t time steps for the DL-ROM). N_{surr} : surrogate parameters, equation (40). Latency, FLOPs and peak memory are measured at batch size 1 on an idle GPU. The four deep rows come from one benchmarking run and are therefore mutually comparable; the POD row is measured separately by the same procedure. FLOPs, memory and parameter counts are deterministic across runs; latency varies by two to three percent. Best per column in bold.

Accuracy and cost orderings. Two orderings emerge, and they are not the same. On accuracy, the DL-ROM leads, LLAE follows within half a point, SLAE trails by a further three points, and POD is last; every deep model improves on POD, by 15 to 57%

at this operating point, and SLAE narrows but does not close its deficit once its contour is optimised at $K = 32$ (Section 3.5). On cost the ordering reverses, unevenly across axes: among the deep models SLAE is the cheapest on all four and the DL-ROM the most expensive on all four, LLAE sitting between them but not at the same place on each axis, matching the DL-ROM’s arithmetic while being appreciably faster and lighter. The linear POD is two orders of magnitude below SLAE in arithmetic yet far less so in wall-clock time and no lighter in memory, the Tikhonov inversion being a floor no reduction in network cost can go below. No model dominates in the Pareto sense (Figure 16): the frontier at 150-step output is occupied by the DL-ROM, LLAE and SLAE, POD being dominated on accuracy by all three. Section 5.1 attributes the accuracy gaps to the ingredient each reference removes, Section 4.2 accounts for the divergence between the latency and parameter axes, and Section 4.3 for the restriction of this ranking to a fixed output resolution.

Sensitivity to the damping abscissa. The prescribed rows of Table 11 all fix $\gamma = 0.01$, a convention that does not favour the three families equally. Damping acts on two contributions of opposite sign: a purely imaginary contour lowers the truncation error at every frequency, while a larger γ attenuates the inversion error propagating those residuals back to the time domain (Section 3.2.1). Their sum has no universal sign. POD improves with damping ($8.42 \rightarrow 8.09\%$), the SLAE floor degrades by 1.14 pp ($7.79 \rightarrow 8.93\%$) and its surrogate by 0.20, and LLAE is insensitive to within 0.16 pp. Small margins thus reflect cancellation rather than an absence of effect, and the favourable abscissa is a property of the family and of the stage measured, not a convention transferable between them. The offline criterion, initialised on the purely imaginary contour and admitting $\sigma_k \geq -0.05$, returns $\sigma_k \approx 0.035$ (Section 3.5). The abscissa was swept at $K = 16$ only for the two deep families (Section 5.3).

4.2 Cost Axes Reached by Temporal Compression

Table 11 is read at a fixed deliverable, one full 150-step trajectory per query. At that operating point the question is which cost axes the temporal compression actually reduces, and the answer turns on where the transform is applied.

Latent-space compression reaches the surrogate only. Equation (40) predicts that the Laplace surrogates are $6.8\times$ lighter than the DL-ROM, and Table 11 confirms it exactly. That saving does not carry over to the other axes: LLAE runs only $1.38\times$ faster, and performs the same arithmetic as the model it replaces (Table 11). The pipeline spends its time in the decoder, not the surrogate (Sections 3.4.1–3.4.3). Compressing the time axis *in latent space* shrinks what the MLP must regress, from 150 latent frames to 16 complex ones, but the inverse transform expands it back to 150 frames before decoding. Both models therefore invoke the same convolutional decoder 150 times, and the dominant term of the forward pass is untouched. The residual differences follow from the surrogate and favour LLAE: $1.5\times$ less peak memory, the 20M parameters it does not allocate, and $1.38\times$ less wall-clock time, the DL-ROM evaluating 150 per-time-step heads where LLAE evaluates 16 per-frequency ones. That the two differ in latency while agreeing in arithmetic is why we report both: at equal FLOPs, the model whose predictor is split into ten times fewer kernels is the faster one, and no operation count can express that.

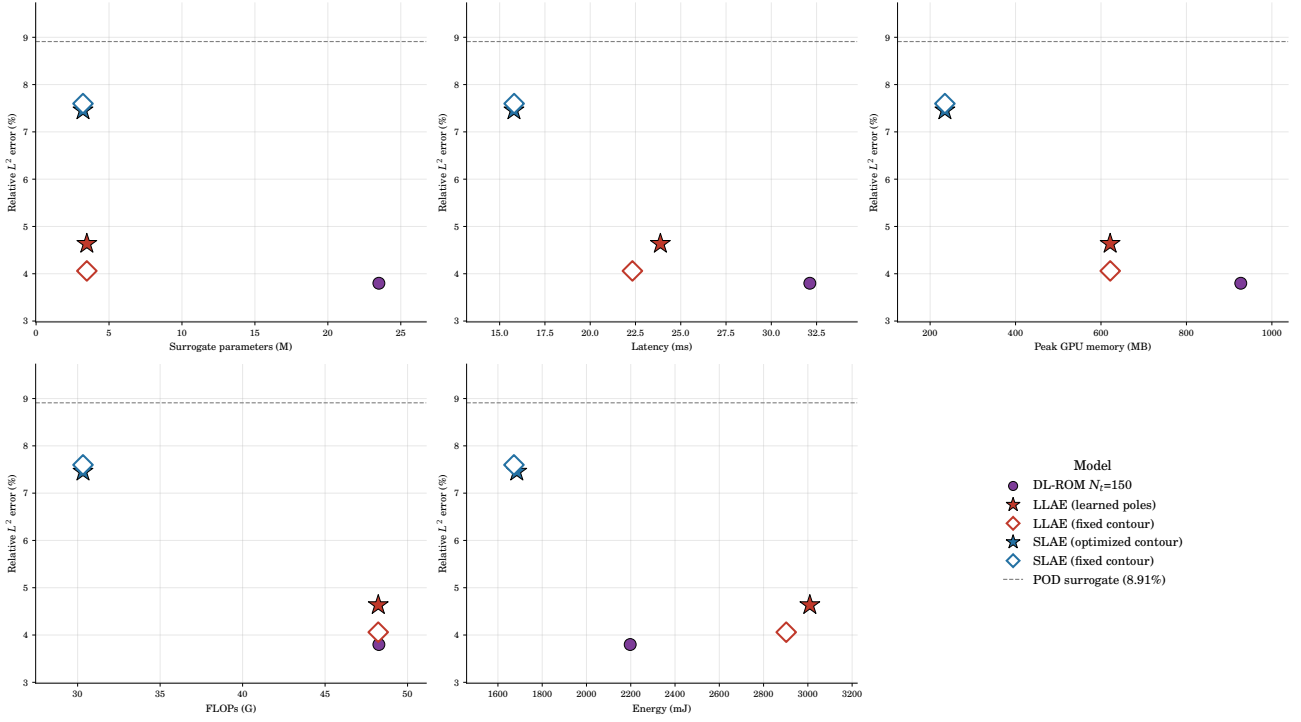


Figure 16: Accuracy against five inference-cost axes at fixed 150-step output (batch size 1): surrogate parameters, latency, peak GPU memory, FLOPs and energy. Bottom-left is best. Stars: optimised contours (LLAE latent learnable, SLAE offline spatial); open diamonds: prescribed contours; green disc: DL-ROM at $N_t = 150$. The dashed line marks the POD reference (8.92%).

Physical-domain compression reaches the decoder. SLAE is the configuration in which temporal compression does reach the decoder. Applying the transform in the physical domain leaves $K = 16$ complex frequency frames to decode instead of 150 temporal ones, so the whole forward pass, and not merely the surrogate, is $O(K)$. The resulting $1.6\times$ reduction in arithmetic this time *does* convert into wall-clock time: $2.0\times$ lower latency, $3.9\times$ lower peak memory and $1.7\times$ lower energy at identical 150-step output. The three points of accuracy this costs are the counterpart, the trajectory being reconstructed from a 16-point spectrum; the offline spatial contour of Section 3.5 recovers part of it, down to 5.04% at $K = 32$ while retaining a $1.6\times$ speed advantage. The principle is that a Laplace-domain pipeline is cheap on the axes its compressed representation actually reaches: latent-space compression reaches the surrogate alone, whereas physical-domain compression reaches the decoder as well. Reporting parameter counts alone, the natural reading of equation (40), would have credited LLAE with a $6.8\times$ gain that no user would ever measure, an instance of the efficiency-misnomer effect documented by Dehghani et al. [5]; hence the several axes reported jointly in Table 11 and Figure 16.

4.3 Cost and Accuracy across Output Time Grids

Every result so far has been stated at a single deliverable, the full 150-step trajectory: the ranking of Section 4.1 and the cost analysis of Section 4.2 alike. We now let the deliverable itself vary, sweeping the requested resolution $N_t \in \{15, 30, 50, 75, 150\}$ at fixed horizon T , and ask what each family costs and what it delivers as the output grid changes. The Laplace pipelines answer from a single $K = 16$ checkpoint throughout; the DL-ROM requires one trained model per resolution. The comparison therefore sets a

specialised model against one evaluated away from its training grid, which is the property at issue rather than a confound.

4.3.1 Accuracy and the Sampling Floor

Accuracy above and below the floor. At the coarsest point of the sweep the requested grid has Nyquist frequency $\omega_{\text{Nyq}} = 0.314 \text{ rad s}^{-1}$, below the ω_{max} of every contour tested: all four are aliased by (33), and all four collapse, one to two orders of magnitude above their 150-step error. We report that regime only to locate the floor. Clear of it, every configuration is essentially independent of the requested temporal resolution, and this holds for both families (Figure 17; per-resolution medians in Table 13). At any resolution at least $1.25\times$ its floor, no configuration departs from its 150-step error by more than 0.16 pp; LLAE under a prescribed contour is even marginally more accurate at $N_t = 50$ than at full resolution. Within that margin a single checkpoint serves the whole range at no cost our protocol resolves, which for the prescribed contours means a threefold reduction of the delivered grid. Degradation appears only on approaching the floor and stays moderate there, the worst case being +1.98 pp for the offline spatial contour at $N_t = 50$, a mere 8% above its own floor. The ordering between families is unaffected throughout, widening slightly as the grid coarsens. What separates the configurations is therefore not a rate at which accuracy decays with resolution, of which there is none, but where each one’s floor sits.

The flatness is conditional on the regularisation transport, which carries (α_t, λ) to the requested grid by (32), a rule derived from the continuous functional rather than fitted. Repeating the sweep with the weights left at their training values (right panel of Figure 17) widens the 0.16 pp bound above to 0.82, a fivefold loss, and the excess falls entirely on the optimised contours: LLAE with learnable poles still trends with resolution when the weights are untransported, and is flat once they are. The prescribed contours are indifferent to the correction, SLAE unchanged to two decimals at every resolution and LLAE within 0.05 pp, so the correction is required precisely where the spectral window has been widened.

Optimised contours raise their own floor. What sets the usable ranges apart is where each contour’s floor falls. Both optimisation procedures dilate the imaginary parts of the sampling points (Section 3.5), so each leaves the closed form (34) and must be scored through the general bound (33). Evaluated on the stored poles, ω_{max} rises from 0.628 rad s^{-1} under the prescribed contour to 0.838 for the LLAE learnable contour and 0.971 for the SLAE offline one, placing their floors progressively higher (dotted lines, Figure 17). The ordering of the floors predicts the ordering of the failures exactly. At $N_t = 50$ the offline spatial contour is the only configuration within reach of its floor, and the only one to degrade. At $N_t = 30$ the two prescribed contours sit exactly at their floor and hold, the critical regime of Section 2.5 proving marginal but serviceable here; the latent learnable contour is below its own and degrades; and the offline spatial contour, furthest below, collapses by an order of magnitude. What Section 3.5 obtains as accuracy at no inference cost is therefore paid for in resolution range: the wider spectral window that lowers the error at $N_t = 150$ is what makes the representation unserviceable on a coarse grid.

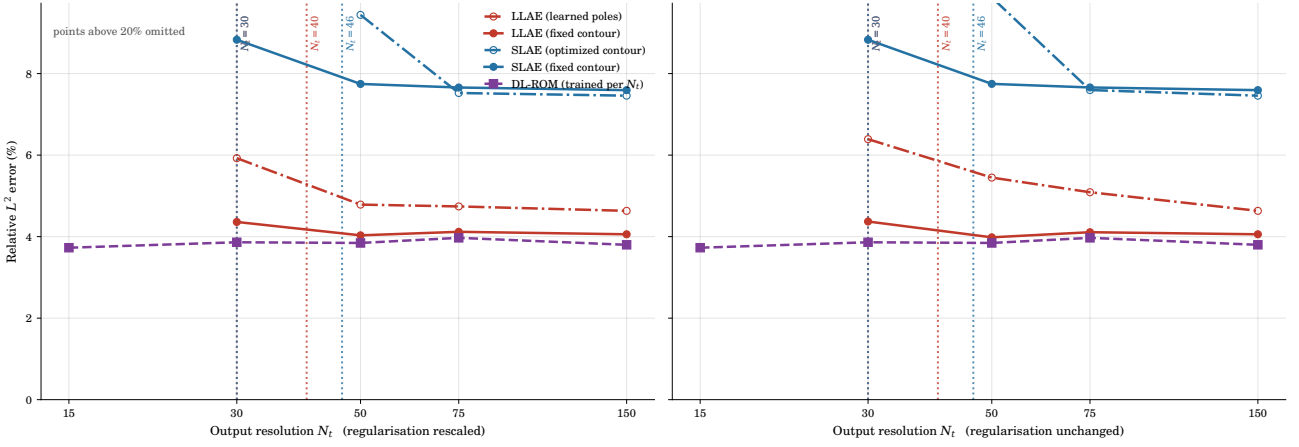


Figure 17: Relative L^2 error against requested output resolution at fixed horizon T (test set, $n = 1620$). Solid lines: a single Laplace checkpoint queried at each resolution. Dashed: the DL-ROM, one trained checkpoint per resolution. The dotted vertical lines mark the sampling floor (33) of each contour, computed from its own stored poles: $N_t = 30$ for the prescribed contours, 40 for the LLAE learnable one and 46 for the SLAE offline one. Left: Tikhonov terms rescaled to the output grid; right: left unchanged.

4.3.2 Inference Cost and Grid Flexibility

The DL-ROM’s single lever. The DL-ROM has exactly one lever for reducing its temporal cost, producing fewer time steps; sweeping the stride κ over the training grid (Section 2.6.2) traces it, one trained model per point (Figure 18; per-stride costs in Table 12). First, the DL-ROM’s per-step accuracy is flat across a tenfold reduction of N_t , varying by a quarter of a point with no monotone trend: subsampling does not make the individual frames harder to predict, so the $9\times$ cost reduction from $\kappa = 1$ to $\kappa = 10$ is obtained solely by delivering fewer steps. Second, the parameter-matched point falls at $\kappa = 10$: there the DL-ROM surrogate falls to 3.06 M parameters, the size of LLAE at $K = 16$, both compressing the time axis roughly tenfold, uniformly on one side and spectrally on the other. The two differ in what they deliver: 15 time steps for the first, the complete 150-step trajectory for the second, recovering the missing 135 steps requiring either interpolation between samples 10 time units apart or retraining at a finer stride.

Cost profiles across resolutions. The coefficients a Laplace surrogate predicts carry no reference to a time grid (Section 2.5), so one trained model answers at any requested resolution; the DL-ROM’s per-time-step heads, by contrast, are fixed to the grid they were trained on.¹ The three families accordingly scale differently with the requested N_t (Figure 18). The DL-ROM grows on every axis, adding both a decoder pass and a surrogate head per requested step (equation (40)). LLAE removes the second of these but not the first: it decodes N_t frames through the same backbone, so its arithmetic follows the DL-ROM curve exactly, while its latency and peak memory, relieved of the per-step heads, grow at roughly half the rate, a distinction no operation count can express. SLAE removes both, decoding $K = 16$ frames whatever is asked, and is flat on every axis; refining the output grid is therefore free for it, and its margin over the other two grows without bound. Where that margin opens depends on the axis, and the arithmetic reaches it last: SLAE’s flat memory undercuts the DL-ROM’s allocation from $N_t \approx 30$, its flat latency from $N_t \approx 75$, and its FLOPs and energy only near $N_t \approx 95$ and 100.

¹The head-indexed formulation is our own; Section 5.3 states what this claim owes to that choice.

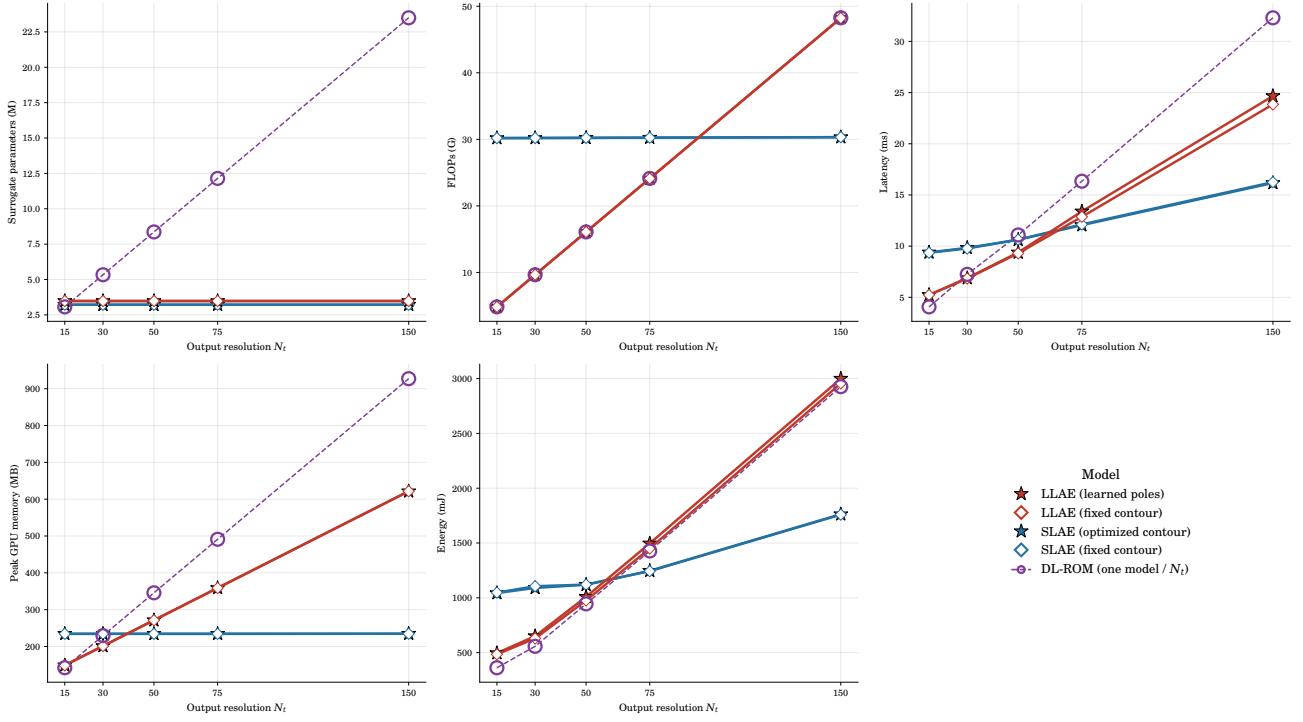


Figure 18: Inference cost against requested output resolution N_t at fixed horizon T (batch size 1). Solid lines: a *single* Laplace checkpoint per curve, queried at each N_t . Dashed line with open circles: the DL-ROM, one separately trained model per point, each evaluable only at its own N_t .

Peak memory is therefore the axis on which the physical-domain transform becomes advantageous first, at roughly a third of the resolution an operation count would indicate; the crossovers are a property of the decoder, not of the transform.

Consequence for many-query workflows. The five DL-ROM points of Figure 18 are five trainings, each a fixed answer to a question typically settled only after deployment and differently for each downstream use: at what temporal resolution is the trajectory required? The two Laplace curves are one training each and answer the entire range at inference time; for a many-query workflow in which a coarse scan precedes a refined analysis of a few selected parameter values, that flexibility, rather than the parameter count, is the operative advantage. The shallower memory profile compounds it, and on this axis the advantage widens with the resolution requested: at the full 150-step output LLAE’s allocation is two thirds of the DL-ROM’s (Table 11), so roughly half again as many trajectories fit in a batch on the same device. A many-query sweep is throughput-bound rather than latency-bound, and it is peak memory rather than per-query latency that limits the batch.

5 Discussion

5.1 Attribution of the Observed Gains

Accuracy attribution. The two references remove one ingredient each, and read together they attribute the accuracy. Removing the non-linearity is costly: POD in the Laplace domain is outperformed by every deep model (Table 11), and additional POD modes do not close the gap, $k_{\text{pod}} = 128$ improving the reconstruction floor to 7.04%

while *degrading* the deployed error to 11.30%. Removing the Laplace transform has no effect our measurements resolve: 0.04 pp separate the best DL-ROM from the best LLAE (3.80 against 3.84%), below the threshold this study treats as significant. SLAE is the one family whose deficit is substantive, and the confound of Section 5.3 attributes a substantial share of it to a Phase-1 objective mismatch rather than to the transform. On accuracy, temporal compression is therefore neutral in latent space and costs some three points in the physical domain.

Placement of the transform and inference cost. The substantive findings lie on the cost and flexibility side. Sections 4.2–4.3 establish that *where* the transform is applied determines which cost axes benefit: a Laplace pipeline is economical on exactly the axes its compressed representation reaches, the parameter count predicted by equation (40) being the axis least representative of the cost a user measures.

Design of the regularised inversion. The regularised inversion, the poles *and* the associated weights (α_t, λ) , fixed by convention in the Laplace-domain reduced-order models known to us, admits a systematic selection rule, provided the candidate is scored on the deployed error rather than on the reconstruction floor. What makes it optimisable is that the bound (25) draws no distinction between placing a sampling point and weighting a penalty: both scalars depend on the triplet alone, so a candidate can be scored without training any network. Only the offline criterion improves the emulator, by up to 1.85 pp end-to-end, at contours whose reconstruction floor is degraded by as much as 7.12 pp; the criterion is minimised at a nominal coefficient-error magnitude and is therefore not expected to minimise the reconstruction floor (Section 3.5). The learnable route produces the best Laplace autoencoder of the study (2.60%) and no better surrogate (Appendix E, which also states why the two routes are not a controlled comparison). The margin over the convention is uneven and modest, 1.85 pp where the truncation is severest and 0.13 at $K = 16$, where a damping sweep reaches the same level by training three candidates; the rule is nonetheless the only lever in this study that is free in both training and inference.

Latent post-compression: a negative result. Compressing the latent target before the surrogate, by SVD along one mode or by Tucker along two, improves neither accuracy nor latency: it shrinks the prediction target by up to $4\times$ for a fraction of a point, and is useful only when the surrogate rather than the decoder is the binding constraint. Its value here is diagnostic: a rank-4 basis over the frequency mode performs as well as a rank-8 one, so the $K = 16$ coefficients span a subspace of substantially lower dimension than their count, a redundancy established by a tensor factorisation fitted blind to the physics and therefore independent of the argument that motivated the transform. The truncation is applied after the transform and does not alter the sampling density, so it constrains the dimension of the surrogate target rather than the adequacy of K , which remains a binding bottleneck for SLAE (Section 3.4.1).

5.2 Positioning with Respect to Related Work

Spatial compression. Our pipeline reuses well-established components and combines them along a new axis. The *spatial* compressor is a convolutional encoder–decoder applied directly to the field, i.e. the deep convolutional-autoencoder ROM of Lee and Carlberg [23] and of the original DL-ROM [9]; we omit the preliminary POD reduction of

POD-DL-ROM [10], an efficiency device rather than an accuracy one and unnecessary for a convolutional network operating on a structured grid. The *linear* baseline is a POD basis followed by a parameter-to-coefficient network, the non-intrusive POD-NN of Hesthaven and Ubbiali [19], here applied frequency-by-frequency in the Laplace domain. Our optional SVD and Tucker stages (Sections 2.3.2–2.3.3) also use linear projections, but in the opposite role: they act *after* the non-linear autoencoder, in the latent space, to shrink the surrogate target.

Treatment of time. What distinguishes our approach is not the spatial compressor but the treatment of *time*. All the DL-ROMs above keep the full temporal resolution of the trajectory: the reduced dynamics are queried once per time step through a (μ, t) -conditioned network [9, 10] or propagated sequentially in time [31], so the temporal work scales with N_t . We instead compress the time axis itself, predicting $K \ll N_t$ spectral coefficients in a single shot and recovering the full trajectory by a cheap linear inversion. This transposes the spectral intuition of Laplace-domain linear ROMs [34, 14] and of the Laplace Neural Operator [3] into the *non-linear, parameter-to-field* DL-ROM setting, where, to our knowledge, temporal Laplace compression has not previously been used. The Laplace stage is orthogonal to the spatial-compression choices above and could be layered on top of any of them, including POD-DL-ROM.

Relation to neural operators. The learnable-contour variant of LLAE (Section 2.4) resembles operator-learning methods along three axes. It applies its spectral transform *inside a learned latent space* rather than on the physical field, the design principle of the Latent Neural Operator [36]. It learns the complex poles $s_k = \sigma_k + i\omega_k$ end-to-end, a latent-space counterpart of the pole–residue representation at the heart of the Laplace Neural Operator [3]. And its shared trunk, conditioned on a sinusoidal encoding of s_k , maps $(\mu, s_k) \mapsto \hat{z}(s_k)$, the Laplace frequency playing the role the trunk coordinate does in a DeepONet [25]. The difference is one of *setting*, not of mechanism: genuine neural operators are field-to-field maps between function spaces, discretisation-invariant in their input, whereas our input is a finite parameter vector μ and our decoder a fixed-resolution convolutional network. LLAE with a learnable contour is thus best understood as a *parameter-conditioned, latent Laplace operator* rather than as a neural operator in the strict sense.

5.3 Limitations

Several qualifications bound the above, and we state them in decreasing order of consequence.

Confounds in the SLAE/LLAE comparison. The two families do not share a Phase-1 objective: LLAE is supervised on the physical-time reconstruction, SLAE only in the Laplace domain, and their batch sizes differ accordingly. The asymmetry is constitutive rather than incidental. Supervising SLAE in physical time requires decoding N_t frames per sample, which is precisely the cost the family is constructed to avoid, so the two objectives cannot be equalised without dissolving the distinction the comparison is about. What is compared is therefore two complete design points rather than two placements of the transform holding everything else fixed. The effect of the asymmetry is nonetheless measured: the SLAE Gap is negative in every configuration, Phase 2 introducing the physical-time term for the first time and improving on the standalone autoencoder by

up to 8.59 pp under an optimised contour, which indicates that a substantial share of SLAE’s deficit is objective mismatch rather than architectural limitation.

Statistical resolution. Every number comes from a single training run per configuration, at one fixed global seed (Appendix G): medians over 1,620 test instances resolve the error well with respect to test sampling, but we have no estimate of run-to-run variability under reseeding. Obtaining one requires replicating each configuration over several seeds, that is repeating a campaign of two-phase trainings comprising 14 autoencoders and 14 direct surrogates before the compression, contour and resolution variants are counted, and we did not carry that cost. We therefore apply one standard uniformly, that differences below roughly one point are unresolved by our evidence rather than measured effects. This covers the SVD and Tucker variants (+0.18 and +0.27 pp for LLAE) and the 0.04 pp separating LLAE from the DL-ROM, which is accordingly neither a penalty of the temporal compression nor a claim of equivalence. Differences above a point (the POD gap, the SLAE deficit, the 1.85 pp contour margin at $K = 8$, every cost ratio) are unaffected; the two smaller contour margins fall under the same standard and are read here as the rule matching the convention rather than improving on it.

Interpretation of the cost axes. Wall-clock latency measures an implementation as much as an architecture: at the reconstruction stage the latency ratios are compressed relative to the arithmetic ones (Table 5), the fixed cost of the forward transform and its inverse being invisible to a FLOP count. The surrogate stage, which applies no forward transform, is free of that asymmetry, and it is the stage the conclusions rest on. Energy carries little independent information at this scale: the board draws near-constant power over kernels of a few milliseconds, so the energy column is proportional to latency to within a few percent.

Structure of the design. The parameter design is full factorial, 225 points in (k, C) crossed with 36 injection angles, and partitioned uniformly over the product grid (Section 3.1). Every (k, C) pair being replicated at all 36 angles, the test error measures interpolation at held-out points of a dense design rather than extrapolation to a (k, C) pair outside it. The per-direction breakdown of Section 3.4.1 locates the error but does not remedy this; a second partition holding out whole (k, C) pairs would, and was not run here.

Attribution within the triplet. The offline criterion moves the poles and the two regularisation weights at once, so the gains of Section 3.5 are attributable to the triplet as a whole. The damping sweep bounds the abscissa’s share at 0.20 pp of the 1.85. Separating the imaginary parts from (α_t, λ) would require optimising the poles with the weights held fixed: producing such a triplet costs no network training, but scoring it on the deployed error costs a complete two-phase training per variant, and the ablation was not run here.

Choice of DL-ROM formulation. Our baseline gives the DL-ROM surrogate one head per time step, which is what makes it 23.5M parameters against 3.48M for LLAE. It is the more capacious of the two formulations and the one in which the baseline leads on accuracy (Table 11); a surrogate taking t as an input coordinate, as in [9], would be far lighter in parameters and, representing the whole trajectory with shared capacity, may reasonably be expected to be the less accurate. What is missing is the comparison against

that lighter variant, which would locate the accuracy traded for the parameter count; the $6.8\times$ margin is accordingly specific to the head-indexed formulation, whereas the FLOPs, latency and memory comparisons are decoder-dominated and insensitive to the choice.

Range of the temporal-resolution claim. A single checkpoint serves any output time grid only above the sampling floor (33) set by its own poles, and the optimised contours raise that floor (Section 4.3). The flexibility is therefore bounded, and the bound is the counterpart of the accuracy the optimised inversion gains. It concerns the time axis alone: the spatial discretisation is fixed by the convolutional decoder throughout, and nothing here claims the discretisation invariance of a neural operator (Section 5.2).

Scope. All results concern one two-dimensional convection–diffusion–reaction problem with $p = 3$ parameters and a single output field, on a structured grid, over a fixed horizon. The proxy (30) moreover enters at a nominal error magnitude set through λ_{AE} , held at one value throughout, so the sensitivity of the selected triplet to that setting is not characterised, although it costs no training to assess.

5.4 Perspectives

The most direct continuations follow from the limitations: a partition holding out whole (k, C) pairs and an estimate of run-to-run variability under reseeding are prerequisites to any quantitative claim beyond the orderings established here. The directions below appear the most promising.

Making the learnable contour deliver. The learnable route yields the most accurate autoencoder of the study and no better surrogate (Appendix E, which states what bounds that comparison). Turning that reconstruction gain into a deployed one is the open item: the poles are differentiable at both phases, so the step size applied to them, their parameterisation and the constraints keeping the pole set conjugate-symmetric are the levers to revisit.

Exploiting the temporal-grid freedom. Because the trajectory is materialised by a linear solve on requested nodes, non-uniform time grids, dense during the transient and sparse in the tail, come at no additional cost and suit a many-query workflow. Whether the same coefficients support extrapolation beyond the training horizon T , where the decay axis γ of the Laplace transform has a natural interpretation that Fourier lacks, is the more ambitious version of the same question, and the one that most distinguishes the transform from a truncated DFT.

Composition with other reduction stages. The Laplace stage acts on time and is orthogonal to every spatial-compression choice in the DL-ROM literature, so it could be layered onto POD-DL-ROM [10] or onto a parameter-conditioned U-Net [20] without modification, and the SLAE variant, the one whose compression reaches the decoder, is where such a combination would pay most.

Scaling. Scaling to three-dimensional fields, to several coupled species, and to higher-dimensional parameter spaces is the necessary test of whether the accuracy cost of spectral truncation, 0.64 pp on the reconstruction floor here for LLAE (Section 3.3), remains as modest when the transient structure is richer.

6 Conclusion

We have introduced the Laplace transform as a temporal compression stage within the DL-ROM framework, predicting $K \ll N_t$ spectral coefficients from a parameter vector and recovering the full trajectory through a differentiable Tikhonov inversion, in two instantiations differing in where the transform is applied, the physical domain (SLAE) or the learned latent space (LLAE). On a parametric transient combustion problem with $N_t = 150$, LLAE reaches 3.84% median relative L^2 error and SLAE 7.46% at $K = 16$, falling to 5.04% at $K = 32$ under an optimised contour, against 8.92% for the linear POD surrogate and 3.80% for the Laplace-free DL-ROM ablation.

The benefit is confined to the axes the compressed representation actually reaches. Applied in latent space it shrinks the surrogate alone, leaving the arithmetic of the forward pass untouched; applied in the physical domain it reaches the decoder and converts into $2.0\times$ latency and $3.9\times$ memory over the DL-ROM at identical output, at $K = 16$, raising K to 32 returning part of that margin. It also changes what a trained model can be asked for. The predicted coefficients refer to no time grid, so one trained model answers on any output *time* grid above the sampling floor set by its own poles, a floor the optimised contours raise. The head-indexed DL-ROM used here required one training per grid, where a (μ, t) -conditioned variant would instead pay one network evaluation per requested step. A many-query sweep being throughput- rather than latency-bound, the axis that limits it is peak memory, on which the physical-domain transform overtakes the DL-ROM from $N_t \approx 30$ against ≈ 95 for the arithmetic.

Compressing the latent target further, along one mode by SVD or along two jointly by Tucker, improves neither accuracy nor latency: the decoder, not the predictor, is the binding constraint. The outcome is nonetheless informative, a rank-4 basis over the frequency mode performing as well as a rank-8 one, so that the $K = 16$ coefficients span a subspace of markedly lower dimension than their count, a redundancy a tensor factorisation fitted blind to the physics recovers on its own. All of the above rests on one two-dimensional convection–diffusion–reaction problem and on a single training run per configuration, under a standard treating differences below roughly one point as unresolved.

The second principal contribution is a rule for setting the regularised inversion, the poles and the regularisation weights jointly rather than the contour alone, where the practice is to fix them by convention. The bias–amplification bound (25) shows that everything it does to the reconstructed trajectory passes through two scalars depending on the triplet (s_k, α_t, λ) and on nothing else, which makes a candidate scorable without training any network. The rule is never worse than the convention it replaces ($\gamma = 0.01$) at any K and improves on it by up to 1.85 pp on the deployed error, for a few hundred gradient steps on a purely linear-algebraic functional and at no inference cost, while degrading the reconstruction floor by as much as 7.12 pp. That sign reversal is the methodological point we would most like to carry beyond this paper: an inversion, and more generally a representation, must be judged at the magnitude of error the deployed model actually incurs, not at the floor it permits.

References

- [1] J. Abate and W. Whitt. The Fourier-series method for inverting transforms of probability distributions. *Queueing Systems*, 10(1–2):5–87, 1992.
- [2] Pierre Baldi and Kurt Hornik. Neural networks and principal component analysis: Learning from examples without local minima. *Neural Networks*, 2(1):53–58, 1989.
- [3] Qianying Cao, Somdatta Goswami, and George Em Karniadakis. LNO: Laplace neural operator for solving differential equations, 2023. arXiv:2303.10528.
- [4] L. De Lathauwer, B. De Moor, and J. Vandewalle. On the best rank-1 and rank- $(R_1, R_2, \dots, R_N)$ approximation of higher-order tensors. *SIAM Journal on Matrix Analysis and Applications*, 21(4):1324–1342, 2000.
- [5] Mostafa Dehghani, Yi Tay, Anurag Arnab, Lucas Beyer, and Ashish Vaswani. The efficiency misnomer. *International Conference on Learning Representations (ICLR)*, 2022.
- [6] C. Eckart and G. Young. The approximation of one matrix by another of lower rank. *Psychometrika*, 1(3):211–218, 1936.
- [7] C. L. Epstein and J. Schotland. The bad truth about Laplace’s transform. *SIAM Review*, 50:504–520, 2008.
- [8] X. Fan, Y. Lin, B. Gong, and H. Li. Offline meteorology-pollution coupling global air pollution forecasting model with bilinear pooling, 2025. arXiv:2503.18405.
- [9] Stefania Fresca, Luca Dedè, and Andrea Manzoni. A comprehensive deep learning-based approach to reduced order modeling of nonlinear time-dependent parametrized PDEs. *Journal of Scientific Computing*, 87(2):61, 2021.
- [10] Stefania Fresca and Andrea Manzoni. POD-DL-ROM: Enhancing deep learning-based reduced order models for nonlinear parametrized PDEs by proper orthogonal decomposition. *Computer Methods in Applied Mechanics and Engineering*, 388:114181, 2022.
- [11] Matteo Giacomini and Pedro Díez. Surrogates for physics-based and data-driven modelling of parametric systems: Review and new perspectives. *Archives of Computational Methods in Engineering*, 2026. Published online.
- [12] Silke Glas, Antonia Mayerhofer, and Karsten Urban. Two ways to treat time in reduced basis methods. In Peter Benner, Mario Ohlberger, Anthony Patera, Gianluigi Rozza, and Karsten Urban, editors, *Model Reduction of Parametrized Systems*, volume 17 of *MS&A*, pages 113–128. Springer, 2017.
- [13] Nicola Guglielmi, María López-Fernández, and Giancarlo Nino. Numerical inverse Laplace transform for convection-diffusion equations. *Mathematics of Computation*, 89(323):1161–1191, 2020.
- [14] Nicola Guglielmi and Mattia Manucci. Model order reduction in contour integral methods for parametric PDEs. *SIAM Journal on Scientific Computing*, 45(4):A1711–A1740, 2023.
- [15] Q. Guo, M. Ren, S. Wu, et al. Applications of artificial intelligence in the field of air pollution: a bibliometric analysis. *Frontiers in Public Health*, 10:933665, 2022.

- [16] N. Halko, P.-G. Martinsson, and J. A. Tropp. Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review*, 53(2):217–288, 2011.
- [17] P. C. Hansen. *Rank-Deficient and Discrete Ill-Posed Problems: Numerical Aspects of Linear Inversion*. SIAM, Philadelphia, 1998.
- [18] Jan S Hesthaven, Gianluigi Rozza, and Benjamin Stamm. *Certified Reduced Basis Methods for Parametrized Partial Differential Equations*. Springer, 2015.
- [19] Jan S Hesthaven and Stefano Ubbiali. Non-intrusive reduced order modeling of nonlinear problems using neural networks. *Journal of Computational Physics*, 363:55–78, 2018.
- [20] Michael Urs Lars Kastor, Jan Rottmayer, Anna Hundertmark, and Nicolas Ralph Gauger. Parameter conditioned interpretable U-Net surrogate model for data-driven predictions of convection-diffusion-reaction processes, 2026. arXiv:2601.22654.
- [21] T. G. Kolda and B. W. Bader. Tensor decompositions and applications. *SIAM Review*, 51(3):455–500, 2009.
- [22] Toni Lassila, Andrea Manzoni, Alfio Quarteroni, and Gianluigi Rozza. Model order reduction in fluid dynamics: Challenges and perspectives. In Alfio Quarteroni and Gianluigi Rozza, editors, *Reduced Order Methods for Modeling and Computational Reduction*, volume 9 of *MS&A*, pages 235–274. Springer, 2014.
- [23] Kookjin Lee and Kevin T Carlberg. Model reduction of dynamical systems on nonlinear manifolds using deep convolutional autoencoders. *Journal of Computational Physics*, 404:108973, 2020.
- [24] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations (ICLR)*, 2021. arXiv:2010.08895.
- [25] L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3:218–229, 2021.
- [26] Lu Lu, Xuhui Meng, Shengze Cai, Zhiping Mao, Somdatta Goswami, Zhongqiang Zhang, and George Em Karniadakis. A comprehensive and fair comparison of two neural operators (with practical extensions) based on FAIR data. *Computer Methods in Applied Mechanics and Engineering*, 393:114778, 2022.
- [27] Swaroop Mallick and Monika Mittal. AI-based model order reduction techniques: A survey. *Archives of Computational Methods in Engineering*, 32(4):2321–2346, 2025.
- [28] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng. NeRF: Representing scenes as neural radiance fields for view synthesis. In *European Conference on Computer Vision (ECCV)*, 2020.
- [29] L. Mirsky. Symmetric gauge functions and unitarily invariant norms. *The Quarterly Journal of Mathematics*, 11(1):50–59, 1960.
- [30] Jad Mounayer, Sebastian Rodriguez, Chady Ghnatios, Charbel Farhat, and Francisco Chinesta. Rank reduction autoencoders, 2025. arXiv:2405.13980.

- [31] Pranshu Pant, Ruchit Doshi, Pranav Bahl, and Amir Barati Farimani. Deep learning for reduced order modelling and efficient temporal evolution of fluid simulations. *Physics of Fluids*, 33(10), 2021.
- [32] Eric Parish, Masayuki Yano, Irina Tezaur, and Traian Iliescu. Residual-based stabilized reduced-order models of the transient convection–diffusion–reaction equation obtained through discrete and continuous projection. *Archives of Computational Methods in Engineering*, 32(3):1885–1929, 2025.
- [33] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville. FiLM: Visual reasoning with a general conditioning layer. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.
- [34] Ricardo Reyes. The frequency reduced-basis method: Reduced order models for time-dependent problems using the Laplace transform, 2025. arXiv:2502.19120.
- [35] C. E. Shannon. Communication in the presence of noise. *Proceedings of the IRE*, 37(1):10–21, 1949.
- [36] Tian Wang and Chuang Wang. Latent Neural Operator for solving forward and inverse PDE problems. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2406.03923.

A Per-Checkpoint Error Distributions

POD reconstruction. Figure 19 shows the distribution of relative L^2 errors for each of the 7 POD reconstruction configurations evaluated in Section 3.2.1.

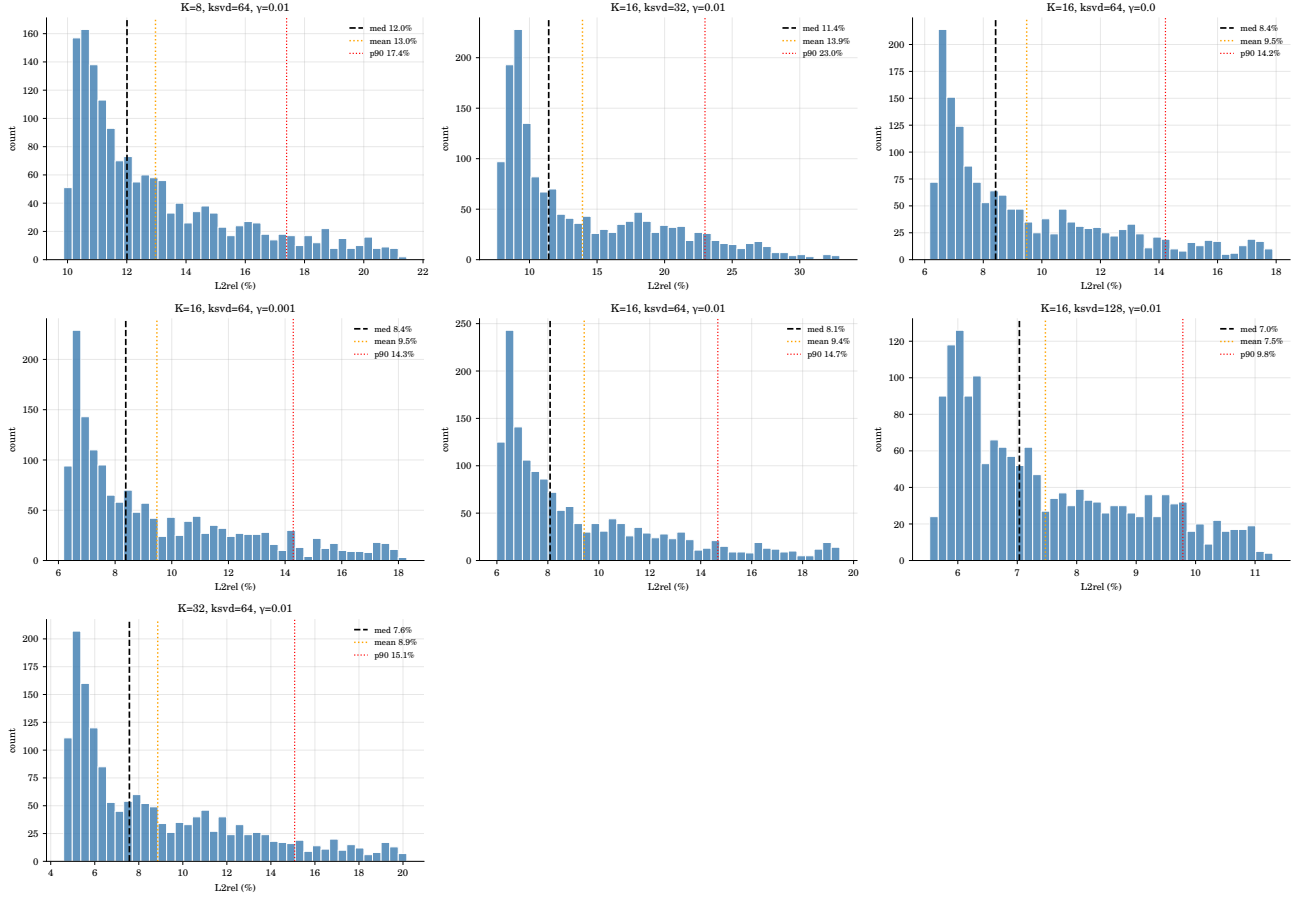


Figure 19: Per-configuration error histograms for the POD reconstruction (test set, $n = 1620$). The dashed black line is the median, dashed orange the mean, dotted red the 90th percentile.

POD surrogate. Figure 20 shows the corresponding distributions for the end-to-end POD surrogate (Section 3.2.1).

AE checkpoints. Figure 21 shows the distribution of relative L^2 errors across the 1620 test simulations for each of the 14 AE checkpoints evaluated in Section 3.3. Dashed lines indicate the median (black) and mean (orange); the dotted red line marks the 90th percentile.

Surrogate checkpoints. Figure 22 shows the per-sample error distribution for each of the 14 direct surrogate checkpoints evaluated in Section 3.4.1.

DL-ROM baseline. Figure 23 shows the corresponding distribution for the Laplace-free DL-ROM baseline of Section 3.2.2.

Optimised contours. Figure 24 shows the end-to-end error distributions of the six optimised-contour configurations of Section 3.5, as a function of K .

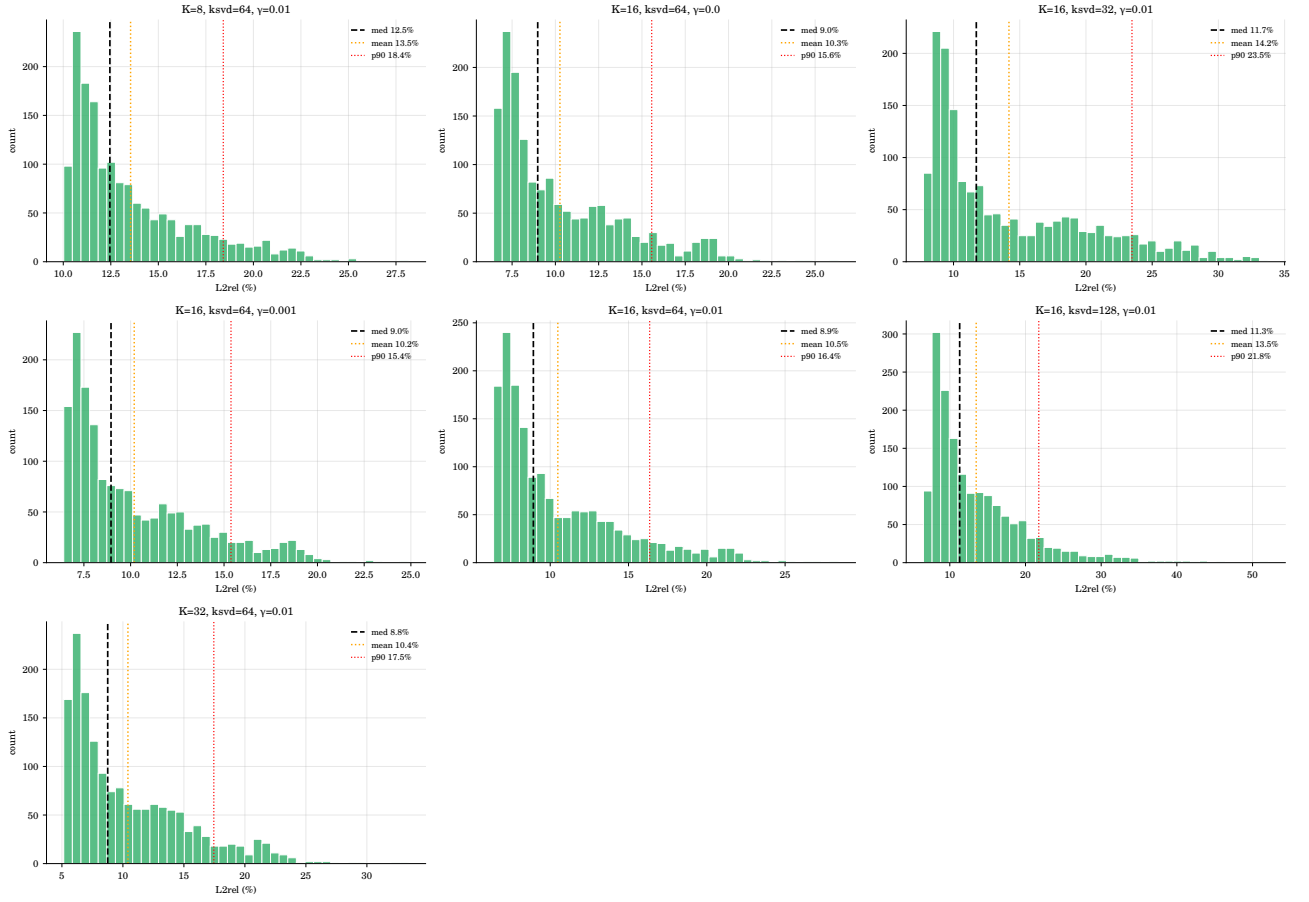


Figure 20: Per-configuration error histograms for the POD surrogate (test set, $n = 1620$). Same layout as Figure 19; green bars indicate surrogate predictions.

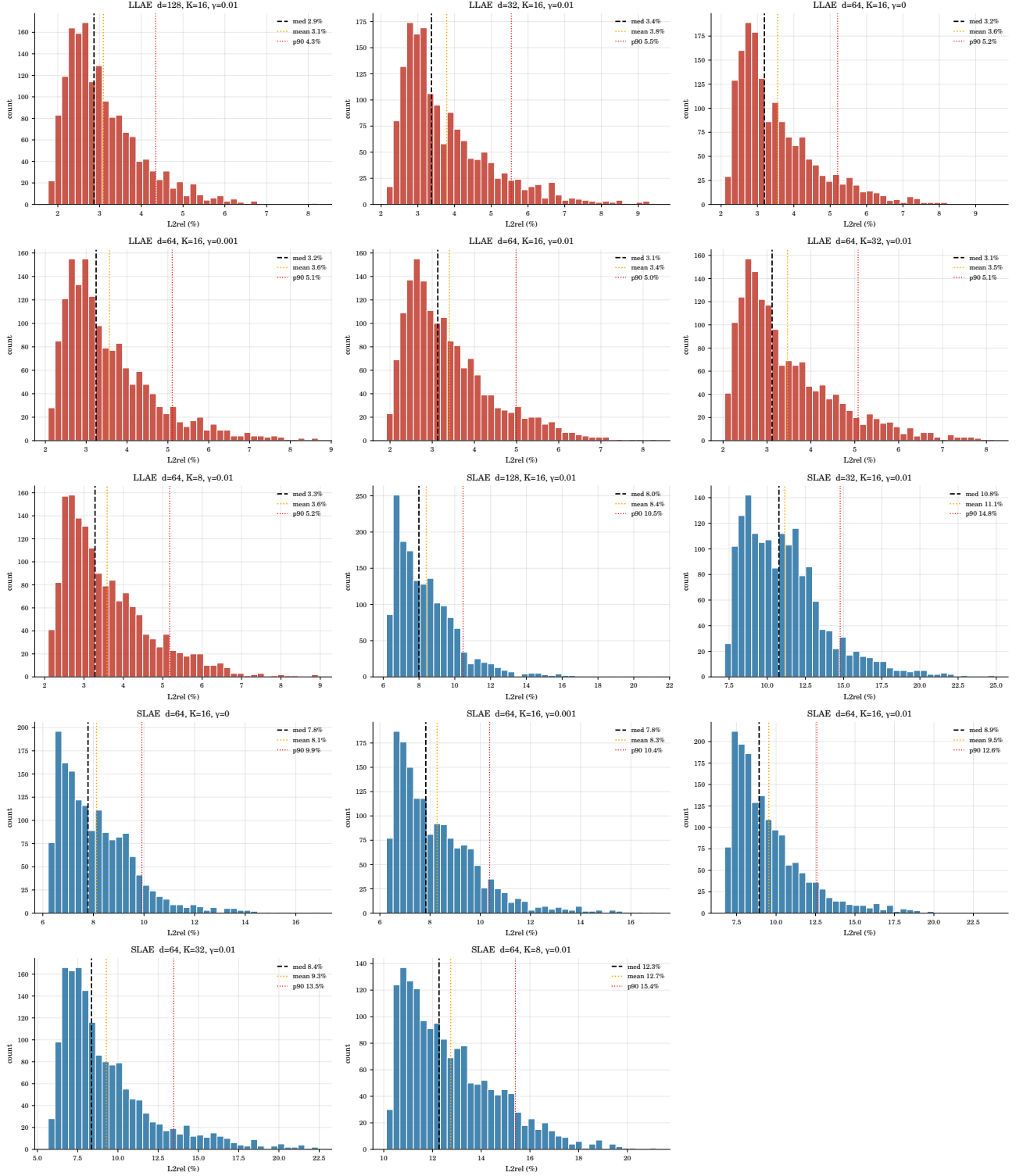


Figure 21: Per-checkpoint error histograms on the test set ($n = 1620$). Each panel corresponds to one checkpoint; SLAE in blue, LLAE in red. The dashed black line is the median, dashed orange the mean, dotted red the 90th percentile.

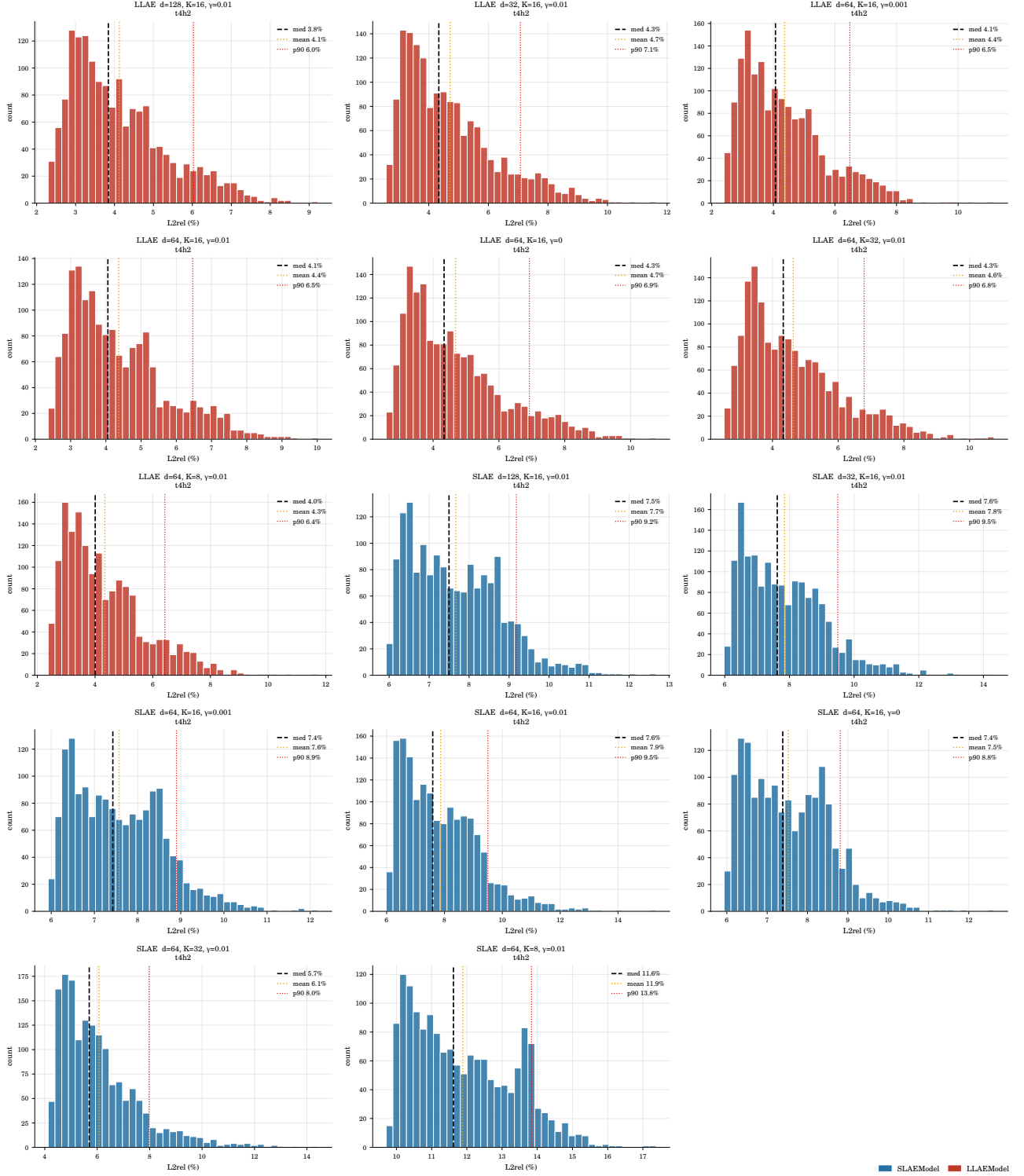


Figure 22: Per-checkpoint error histograms for all 14 direct surrogate models (test set, $n = 1620$). SLAE checkpoints in blue, LLAE checkpoints in red. The dashed black line is the median, dashed orange the mean, dotted red the 90th percentile.

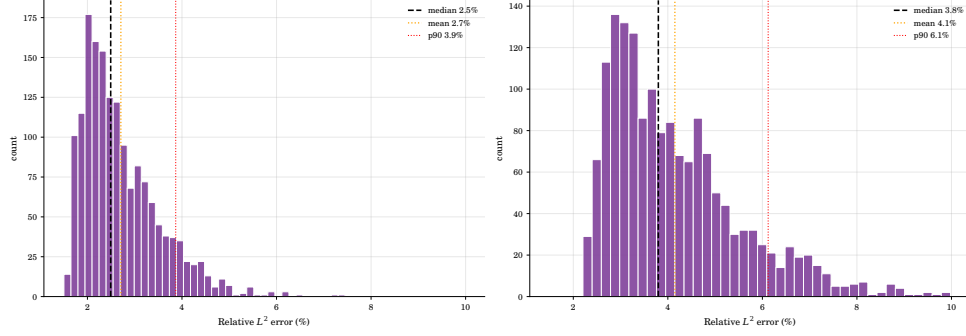


Figure 23: DL-ROM baseline at full temporal resolution ($N_t = 150$, $d_z = 64$): distribution of the relative L^2 error on the physical field $U(t)$ over the test set ($n = 1620$). Median 3.80%, mean 4.15%, p90 6.12%.

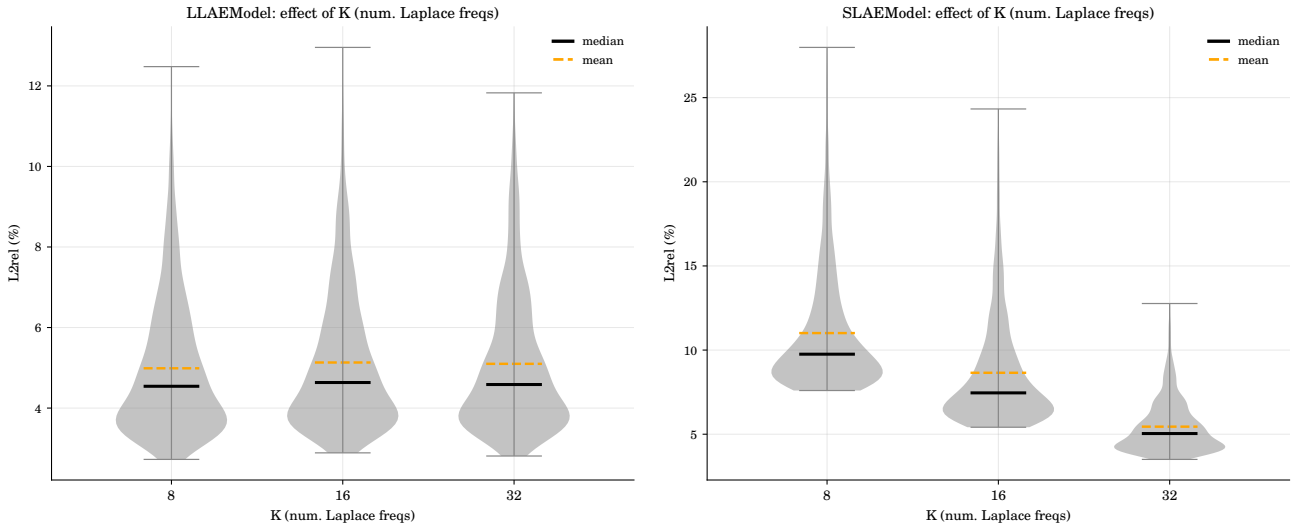


Figure 24: End-to-end error distributions under the optimised contour, as a function of the number of Laplace frequencies K ($d_z = 64$, test set, $n = 1620$). Left: LLAEM, latent learnable contour. Right: SLAEM, offline spatial contour. Black line: median; dashed orange line: mean.

Tucker surrogate checkpoints. Figure 25 shows the per-sample error distribution for each of the 12 Tucker surrogate checkpoints evaluated in Section 3.4.3.

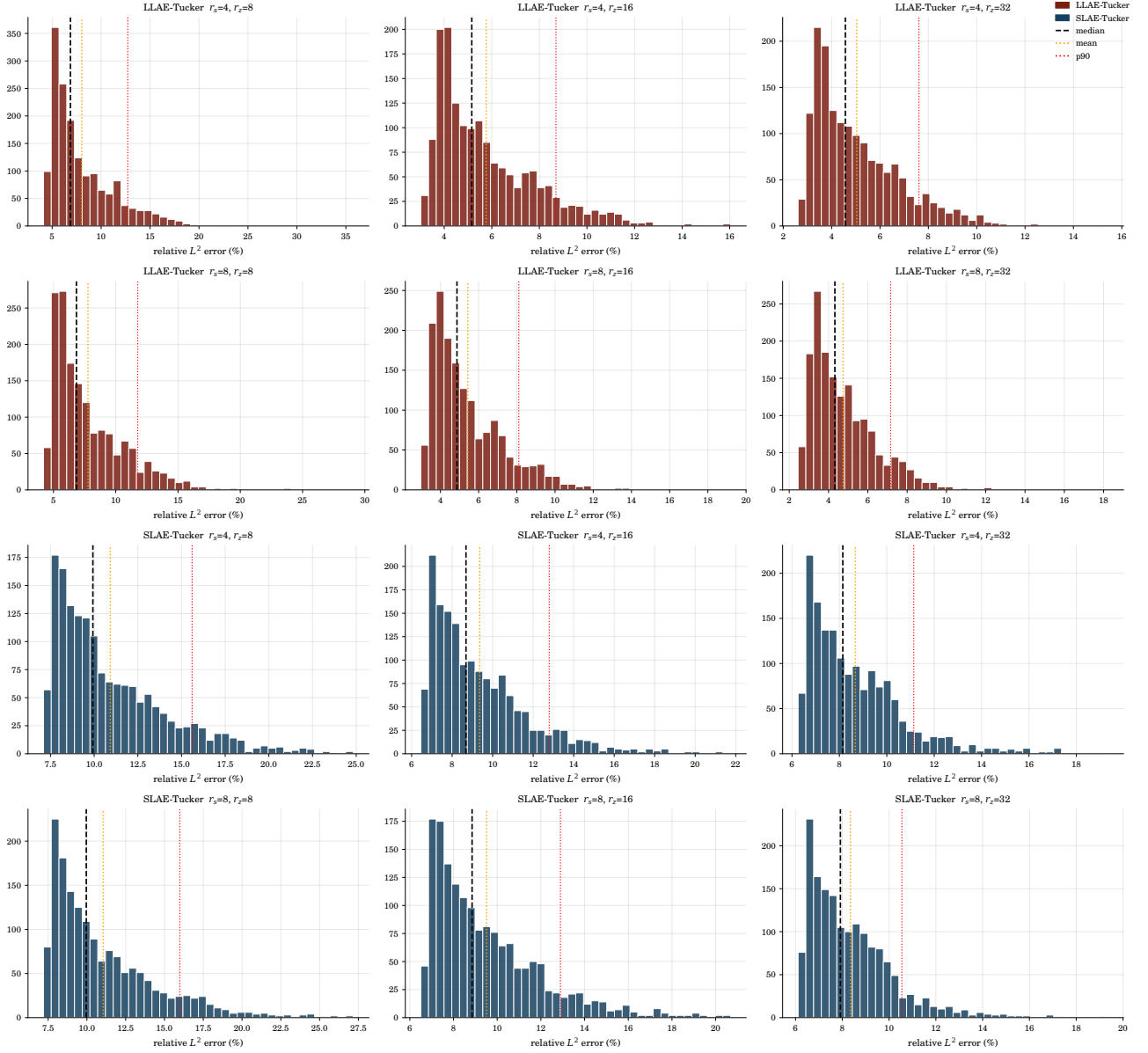


Figure 25: Per-checkpoint error histograms for all 12 Tucker surrogate models (test set, $n = 1620$). SLAE-Tucker in blue, LLAE-Tucker in red. The dashed black line is the median, dashed orange the mean, dotted red the 90th percentile.

B POD Baseline Ablations

This appendix collects the complete hyperparameter sweeps behind Section 3.2.1, for both the pure POD reconstruction and the end-to-end POD surrogate.

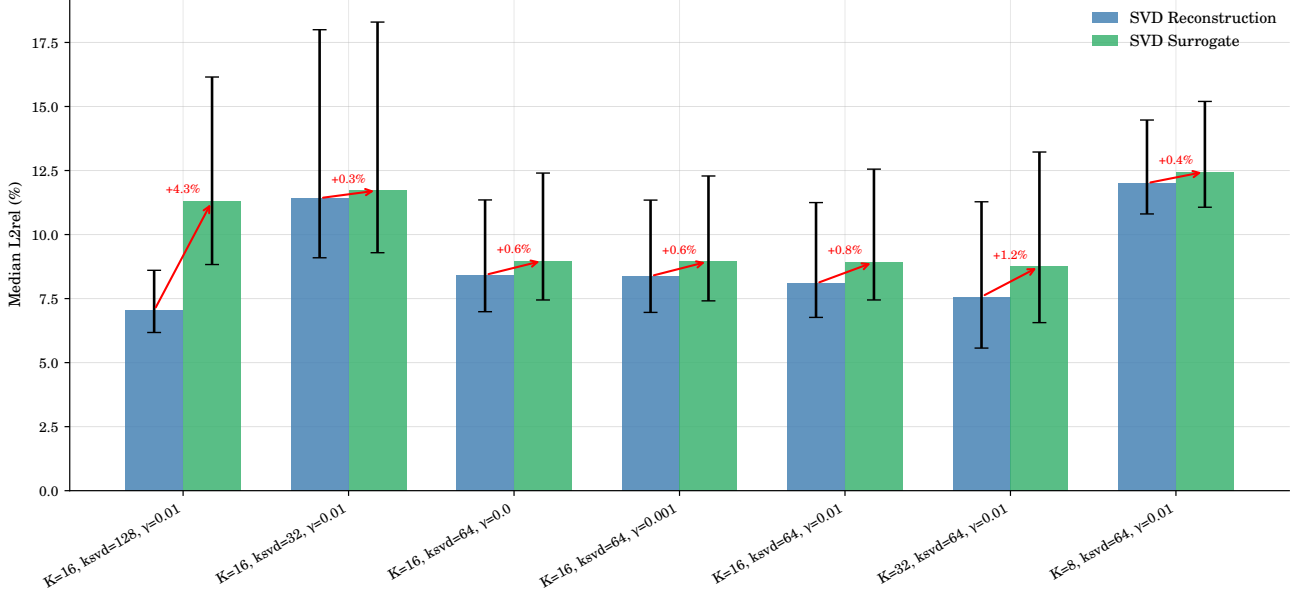


Figure 26: POD reconstruction (blue) vs. end-to-end surrogate (green): median relative L^2 error for all configurations of Table 2 (median $\pm$ IQR). Arrows indicate the surrogate overhead.

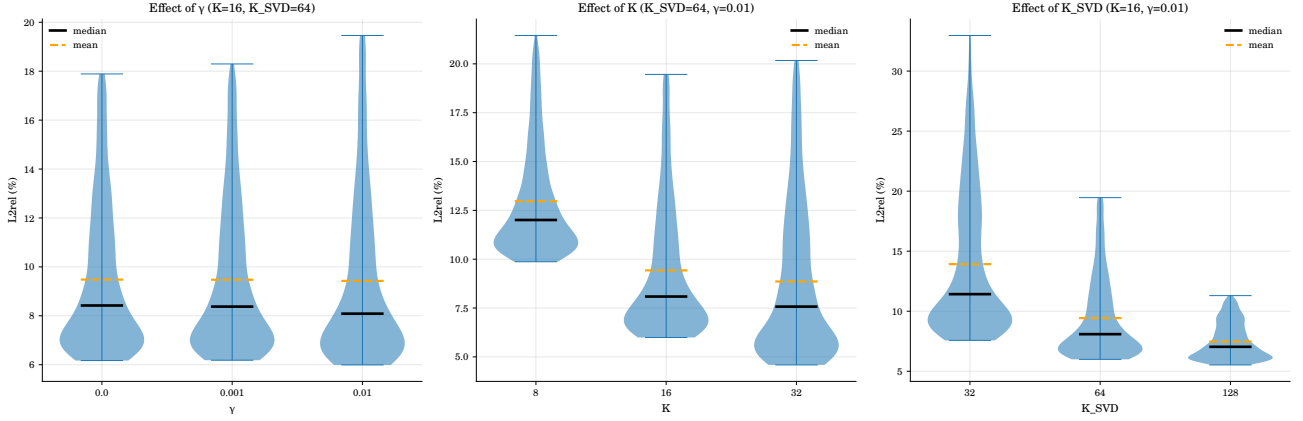


Figure 27: POD reconstruction ablation (test set, median $\pm$ IQR, violin plots; black line = median, dashed orange = mean). From left to right: effect of the damping parameter γ ($K = 16, k_{\text{pod}} = 64$), effect of the number of Laplace frequencies K ($k_{\text{pod}} = 64, \gamma = 0.01$), and effect of the number of POD modes k_{pod} ($K = 16, \gamma = 0.01$).

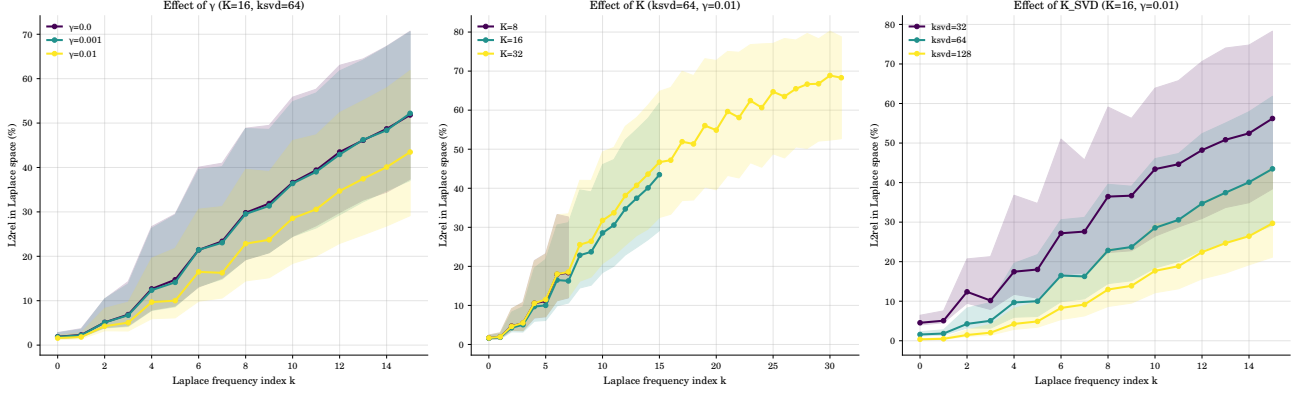


Figure 28: POD reconstruction: per-frequency Laplace-domain error (median $\pm$ IQR, test set). Left: effect of γ ; centre: effect of K ; right: effect of k_{pod} .

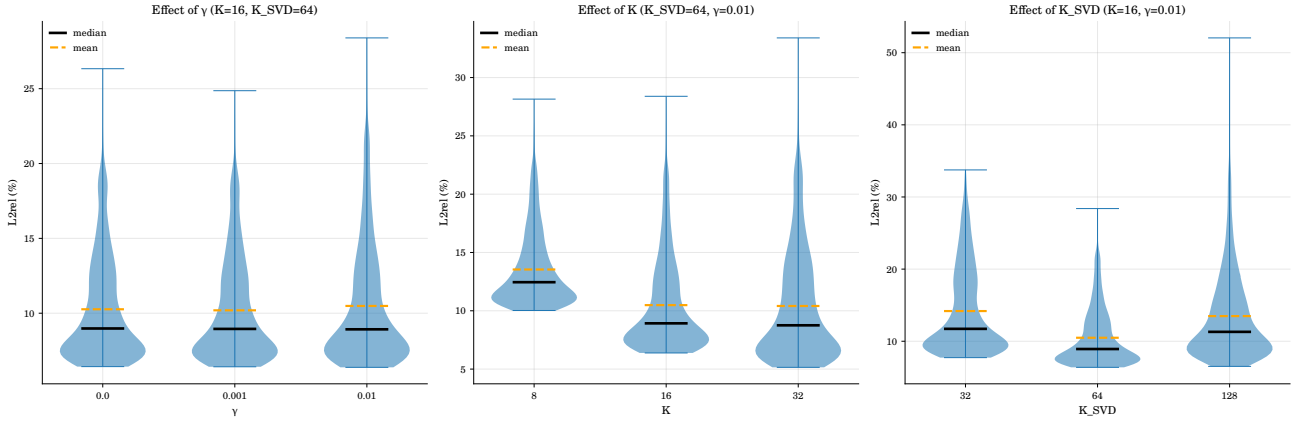


Figure 29: POD surrogate ablation (test set, median $\pm$ IQR). From left to right: effect of γ ($K = 16$, $k_{pod} = 64$), effect of K ($k_{pod} = 64$, $\gamma = 0.01$), and effect of k_{pod} ($K = 16$, $\gamma = 0.01$). Black line = median, dashed orange = mean.

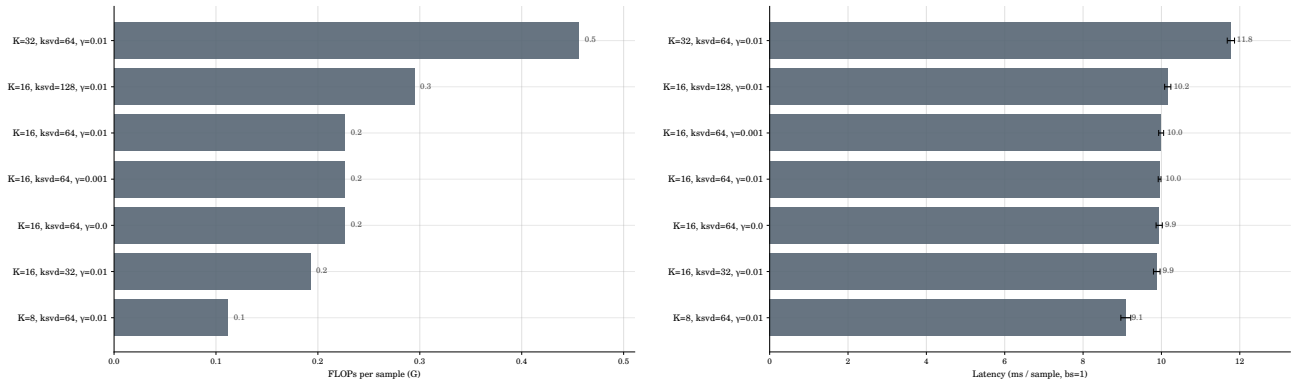


Figure 30: POD surrogate inference benchmark (50 timed CUDA Event runs, 10 warm-up runs). Left: per-sample latency at batch size 1, sorted by latency. Right: throughput (samples/s) vs. k_{pod} at batch size 32.

C Autoencoder Ablations and Reconstruction Cost

Figures 31 and 32 show the effect of each hyperparameter on the reconstruction error of Section 3.3, varying one at a time while holding the others fixed. Figure 33 gives the corresponding inference cost across all 14 checkpoints, of which Table 5 lists 10.

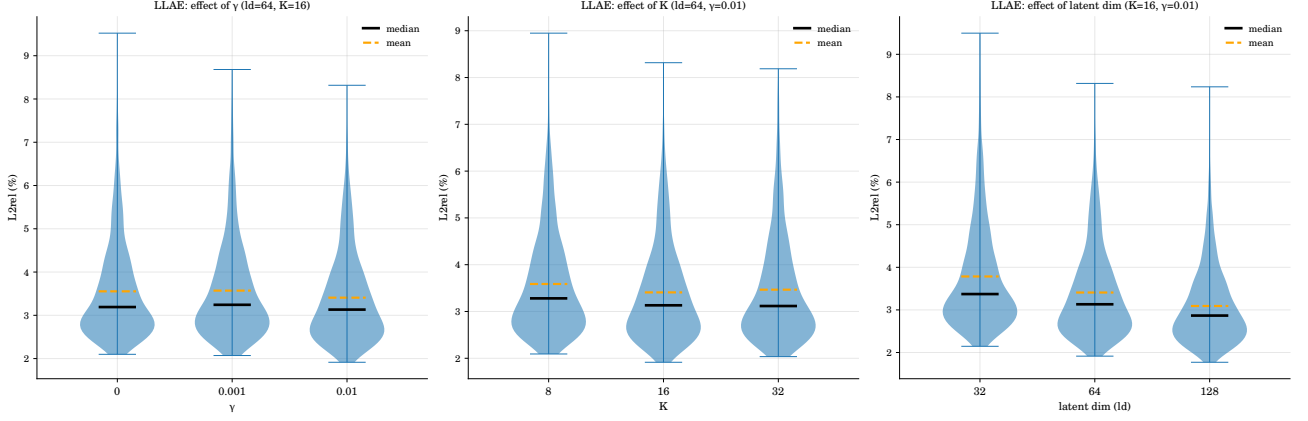


Figure 31: LLAE ablation study. From left to right: effect of latent dimension d_z ($K = 16$, $\gamma = 0.01$), number of Laplace frequencies K ($d_z = 64$, $\gamma = 0.01$), and damping parameter γ ($d_z = 64$, $K = 16$). Violin plots show the full error distribution; black line is the median, dashed orange line is the mean.

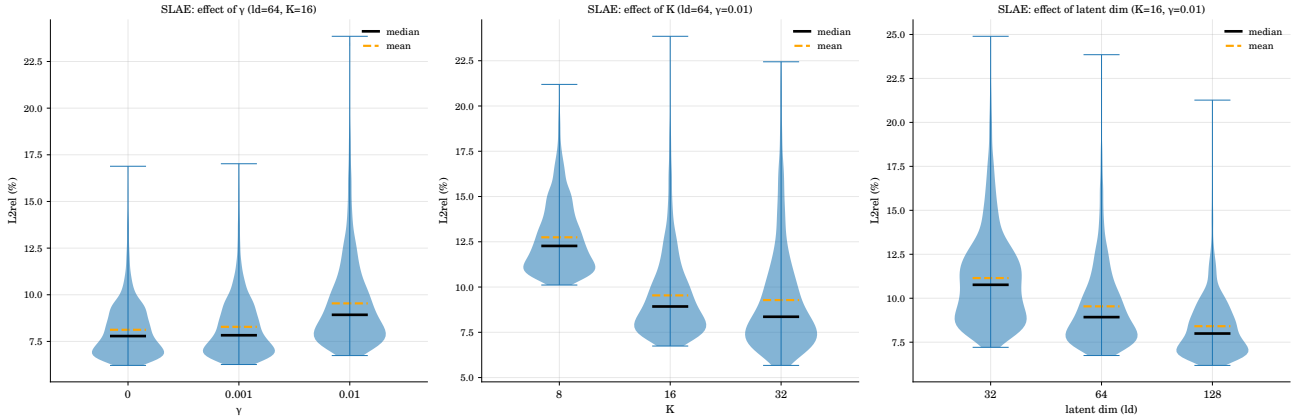


Figure 32: SLAE ablation study. Same layout as Figure 31 (fixed parameters: $K = 16$, $\gamma = 0.01$ / $d_z = 64$, $\gamma = 0.01$ / $d_z = 64$, $K = 16$). Note the larger y-axis range compared to LLAE.

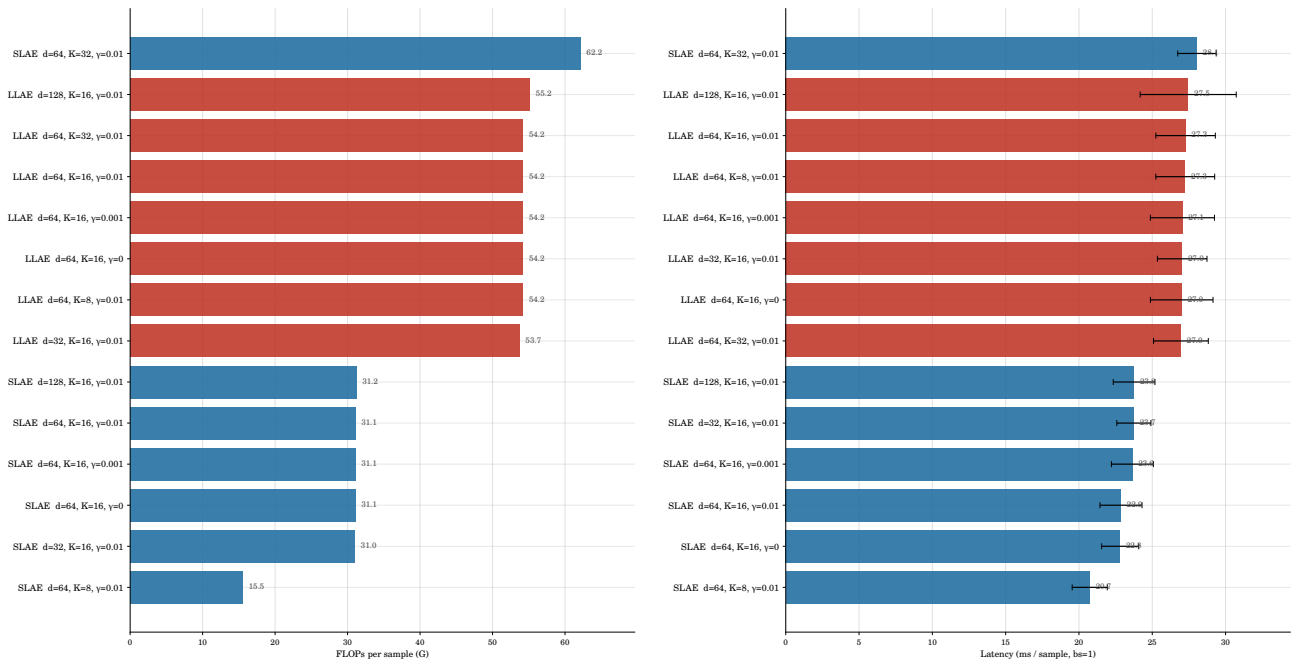


Figure 33: Reconstruction cost for all 14 AE checkpoints (batch size 1). Left: arithmetic work per sample. Right: wall-clock latency. LLAE in red, SLAE in blue.

D Surrogate Ablations

Model	κ	Output steps	Median (%)	N_{surr}	Lat. (ms)	FLOPs (G)
DL-ROM	1	150	3.80	23.50 M	31.4	48.3
DL-ROM	2	75	3.97	12.15 M	15.8	24.1
DL-ROM	3	50	3.84	8.36 M	10.6	16.1
DL-ROM	5	30	3.86	5.33 M	6.7	9.7
DL-ROM	10	15	3.73	3.06 M	3.8	4.8
LLAE ($K = 16$)	—	≥ 30	4.06	3.48 M	Fig. 18	4.8–48.2
SLAE ($K = 16$)	—	≥ 30	7.59	3.22 M	Fig. 18	30.2–30.3

Table 12: Temporal-resolution sweep of the DL-ROM against the two Laplace pipelines. Latencies come from the same benchmarking run as Table 11. Each DL-ROM row is a separately trained model; the two Laplace rows are a *single* checkpoint each, queried at output resolutions $N_t \in \{15, 30, 50, 75, 150\}$; their ≥ 30 is the sampling floor (34) at $K = 16$, below which no output grid is serviceable, the FLOPs range spanning that interval and the corresponding latency profile being given in Figure 18. Errors are measured on each model’s own output grid (the DL-ROM at stride κ against the ground truth subsampled at the same stride), with no interpolation. The two Laplace medians are those measured at the 150-step output; accuracy at coarser grids is reported in Table 13.

Model	Contour	$N_t = 150$	75	50	30	15
SLAE	prescribed	7.59	7.66	7.75	8.83	61.4
	offline	7.46	7.52	9.44	93.2	632
LLAE	prescribed	4.06	4.12	4.03	4.36	49.0
	learnable	4.63	4.74	4.79	5.93	84.5

Table 13: Median relative L^2 error (%) against requested output resolution, at fixed horizon T , test set ($n = 1620$). A single $K = 16$ checkpoint per row is queried at every resolution, with the Tikhonov terms rescaled to the output grid by (32). Ground truth is the dataset subsampled on the same grid; no interpolation is involved. $N_t = 15$ lies below the sampling floor (33) and is reported to locate it. Best per column within each family in bold.

Figures 34 and 35 show the effect of each hyperparameter on the end-to-end surrogate error, using violin plots of the full test-set distribution (one hyperparameter varied at a time, others fixed at the baseline values of Table 14).

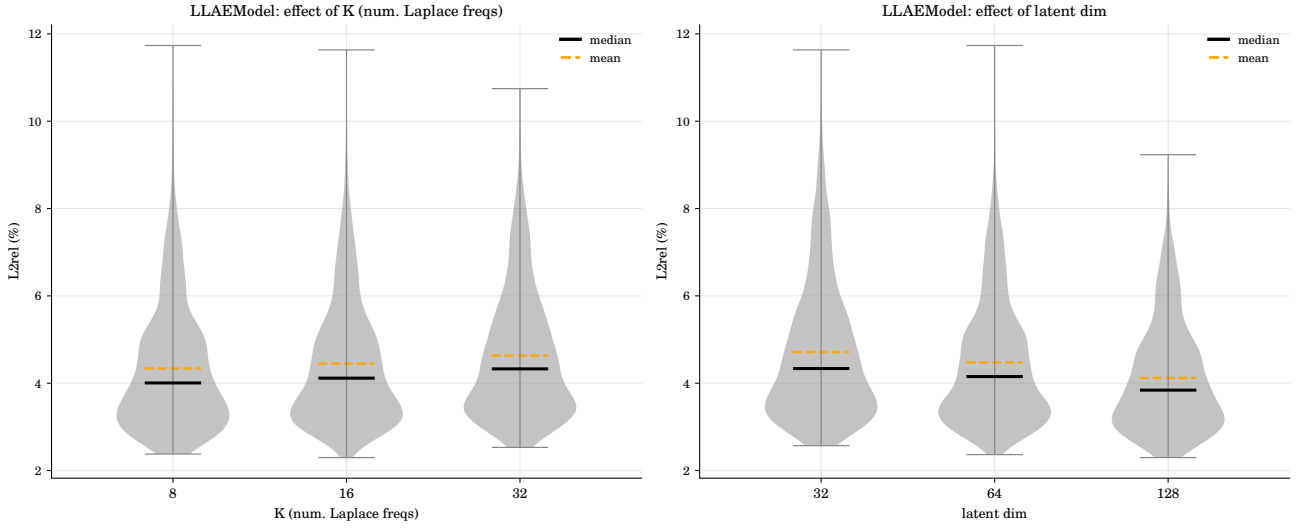


Figure 34: LLAEM surrogate ablation (test set, $n = 1620$, violin plots; black line = median, dashed orange = mean). From left to right: effect of the number of Laplace frequencies K ($d_z = 64$, $\gamma = 0.01$) and effect of the latent dimension d_z ($K = 16$, $\gamma = 0.01$).

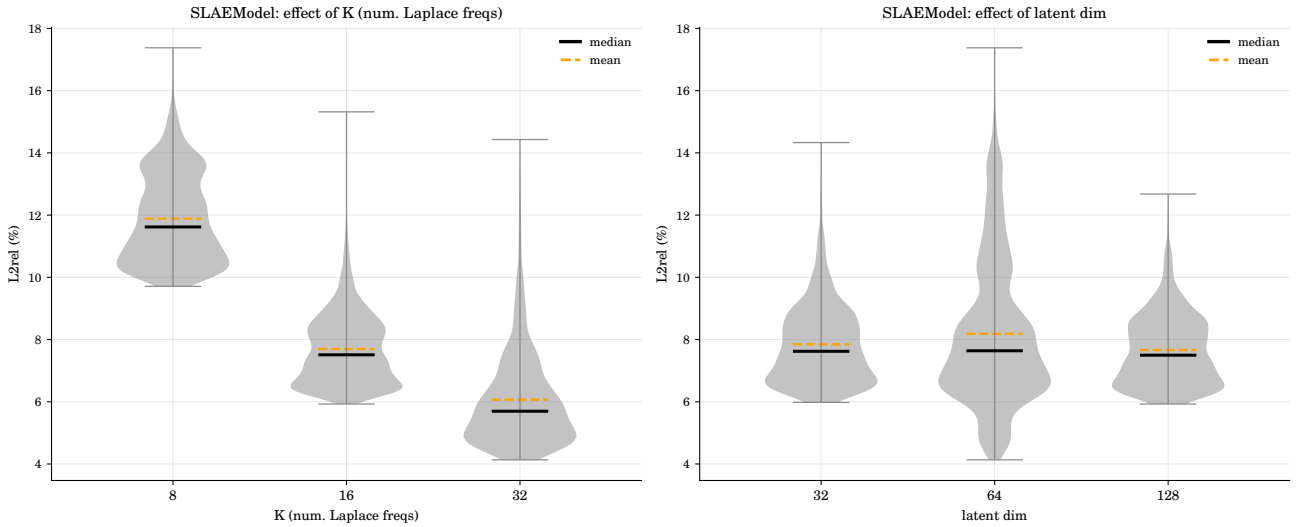


Figure 35: SLAEM surrogate ablation (test set, $n = 1620$). Same layout as Figure 34. Note the wider y-axis range.

E Optimised Contours at the Reconstruction Stage

Section 3.5 assesses the two contour-selection strategies at the surrogate stage, which is the deliverable. This appendix reports their behaviour at the reconstruction stage, where the two criteria diverge most visibly. Figure 37 summarises the six configurations.

The latent learnable contour is what contour optimisation amounts to for LLAE, whose poles are differentiable end to end and call for no offline criterion (Section 2.4). At the reconstruction stage it works, and well. The most accurate LLAE autoencoder of the study is obtained at $K = 32$ (2.60% median error), and even $K = 8$ (2.89%) improves upon every prescribed-contour instance of Table 4, including the one at $d_z = 128$ (2.87%): at this stage, optimising the contour is worth more than doubling the latent dimension, at no additional inference cost since the sampling points enter only the transform. The residual sensitivity to K all but vanishes (2.60–2.89% over a fourfold variation), consistent with the geometry of Figure 36, where the sampling points are redistributed towards the spectral region carrying the signal energy; the gain is measured on the held-out set, so nothing here is fitting noise.

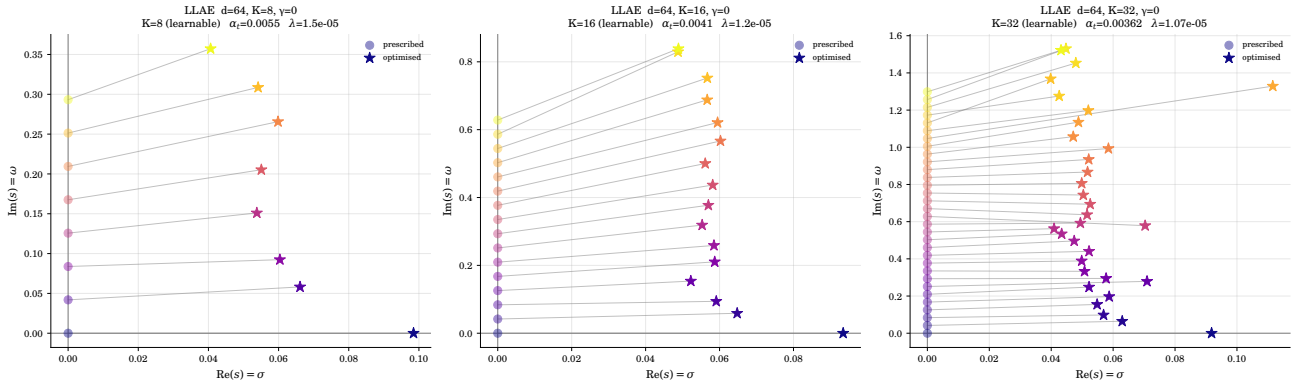


Figure 36: LLAE latent learnable contours in the complex plane ($d_z = 64$, $K \in \{8, 16, 32\}$). Same conventions as Figure 15: circles are the prescribed sampling points, stars the learned ones, joined by a grey segment; colour encodes the frequency index k .

We did not succeed in turning that autoencoder into a better surrogate: the deployed columns of Table 10 run the other way. We draw no further conclusion from the comparison, the two strategies differing in more than their criterion, the learnable poles continuing to be refined during Phase 2 while the offline triplet is frozen after selection. Two features of the learnable route bound it further: the Phase-2 learning rate on the poles is 10^{-5} against 3×10^{-4} for the surrogate (Appendix G), and applies equally to (α_t, λ) , which are parameterised logarithmically whereas σ_k and ω_k are not, so a common step size displaces them unequally; and no box constraint is imposed, so ω_k may cross zero, whereupon the pole leaves the conjugate-symmetric set (2), K_{full} drops by one and the inversion changes discontinuously during training. Setting that rate to zero and to the surrogate’s would bound the first effect.

The offline spatial contour degrades the reconstruction instead, by +0.65 and +0.17 pp at $K = 8$ and $K = 32$ and by +7.12 pp at $K = 16$. The penalty is analysed in Section 3.5: it is incurred at a coefficient-error magnitude the criterion never scored, and is subsequently absorbed during Phase 2, whose objective is formulated after the inversion rather than before it.

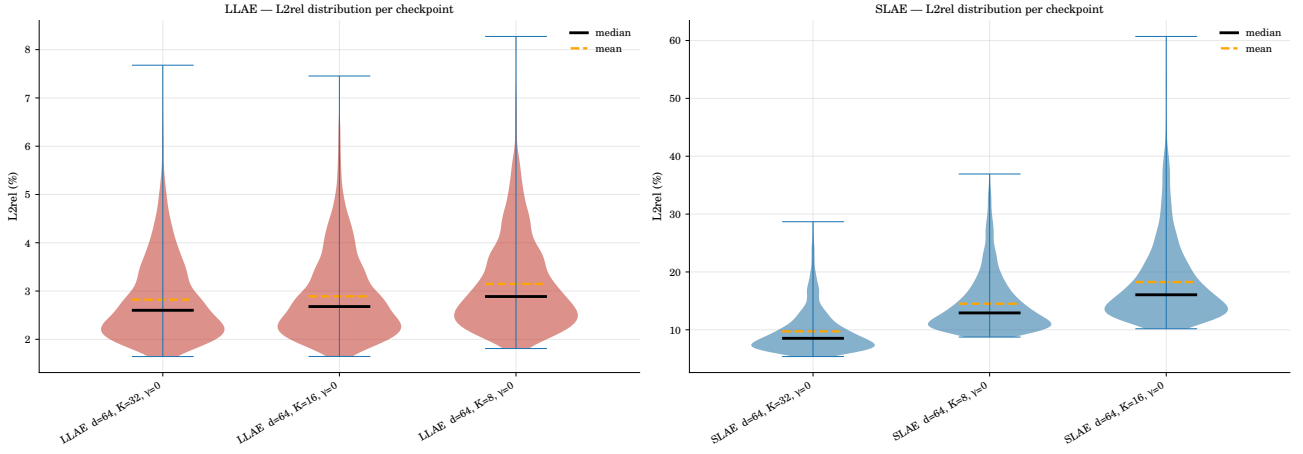


Figure 37: Standalone reconstruction error under the optimised contours: distribution of the relative L^2 error on the test set ($n = 1620$), one panel per family, configurations ordered by median. Black line: median; dashed orange line: mean. Left: LLAE, latent learnable contour. Right: SLAE, offline spatial contour, on a y-axis an order of magnitude wider.

F Governing Equations

Our neural surrogate model is designed to emulate the transient dynamics of a reacting flow system. The physical system is governed by a set of coupled partial differential equations representing the conservation of mass, momentum, energy, and chemical species.

1. Fluid Mechanics (Momentum):

$$\frac{\partial(\rho \mathbf{U})}{\partial t} + \nabla \cdot (\rho \mathbf{U} \mathbf{U}) - \nabla \cdot \boldsymbol{\tau} = -\nabla p + \rho \mathbf{g} \quad (45)$$

2. Chemical Species Transport:

$$\frac{\partial(\rho Y_i)}{\partial t} + \nabla \cdot (\rho \mathbf{U} Y_i) - \nabla \cdot (\mu_{\text{eff}} \nabla Y_i) = \dot{\omega}_{i,\text{eff}} \quad (46)$$

under the constraint $\sum Y_i = 1$.

3. Energy:

$$\frac{\partial(\rho h)}{\partial t} + \nabla \cdot (\rho \mathbf{U} h) + \frac{\partial(\rho K)}{\partial t} + \nabla \cdot (\rho \mathbf{U} K) - \frac{\partial p}{\partial t} - \nabla \cdot (\alpha_{\text{eff}} \nabla h) = \dot{q}_{\text{chem}} \quad (47)$$

where $K = \frac{1}{2}|\mathbf{U}|^2$ is the kinetic energy.

4. Combustion Model (PaSR): The effective reaction rate is modulated by the Partially Stirred Reactor model:

$$\dot{\omega}_{i,\text{eff}} = \kappa \dot{\omega}_i(c^*, T^*), \quad \text{with} \quad \kappa = \frac{\tau_c}{\tau_c + \tau_{\text{mix}}} \quad (48)$$

where τ_c and τ_{mix} are the characteristic chemical and turbulent mixing times, respectively.

5. Equation of State: The ideal gas law closes the system:

$$p = \rho RT \quad \text{or} \quad \rho = \psi p \quad \text{with} \quad \psi = \frac{1}{RT} \quad (49)$$

G Architectural Details and Setup

Hardware and software. Every model of this study was trained, and every cost figure measured, on identically configured workstations of the same pool: an Intel Xeon W-1270P CPU (8 cores, 16 threads) with 62 GB of RAM and a single NVIDIA RTX A5000 GPU (24 GB). The software stack is Python 3.13, PyTorch 2.12 (CUDA 13.0) and PyTorch Lightning 2.6. All inference-cost figures reported in this paper (latency, FLOPs, peak GPU memory and energy) were collected on one such idle machine, at batch size 1 unless stated otherwise, with 50 timed CUDA Event runs after 10 warm-up runs; the device is not named again in the individual captions.

Optimisation protocol. Both phases share the same protocol, and it is the one written in the configuration files distributed with the code; the only departures are the sweep overrides named in the tables below. Optimiser AdamW with weight decay 10^{-4} ; learning-rate schedule `ReduceLROnPlateau` (factor 0.5, patience 15, floor 10^{-6}) driven by the monitored validation quantity; mixed 16-bit precision; gradient-norm clipping at 1.0; at most 300 epochs with early stopping at patience 40. The monitored quantity is the validation loss for the autoencoders and the validation relative L^2 error for the surrogates, and it also selects the checkpoint that is subsequently evaluated. The global seed is 42 for every run.

Data. The partition is the one described in Section 3.1: 5,184 training, 1,296 validation and 1,620 test instances, drawn by a fixed permutation of seed 42 and identical across every model, phase and hyperparameter configuration of this study. No test instance enters any training or selection decision at any point. Fields are standardised pixel-wise using training-set statistics; for SLAE the Laplace frames are standardised per frequency and per pixel, and the compressed targets of the SVD and Tucker variants are standardised as well, so all losses below are evaluated in normalised space. The forward quadrature uses the trapezoidal rule on the $N_t = 150$ nodes of the $\Delta t = 1$ s grid.

Offline contour minimisation. The SLAE contours of Section 3.5 are obtained by 500 AdamW steps on the proxy (30) (learning rate 5×10^{-3} on plateau decay, weight decay 10^{-4} , gradient clipping at 1.0, double precision), with $\lambda_{\text{AE}} = 1.25$, $\lambda_{\Delta t} = \lambda_{\Delta x} = 0.5$ and rank $d_z = 64$ for the SVD residual, both terms being evaluated over the same $N_{\text{opt}} = 100$ training instances sampled at a single rotation angle. The poles are initialised on the prescribed Bromwich contour at $\gamma = 0$ and (α_t, λ) at 10^{-3} ; all four are optimised jointly under the box constraints $\sigma_k \geq -0.05$, $\omega_k \geq 0$ and $\alpha_t, \lambda \in [10^{-6}, 1]$.

Table 14 summarises the autoencoders (Phase 1) and Table 15 the surrogates (Phase 2).

Group	Hyperparameter	SLAE	LLAE
Architecture (baseline [†])	Latent dim d_z	64	64
	Frequencies K	16	16
	Contour γ	0.01	0.01
	Conditioning coord.	$k/(K-1)$	$t/(N_t-1)$
	Sinusoidal levels L	8	8
Backbone	Grid $N \times N$	128^2	128^2
	Input channels	2 (Re, Im)	1
	Encoder channels	$16 \rightarrow 32 \rightarrow 64$, then FC $\rightarrow 2d_z \rightarrow d_z$	
	Decoder channels	$128 \rightarrow 128 \rightarrow 64 \rightarrow 32$	$64 \rightarrow 64 \rightarrow 32 \rightarrow 16$
	Decoder norm	BatchNorm	GroupNorm(8)
	Decoder output heads	2 (Re, Im)	1
Regularization	Ridge β on z	10^{-2}	10^{-2}
	Latent roundtrip β_{latent}	—	0.1
	Tikhonov α_t	7×10^{-3} (eval)	7×10^{-3}
	Tikhonov λ	3×10^{-5} (eval)	3×10^{-5}
Training	Epochs (max)	300	300
	Batch size	$16^\ddagger$	16
	Learning rate	5×10^{-4}	5×10^{-4}
	Weight decay	10^{-4}	10^{-4}
	Early stopping patience	40	40
	Monitored quantity	validation reconstruction loss	

Table 14: Autoencoder hyperparameters (Phase 1), as set in `configs/model` and `configs/training/ae.yaml`. FiLM modulation is $x \mapsto x(1 + \tanh \alpha) + \delta$ at every convolutional stage, the $\tanh$ bounding the gain in $(0, 2)$; the last convolution of each decoder head is zero-initialised. [†]Baseline values held fixed when the other architectural parameters are swept: $d_z \in \{32, 64, 128\}$, $K \in \{8, 16, 32\}$, $\gamma \in \{0, 0.001, 0.01\}$. [‡]The batching unit differs between families: a batch of 16 groups 16 (simulation, frequency) pairs for SLAE, which encodes frequency frames independently, against 16 complete simulations for LLAE, which encodes the whole trajectory.

Group	Hyperparameter	SLAE	LLAE
Architecture (fixed)	Trunk layers n_{trunk}	4	4
	Trunk hidden dim	512	512
	Head layers n_{head}	2	2
	Head hidden dim	256	256
	Frequency encoding levels	$L = 6$, raw sin/cos (no MLP)	
Loss	Latent weight α_{lat}	1.0	1.0
	Spatial weight α_{spat}	1.0	1.0
	Tikhonov α_t	7×10^{-3}	7×10^{-3}
	Tikhonov λ	3×10^{-5}	3×10^{-5}
Trainable parts	Encoder	frozen	
	Surrogate MLP	3×10^{-4} direct, 5×10^{-4} SVD/Tucker	
	Decoder (fine-tuned)	$lr = 5 \times 10^{-5}$	
	Poles (s_k, α_t, λ)	frozen	10^{-5} if learnable
Compression variants	SVD rank k_{svd}	$\{8, 16, 32\}$, basis frozen	
	Tucker ranks (r_s, r_z)	$\{4, 8, 16\} \times \{8, 16, 32\}$, frozen	
	HOOI iterations	5	
Training	Epochs (max)	300	300
	Batch size	16	16
	Weight decay	10^{-4}	10^{-4}
	Early stopping patience	40	40
	Monitored quantity	validation relative L^2 error	

Table 15: Surrogate hyperparameters (Phase 2), as set in `configs/training/surrogate_*.yaml`. One surrogate is trained per autoencoder checkpoint, inheriting its d_z , K and contour. The encoder is frozen throughout; the decoder is fine-tuned at a reduced learning rate, and the SVD basis and Tucker factors, computed offline, are held fixed. The DL-ROM baseline uses this same table with $N_t = 150$ per-time-step heads in place of the K frequency heads.

H Derivations for the Bias–Amplification Contour Optimisation

Throughout this appendix, $\mathbf{F} \in \mathbb{C}^{K_{\text{full}} \times N_t}$ and the coefficient vectors are those assembled on the conjugate-symmetric set $\mathcal{S}$ of (2), as in Sections 2.1 and 2.4; on $\mathcal{S}$ the matrix $\mathbf{F}^H \mathbf{F}$ and the vector $\mathbf{F}^H \hat{\mathbf{u}}$ are real, so the operator $\Re(\cdot)$ below may be read as an identity and is kept only for uniformity with (4). Two conventions are in force and are used by every statement below. First, the exact coefficient vector is the *discrete* one,

$$\hat{\mathbf{u}}(\boldsymbol{\mu}) := \mathbf{F} \mathbf{u}(\boldsymbol{\mu}), \quad (50)$$

that is, the quadrature (1) evaluated on $\mathcal{S}$ and not the continuous transform $\int_0^T u e^{-st} dt$ it approximates; the quadrature error is thereby excluded from the decomposition rather than assigned to one of its terms, and this is what the implementation computes. Under (50) the conjugate symmetry $\hat{u}(\bar{s}_k) = \overline{\hat{u}(s_k)}$ holds exactly, since $\mathbf{F}_{\bar{k},n} = \overline{\mathbf{F}_{k,n}}$ and $\mathbf{u}$ is real. Second, $\alpha_t \geq 0$ and $\lambda > 0$, as enforced by the box constraints of Section 2.4.

Proposition H.1 (Invertibility of $\mathbf{A}$). *Let $\alpha_t \geq 0$ and $\lambda > 0$. Since $\mathbf{D}_t^T \mathbf{D}_t$ is positive semi-definite, $\mathbf{L}_{\text{reg}} = \alpha_t \mathbf{D}_t^T \mathbf{D}_t + \lambda \mathbf{I}$ is positive definite ($\mathbf{L}_{\text{reg}} \succeq \lambda \mathbf{I}$). Moreover $\Re(\mathbf{F}^H \mathbf{F})$ is symmetric, $\mathbf{F}^H \mathbf{F}$ being Hermitian, and positive semi-definite: for real $\mathbf{x}$, $\mathbf{x}^T \Re(\mathbf{F}^H \mathbf{F}) \mathbf{x} = \Re(\mathbf{x}^H \mathbf{F}^H \mathbf{F} \mathbf{x}) = \|\mathbf{F} \mathbf{x}\|^2 \geq 0$. Thus $\mathbf{A} = \Re(\mathbf{F}^H \mathbf{F}) + \mathbf{L}_{\text{reg}}$ is symmetric positive definite and hence invertible, and the objective (3) is strictly convex on $\mathbb{R}^{N_t}$. Its unique minimiser is the solution of (4), $\mathbf{u}^*(\boldsymbol{\mu}) = \mathbf{A}^{-1} \Re(\mathbf{F}^H \hat{\mathbf{u}}(\boldsymbol{\mu}))$.*

Derivation of the bound (25). Starting from decomposition (23) and applying the triangle inequality:

$$\|\mathbf{u}(\boldsymbol{\mu}) - \mathbf{u}_{\text{rec}}(\boldsymbol{\mu})\| \leq \|\mathbf{u}(\boldsymbol{\mu}) - \mathbf{u}^*(\boldsymbol{\mu})\| + \|\mathbf{u}^*(\boldsymbol{\mu}) - \mathbf{u}_{\text{rec}}(\boldsymbol{\mu})\|.$$

The first term equals $\|\mathbf{A}^{-1} \mathbf{L}_{\text{reg}} \mathbf{u}(\boldsymbol{\mu})\|$ by (24), which uses $\Re(\mathbf{F}^H \hat{\mathbf{u}}) = \Re(\mathbf{F}^H \mathbf{F}) \mathbf{u}$ and hence rests on the convention (50). For the second term, write $\boldsymbol{\delta}(\boldsymbol{\mu}) = \hat{\mathbf{u}}(\boldsymbol{\mu}) - \tilde{\mathbf{u}}(\boldsymbol{\mu}) \in \mathbb{C}^{K_{\text{full}}}$, so that

$$\mathbf{u}^*(\boldsymbol{\mu}) - \mathbf{u}_{\text{rec}}(\boldsymbol{\mu}) = \mathbf{A}^{-1} \Re(\mathbf{F}^H \boldsymbol{\delta}(\boldsymbol{\mu})).$$

Since $\mathbf{A}^{-1}$ is real it commutes with $\Re(\cdot)$, so $\mathbf{A}^{-1} \Re(\mathbf{F}^H \boldsymbol{\delta}) = \Re(\mathbf{A}^{-1} \mathbf{F}^H \boldsymbol{\delta})$. Using $\|\Re(\mathbf{w})\|_2 \leq \|\mathbf{w}\|_2$ for $\mathbf{w} \in \mathbb{C}^{N_t}$, immediate from $\|\mathbf{w}\|_2^2 = \|\Re \mathbf{w}\|_2^2 + \|\Im \mathbf{w}\|_2^2$, and then sub-multiplicativity:

$$\|\mathbf{u}^*(\boldsymbol{\mu}) - \mathbf{u}_{\text{rec}}(\boldsymbol{\mu})\| \leq \|\mathbf{A}^{-1} \mathbf{F}^H \boldsymbol{\delta}(\boldsymbol{\mu})\| \leq \|\mathbf{A}^{-1} \mathbf{F}^H\| \cdot \|\boldsymbol{\delta}(\boldsymbol{\mu})\|_{\mathbb{C}^{K_{\text{full}}}},$$

which gives (25). Commuting $\mathbf{A}^{-1}$ past $\Re(\cdot)$ is what produces the constant $\|\mathbf{A}^{-1} \mathbf{F}^H\|$; bounding $\|\Re(\mathbf{F}^H)\| \leq \|\mathbf{F}^H\|$ first would only give the weaker $\|\mathbf{A}^{-1}\| \|\mathbf{F}^H\|$. $\square$

From one pixel to the full field. The bound (25) is stated at a fixed spatial location, whereas the proxy (30) scores full-field quantities: $\mathbf{e}(\boldsymbol{\mu}) \in \mathbb{R}^{N_x \times N_t}$ in the bias term, and an $\mathcal{E}_{\text{SVD}}$ assembled over all N_x nodes in the variance term. The same constant transfers, because $\mathbf{A}^{-1} \Re(\mathbf{F}^H \cdot)$ acts on each node independently. Writing $\boldsymbol{\delta}_p$ for the coefficient error at node p and $\boldsymbol{\Delta} \in \mathbb{C}^{N_x \times K_{\text{full}}}$ for the stack of these rows,

$$\|\mathbf{U}^*(\boldsymbol{\mu}) - \mathbf{U}_{\text{rec}}(\boldsymbol{\mu})\|_F^2 = \sum_{p=1}^{N_x} \|\mathbf{A}^{-1} \Re(\mathbf{F}^H \boldsymbol{\delta}_p)\|^2 \leq \|\mathbf{A}^{-1} \mathbf{F}^H\|_2^2 \sum_{p=1}^{N_x} \|\boldsymbol{\delta}_p\|^2 = \|\mathbf{A}^{-1} \mathbf{F}^H\|_2^2 \|\boldsymbol{\Delta}\|_F^2,$$

whence the field-level counterpart of (25),

$$\|U(\boldsymbol{\mu}) - U_{\text{rec}}(\boldsymbol{\mu})\|_F \leq \|U(\boldsymbol{\mu}) - U^*(\boldsymbol{\mu})\|_F + \|A^{-1}F^H\|_2 \cdot \|\Lambda(\boldsymbol{\mu})\|_F, \quad (51)$$

which is the form the proxy discretises: the amplification factor is unchanged, and both terms are now Frobenius norms over the same $N_x \times N_t$ field. $\square$

On the norm used in practice. The derivation above uses the operator 2-norm, for which sub-multiplicativity holds. The implementation evaluates the amplification factor as the Frobenius norm of the real $2N_t \times K_{\text{full}}$ stack $[A^{-1}\Re(F^H); A^{-1}\Im(F^H)]$, which is exactly $\|A^{-1}F^H\|_F$. Since $\|\cdot\|_2 \leq \|\cdot\|_F$, substituting it in (25) and in the proxy (30) preserves the inequality; it is preferred because it is a smooth function of every entry, hence better behaved under gradient descent than the largest singular value.

Proposition H.2 (SVD residual as a floor for any parameter-to-coefficient predictor). *Let $\mathcal{P}_{d_z}$ be the class of maps $h : \boldsymbol{\mu} \mapsto \mathbf{D} g(\boldsymbol{\mu})$ with $\mathbf{D} \in \mathbb{R}^{2N_x K \times d_z}$ and $g : \mathbb{R}^p \rightarrow \mathbb{R}^{d_z}$ arbitrary. Then, over the optimisation sample $(\boldsymbol{\mu}_i)_{i \leq N_{\text{opt}}}$,*

$$\min_{h \in \mathcal{P}_{d_z}} \frac{1}{N_{\text{opt}}} \sum_{i=1}^{N_{\text{opt}}} \|\mathbf{a}(\boldsymbol{\mu}_i) - h(\boldsymbol{\mu}_i)\|^2 = \mathcal{E}_{\text{SVD}}^2,$$

the minimum being attained whenever the $\boldsymbol{\mu}_i$ are pairwise distinct.

Derivation. The image of any $h \in \mathcal{P}_{d_z}$ lies in $V = \text{col}(\mathbf{D})$, a subspace of dimension at most d_z . For any such V and any $v_i \in V$, $\|\mathbf{a}(\boldsymbol{\mu}_i) - v_i\| \geq \|\mathbf{a}(\boldsymbol{\mu}_i) - P_V \mathbf{a}(\boldsymbol{\mu}_i)\|$, so the mean squared error is bounded below by $\min_{\dim V \leq d_z} \frac{1}{N_{\text{opt}}} \sum_i \|\mathbf{a}(\boldsymbol{\mu}_i) - P_V \mathbf{a}(\boldsymbol{\mu}_i)\|^2$. Constraining all N_{opt} approximants to a common subspace of dimension at most d_z is exactly constraining the matrix they form to rank at most d_z , so that minimum equals $\mathcal{E}_{\text{SVD}}^2$ by the Eckart–Young theorem [6, 29], with V^* spanned by the d_z leading left singular vectors of U . Equality is attained by taking $\mathbf{D}$ a basis of V^* and g any map sending $\boldsymbol{\mu}_i$ to the coordinates of $P_{V^*} \mathbf{a}(\boldsymbol{\mu}_i)$, extended arbitrarily off the sample. This requires the $\boldsymbol{\mu}_i$ to be pairwise distinct, g being a function of $\boldsymbol{\mu}$ and not of the sample index. Should two instances share a parameter vector but not a field, the residual remains a lower bound and ceases to be attained. $\square$

Scale of the substituted quantity. The bound (25) measures the coefficient error on the conjugate-symmetric set $\mathcal{S}$, whereas $\mathbf{a}(\boldsymbol{\mu}) \in \mathbb{R}^{2N_x K}$ of (28) is assembled on the stored half-spectrum $\{s_k\}_{k=1}^K$ alone. By conjugate symmetry the two differ by a fixed factor, $\|\cdot\|_{\mathbb{C}^{K_{\text{full}}}}^2 = 2\|\cdot\|_{\mathbb{C}^K}^2 - \sum_{k:\omega_k=0} |\cdot|^2$, so replacing ϵ_{surr} by $\mathcal{E}_{\text{SVD}}$ carries a factor of at most $\sqrt{2}$. It is absorbed into λ_{AE} , which is set a priori and not calibrated, and is constant across candidate triplets at fixed K and fixed number of real poles, hence without effect on the ordering the criterion produces. Proposition H.2 is exact for the half-spectrum norm in which $\mathcal{E}_{\text{SVD}}$ is defined.

What the statement does and does not assume. Nothing is assumed of g : no regularity, no trainability, not even measurability, and g may interpolate the sample exactly. The floor is therefore a property of the *contour* and of the bottleneck width, not of any learning procedure, which is what makes $\mathcal{E}_{\text{SVD}}$ usable as a score over candidate triplets. It is the parameter-to-coefficient analogue of the classical fact that a linear autoencoder with a d_z -wide bottleneck has no spurious local minima, its global optimum projecting onto the leading d_z -dimensional principal subspace [2]; the encoder is relaxed here from a linear map of the input to an arbitrary function of $\boldsymbol{\mu}$, which is what removes the learning

procedure from the statement entirely. Conversely the bound is confined to a rank- d_z *linear* decoder reading a single unconditioned basis, with a budget of d_z coefficients for the whole spectrum, whereas SLAE decodes non-linearly, modulates that decoder per frequency through FiLM, and disposes of d_z coefficients at each frequency, Kd_z in total. Weight sharing across frequencies is common to both and is not what separates them; the conditioning and the budget are, and on both counts SLAE is the richer class, so we do not expect the floor to bind it. This is an expectation and not a containment: establishing that $\mathcal{P}_{d_z} \subseteq \text{SLAE's class}$ would require the transposed-convolution decoder to realise an arbitrary $\mathbf{D} \in \mathbb{R}^{2N_x \times d_z}$, which its architectural structure does not allow, so neither class contains the other outright. The per-frequency variant $\sum_k \mathcal{E}_{\text{SVD}}(s_k)^2$ is the natural alternative and the closer analogue of SLAE's budget. It is the floor of the class with an unconditioned but frequency-*specific* decoder $\mathbf{D}_k$, obtained by applying the proposition at each s_k and summing; it was not the one used here, the joint form being retained because it depends on the contour through a single spectrum.

Floors bound classes, not trained networks. A network trained by stochastic gradient descent for a finite number of epochs under a particular loss is not guaranteed to attain any floor: SLAE exceeds the linear reconstruction floor of Section 3.2.1 in four of its seven configurations (up to +0.94 pp). Two effects account for this, both outside the proposition: SLAE is trained on a Laplace-domain objective that is not the physical-field error being reported, so the quantity it minimises is not the quantity the comparison scores, and the attained optimum is a local one. Proposition H.2 is accordingly used in Section 2.4 as a computable, deterministic *score* whose variation with the contour is *assumed* to track that of the surrogate error. It is an assumption and not a corollary, the linear rank of the image and the regularity of $\mu \mapsto \mathbf{a}(\mu)$ being independent properties in general. The scalar λ_{AE} introduced there does not repair this: it fixes the nominal magnitude $\mathcal{E}_{\text{SVD}}/\lambda_{\text{AE}}$ at which the variance term enters (30), and that magnitude is set a priori rather than measured.